

```

In [ ]: # =====
# Multimodal DSAF (DINOv3 + Metadata) with 7-Fold CV  $\approx$  70% Train / 15% Val / 15% Te
# - Concise subgroup reports (Gender, Age buckets, Skin tone)
# - Fairness metrics: WGA, EOD with min-support filtering
# - Pooled (weighted) subgroup overall accuracies across folds
# - MODIFIED: Saves model per fold, adds more metrics, and generates/saves multiple
#   for training history, confusion matrix, ROC curves, and scalar metric summaries
# =====

import os, time, json, warnings
from copy import deepcopy
from collections import defaultdict
import numpy as np
import pandas as pd
from itertools import cycle

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms
from PIL import Image

from sklearn.model_selection import StratifiedKFold, StratifiedShuffleSplit
from sklearn.metrics import (
    classification_report, accuracy_score, cohen_kappa_score,
    matthews_corrcoef, roc_auc_score, confusion_matrix, roc_curve, auc
)
from sklearn.preprocessing import label_binarize

from transformers import AutoModel
from torch.amp import GradScaler, autocast

warnings.filterwarnings('ignore')
torch.backends.cudnn.benchmark = True

# Optional FLOPs (best effort)
try:
    from thop import profile
    HAVE_THOP = True
except Exception:
    HAVE_THOP = False

# Optional plotting libs
try:
    import matplotlib.pyplot as plt
    import seaborn as sns
    HAVE_PLOTTING = True
except ImportError:
    HAVE_PLOTTING = False

# -----
# 1) CONFIG – UPDATE THESE TWO PATHS

```

```

# -----
class Config:
    DATA_DIR = r"D:\AIUB\Neural\Synthetic_Skin_Disease_Dataset" # <-- update to
    ORIGINAL_METADATA_CSV = r"D:\AIUB\Neural\complete_metadata_m.csv" # <-- update

    RESULTS_DIR = "metadata_Multimodal_DINOv3_DSAF_KFold"
    MODEL_NAME = "facebook/dinov3-vitb16-pretrain-lvd1689m"
    IMG_SIZE = 224
    BATCH_SIZE = 128
    LEARNING_RATE = 1e-4
    WEIGHT_DECAY = 2e-5
    EPOCHS = 200
    NUM_CLASSES = 5
    DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
    USE_AMP = True
    NUM_WORKERS = 0
    PATIENCE = 20
    WARMUP_EPOCHS = 10
    LABEL_SMOOTHING = 0.1
    PARTIALLY_UNFREEZE = False # set True to unfreeze last block of DINOv3 (then Lo

    # K-Fold settings for ~70/15/15 overall
    N_SPLITS = 7 # ~1/7 ≈ 14.3% Test per fold (close to 15%)
    TARGET_TEST_FRAC = 0.15 # target overall (for docs)
    TARGET_VAL_FRAC = 0.15 # target overall (for docs)

    # Inside each fold's remainder (~85%), choose Val so overall ≈ 15%
    # val_within_remainder = TARGET_VAL_FRAC / (1 - 1/N_SPLITS)
    VAL_WITHIN_REMAINDER = 0.15 / (1.0 - 1.0/7.0) # ≈ 0.175

    # Metadata gating stabilizers
    META_DROPOUT = 0.3
    GATE_SCALE = 0.5 # 0.25-0.5 recommended

    # Fairness: minimum subgroup size for stable WGA/EOD & pooled tables
    MIN_SUPPORT = 15

config = Config()
os.makedirs(config.RESULTS_DIR, exist_ok=True)
print("Device:", config.DEVICE)
print(f"K-Fold: N_SPLITS={config.N_SPLITS} (~Test {100/config.N_SPLITS:.1f}%), Val_
if not HAVE_PLOTTING:
    print("Warning: matplotlib and/or seaborn not found. Plots will be skipped.")

# -----
# 2) CSV STANDARDIZATION
# -----
def standardize_metadata_columns(df: pd.DataFrame) -> pd.DataFrame:
    """
    Accepts a CSV with columns like:
        filepath, split, label, age, dominant_gender, dominant_race
    and returns standardized columns:
        filename, label, age_group, gender, skin_tone
    Keeps/deduces split if provided; K-Fold will override per fold.
    """

```

```

df = df.copy()

if 'filepath' in df.columns:
    df['filepath'] = df['filepath'].astype(str).str.replace('\\', '/', regex=False)

# filename
if 'filename' not in df.columns:
    if 'filepath' in df.columns:
        df['filename'] = df['filepath'].apply(lambda p: os.path.basename(str(p)))
    else:
        raise ValueError("CSV must include 'filepath' or 'filename'.")

# label
if 'label' not in df.columns and 'filepath' in df.columns:
    df['label'] = df['filepath'].str.split('/').str[1]
df['label'] = df['label'].astype(str)

# age_group (bucket by numeric age if present)
if 'age_group' not in df.columns:
    if 'age' in df.columns:
        age_num = pd.to_numeric(df['age'], errors='coerce')
        bins = [-1, 17, 29, 44, 64, 200] # child, young_adult, adult, middle
        names = ['child', 'young_adult', 'adult', 'middle_aged', 'senior']
        df['age_group'] = pd.cut(age_num, bins=bins, labels=names).astype(str)
    else:
        df['age_group'] = 'unknown'

# gender
if 'gender' not in df.columns:
    if 'dominant_gender' in df.columns:
        g = df['dominant_gender'].astype(str).str.strip().str.lower()
        df['gender'] = g.map({'man': 'male', 'male': 'male', 'woman': 'female', 'fema
    else:
        df['gender'] = 'unknown'

# skin_tone / race
if 'skin_tone' not in df.columns:
    if 'dominant_race' in df.columns:
        df['skin_tone'] = df['dominant_race'].astype(str).str.strip().str.lower
    else:
        df['skin_tone'] = 'unknown'

required = ['filename', 'label', 'age_group', 'gender', 'skin_tone']
missing = [c for c in required if c not in df.columns]
if missing:
    raise ValueError(f"Missing required columns after standardization: {missing}")

return df

# -----
# 3) DATASET & VOCAB
# -----
class GPUAugOnlyNorm(nn.Module):
    def __init__(self):
        super().__init__()

```

```

        self.normalize = transforms.Normalize(mean=[0.485, 0.456, 0.406],
                                              std=[0.229, 0.224, 0.225])

    @torch.no_grad()
    def forward(self, x): return self.normalize(x)

class MetaVocab:
    def __init__(self, df_train, cols=('age_group', 'gender', 'skin_tone')):
        self.cols = cols
        self.unk_token = "<UNK>"
        self.vocabs = {}
        for c in cols:
            uniq = sorted([self.unk_token] + sorted(df_train[c].dropna().astype(str)))
            self.vocabs[c] = {v:i for i,v in enumerate(uniq)}
        self.sizes = {c: len(self.vocabs[c]) for c in cols}
    def encode_row(self, row):
        out = {}
        for c in self.cols:
            v = str(row[c]) if pd.notnull(row[c]) else self.unk_token
            out[c] = self.vocabs[c].get(v, self.vocabs[c][self.unk_token])
        return out

class MultimodalDermDataset(Dataset):
    """
    Uses 'filepath' if present (relative to DATA_DIR or absolute),
    else DATA_DIR/split/label/filename (split provided per fold).
    """
    def __init__(self, config, df, class_to_idx, meta_vocab, transform=None):
        self.config = config
        self.df = df.reset_index(drop=True)
        self.class_to_idx = class_to_idx
        self.transform = transform
        self.meta_vocab = meta_vocab

        self.image_paths, self.labels, self.meta = [], [], []
        use_filepath = 'filepath' in self.df.columns
        for _, row in self.df.iterrows():
            if use_filepath and pd.notnull(row.get('filepath', None)):
                raw = str(row['filepath'])
                p = raw if os.path.isabs(raw) else os.path.join(config.DATA_DIR, raw)
            else:
                # Expect a 'split' column already assigned for this fold
                p = os.path.join(config.DATA_DIR, str(row['split']), str(row['label']))
            self.image_paths.append(p)
            self.labels.append(self.class_to_idx[str(row['label'])])
            self.meta.append(self.meta_vocab.encode_row(row))

        # Preload images (disable if memory constrained)
        self.images = []
        for p in self.image_paths:
            with Image.open(p).convert("RGB") as im:
                self.images.append(im.copy())

        self.class_names = sorted(class_to_idx.keys())

    def __len__(self): return len(self.labels)
    def __getitem__(self, idx):

```

```

        image = self.images[idx]
        if self.transform: image = self.transform(image)
        return image, self.labels[idx], self.meta[idx]

# -----
# 4) MODEL (DINOv3 + Metadata-Conditioned DSAF)
# -----
class MetadataEncoder(nn.Module):
    def __init__(self, meta_vocab, emb_dims=None, hidden=128, out_dim=256, dropout=0.1):
        super().__init__()
        sizes = meta_vocab.sizes
        if emb_dims is None:
            emb_dims = {
                'age_group': min(16, max(4, sizes['age_group'])),
                'gender': min(8, max(3, sizes['gender'])),
                'skin_tone': min(16, max(4, sizes['skin_tone'])),
            }
        self.emb_age = nn.Embedding(sizes['age_group'], emb_dims['age_group'])
        self.emb_gen = nn.Embedding(sizes['gender'], emb_dims['gender'])
        self.emb_tone = nn.Embedding(sizes['skin_tone'], emb_dims['skin_tone'])
        concat_dim = emb_dims['age_group'] + emb_dims['gender'] + emb_dims['skin_tone']
        self.mlp = nn.Sequential(
            nn.Linear(concat_dim, hidden), nn.GELU(), nn.Dropout(dropout),
            nn.Linear(hidden, out_dim), nn.GELU()
        )
    def forward(self, meta_idx):
        a = self.emb_age(meta_idx['age_group'])
        g = self.emb_gen(meta_idx['gender'])
        s = self.emb_tone(meta_idx['skin_tone'])
        return self.mlp(torch.cat([a, g, s], dim=-1))

class DSAFHeadMultimodal(nn.Module):
    def __init__(self, hidden_size=384, num_classes=5, num_heads=6, dropout=0.1,
                 meta_dim=256, gate_scale=0.5, meta_dropout=0.3):
        super().__init__()
        self.gate_scale = gate_scale
        self.meta_dropout = nn.Dropout(meta_dropout)

        self.global_stream = nn.Sequential(
            nn.Linear(hidden_size, hidden_size//2), nn.GELU(), nn.Dropout(dropout)
        )
        self.local_attention = nn.MultiheadAttention(
            embed_dim=hidden_size, num_heads=num_heads, dropout=dropout, batch_first=True
        )
        self.local_norm1 = nn.LayerNorm(hidden_size)
        self.local_mlp = nn.Sequential(
            nn.Linear(hidden_size, hidden_size*2), nn.GELU(), nn.Dropout(dropout),
            nn.Linear(hidden_size*2, hidden_size)
        )
        self.local_norm2 = nn.LayerNorm(hidden_size)
        self.local_stream_fc = nn.Sequential(
            nn.Linear(hidden_size, hidden_size//2), nn.GELU(), nn.Dropout(dropout)
        )

        half = hidden_size//2

```

```

self.alpha_gate = nn.Sequential(nn.Linear(meta_dim, half), nn.GELU(), nn.Li
self.beta_gate = nn.Sequential(nn.Linear(meta_dim, half), nn.GELU(), nn.Li

self.classifier = nn.Sequential(
    nn.LayerNorm(half), nn.Linear(half, half//2), nn.GELU(), nn.Dropout(dro
    nn.Linear(half//2, num_classes)
)

def forward(self, hidden_states, v_meta):
    cls = hidden_states[:, 0]
    patches = hidden_states[:, 1:]

    v_global = self.global_stream(cls)
    attn, _ = self.local_attention(patches, patches, patches)
    x = self.local_norm1(patches + attn)
    x = self.local_norm2(x + self.local_mlp(x))
    v_local = self.local_stream_fc(torch.mean(x, dim=1))

    v_meta = self.meta_dropout(v_meta)
    alpha = torch.sigmoid(self.alpha_gate(v_meta))
    beta = torch.sigmoid(self.beta_gate(v_meta))
    alpha = 0.5 + self.gate_scale * (alpha - 0.5)
    beta = 0.5 + self.gate_scale * (beta - 0.5)

    v_fused = alpha * v_global + beta * v_local
    return self.classifier(v_fused)

class DINOv3Multimodal(nn.Module):
    def __init__(self, model_name, num_classes, meta_vocab, meta_out_dim=256):
        super().__init__()
        self.dinov3 = AutoModel.from_pretrained(model_name)
        if not config.PARTIALLY_UNFREEZE:
            for p in self.dinov3.parameters(): p.requires_grad = False
        else:
            for p in self.dinov3.parameters(): p.requires_grad = False
            if hasattr(self.dinov3, "vit"):
                for p in self.dinov3.vit.encoder.layer[-1].parameters(): p.requires
        H = self.dinov3.config.hidden_size
        heads = self.dinov3.config.num_attention_heads
        self.meta_encoder = MetadataEncoder(meta_vocab, out_dim=meta_out_dim)
        self.head = DSAFHeadMultimodal(hidden_size=H,
                                       num_classes=num_classes,
                                       num_heads=heads,
                                       meta_dim=meta_out_dim,
                                       gate_scale=config.GATE_SCALE,
                                       meta_dropout=config.META_DROPOUT)

    def forward(self, pixel_values, meta_idx):
        self.dinov3.eval() if not config.PARTIALLY_UNFREEZE else None
        with torch.no_grad() if not config.PARTIALLY_UNFREEZE else torch.enable_gra
            out = self.dinov3(pixel_values=pixel_values)
        v_meta = self.meta_encoder(meta_idx)
        return self.head(out.last_hidden_state, v_meta)

# -----
# 5) TRAIN / EVAL HELPERS

```

```

# -----
def _to_device_meta(batch_meta, device):
    if isinstance(batch_meta, dict):
        ag = torch.as_tensor(batch_meta['age_group'], dtype=torch.long, device=device)
        ge = torch.as_tensor(batch_meta['gender'], dtype=torch.long, device=device)
        st = torch.as_tensor(batch_meta['skin_tone'], dtype=torch.long, device=device)
        return {'age_group': ag, 'gender': ge, 'skin_tone': st}
    else:
        ag = torch.tensor([m['age_group'] for m in batch_meta], dtype=torch.long, device=device)
        ge = torch.tensor([m['gender'] for m in batch_meta], dtype=torch.long, device=device)
        st = torch.tensor([m['skin_tone'] for m in batch_meta], dtype=torch.long, device=device)
        return {'age_group': ag, 'gender': ge, 'skin_tone': st}

def make_data loaders_from_df(config, df_std):
    classes = sorted(df_std[df_std['split']=='Train']['label'].astype(str).unique())
    if not classes:
        classes = sorted(df_std['label'].astype(str).unique())
    class_to_idx = {c:i for i,c in enumerate(classes)}
    config.NUM_CLASSES = len(classes)

    df_vocab = df_std[df_std['split']=='Train'] if (df_std['split']=='Train').any()
    meta_vocab = MetaVocab(df_vocab, cols=('age_group', 'gender', 'skin_tone'))

    train_tfm = transforms.Compose([
        transforms.Resize((config.IMG_SIZE, config.IMG_SIZE)),
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.RandomApply([transforms.ColorJitter(0.4,0.4,0.2,0.1)], p=0.8),
        transforms.RandomRotation(degrees=15),
        transforms.ToTensor(),
    ])
    eval_tfm = transforms.Compose([
        transforms.Resize((config.IMG_SIZE, config.IMG_SIZE)),
        transforms.ToTensor(),
    ])

    dsets = {}
    for split, tfm in [('Train', train_tfm), ('Val', eval_tfm), ('Test', eval_tfm)]:
        dfi = df_std[df_std['split'] == split]
        dsets[split] = MultimodalDermDataset(config, dfi, class_to_idx, meta_vocab,

    dls = {
        'Train': DataLoader(dsets['Train'], batch_size=config.BATCH_SIZE, shuffle=True,
                           num_workers=config.NUM_WORKERS, pin_memory=True),
        'Val':   DataLoader(dsets['Val'],   batch_size=config.BATCH_SIZE, shuffle=False,
                           num_workers=config.NUM_WORKERS, pin_memory=True),
        'Test':  DataLoader(dsets['Test'],  batch_size=config.BATCH_SIZE, shuffle=False,
                           num_workers=config.NUM_WORKERS, pin_memory=True),
    }
    sizes = {k: len(dsets[k]) for k in dsets}
    return dls, sizes, classes, meta_vocab

# *** NEW PLOTTING FUNCTION ***
def plot_and_save_history(hist, fold_id, results_dir):
    """Plots training and validation loss and accuracy and saves them to a file."""
    if not HAVE_PLOTTING:
        return

```

```

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 6))
epochs = range(1, len(hist['train_loss']) + 1)

# Plotting Loss
ax1.plot(epochs, hist['train_loss'], 'bo-', label='Training loss')
ax1.plot(epochs, hist['val_loss'], 'ro-', label='Validation loss')
ax1.set_title(f'Training and Validation Loss (Fold {fold_id})')
ax1.set_xlabel('Epochs')
ax1.set_ylabel('Loss')
ax1.legend()
ax1.grid(True)

# Plotting Accuracy
ax2.plot(epochs, hist['train_acc'], 'bo-', label='Training accuracy')
ax2.plot(epochs, hist['val_acc'], 'ro-', label='Validation accuracy')
ax2.set_title(f'Training and Validation Accuracy (Fold {fold_id})')
ax2.set_xlabel('Epochs')
ax2.set_ylabel('Accuracy')
ax2.legend()
ax2.grid(True)

plt.tight_layout()
history_path = os.path.join(results_dir, f'training_history_fold_{fold_id}.png')
plt.savefig(history_path, dpi=300)
plt.show()
plt.close()
print(f"Saved training history plot to: {history_path}")

def train_model(model, dls, sizes, num_epochs, fold_id):
    start = time.time()
    best = deepcopy(model.state_dict())
    best_val = float('inf')
    patience = 0
    hist = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}

    scaler = GradScaler(enabled=config.USE_AMP)
    gpu_norm = GPUAugOnlyNorm().to(config.DEVICE)
    opt = optim.AdamW(filter(lambda p: p.requires_grad, model.parameters()),
                        lr=config.LEARNING_RATE, weight_decay=config.WEIGHT_DECAY)

    def lr_lambda(ep):
        if ep < config.WARMUP_EPOCHS: return float(ep+1)/float(config.WARMUP_EPOCHS)
        return max(0.0, 0.5 * (1.0 + np.cos(np.pi * (ep - config.WARMUP_EPOCHS) / (
            sch = optim.lr_scheduler.LambdaLR(opt, lr_lambda)
            criterion = nn.CrossEntropyLoss(label_smoothing=config.LABEL_SMOOTHING)

    best_model_path = os.path.join(config.RESULTS_DIR, f'best_multimodal_dsa_fold_

    print("\n" + "="*80 + f"\nSTART TRAINING (Multimodal DSAF) for {num_epochs} epo
    for ep in range(num_epochs):
        for phase in ['Train', 'Val']:
            model.train() if phase=='Train' else model.eval()
            run_loss, run_correct = 0.0, 0
            for images, labels, metas in dls[phase]:

```

```

        images = images.to(config.DEVICE)
        labels = torch.as_tensor(labels, dtype=torch.long, device=config.DEVICE)
        meta_idx = _to_device_meta(metas, config.DEVICE)
        images = gpu_norm(images)
        opt.zero_grad(set_to_none=True)
        with autocast(device_type=config.DEVICE.split(':')[0], dtype=torch.float16):
            logits = model(pixel_values=images, meta_idx=meta_idx)
            loss = criterion(logits, labels)
        if phase=='Train':
            scaler.scale(loss).backward()
            scaler.step(opt)
            scaler.update()
            preds = logits.argmax(1)
            run_loss += loss.item() * images.size(0)
            run_correct += (preds == labels).sum().item()
        e_loss = run_loss / max(1, sizes[phase])
        e_acc = run_correct / max(1, sizes[phase])
        if phase=='Train':
            hist['train_loss'].append(e_loss); hist['train_acc'].append(e_acc)
        else:
            hist['val_loss'].append(e_loss); hist['val_acc'].append(e_acc)
        if e_loss < best_val:
            best_val = e_loss; best = deepcopy(model.state_dict()); patience = 0
            torch.save(model.state_dict(), best_model_path)
        else:
            patience += 1
    sch.step()
    print(f"Epoch {ep+1}/{num_epochs} | Train {hist['train_loss'][-1]:.4f}/{hist['train_acc'][-1]:.4f} | Val {hist['val_loss'][-1]:.4f}/{hist['val_acc'][-1]:.4f} | LR {sch.get_lr()[0]:.4f}")
    if patience >= config.PATIENCE:
        print("Early stopping.")
        break
model.load_state_dict(best)
print("Training complete in {:.1f} minutes. Best Val Loss {:.4f}".format((time.time() - start_time) / 60, best_val))
print(f"Best model for fold saved to: {best_model_path}")

# ***MODIFIED***: Plot and save training history after training is complete
plot_and_save_history(hist, fold_id, config.RESULTS_DIR)

return model, hist

# -----
# 6) EVALUATION (Concise reports + WGA/EOD with min-support + Pooled subgroup accuracies)
# -----
def _print_section(title):
    print("\n" + "="*80)
    print(title.upper())
    print("="*80 + "\n")

def _print_group_report(group_title, y_true, y_pred, class_names):
    labels_full = list(range(len(class_names)))
    print(f"--- Results for Group: {group_title} ---")
    print(classification_report(y_true, y_pred,
                                labels=labels_full,
                                target_names=class_names,

```

```

                                zero_division=0))

print()

def _group_fairness_metrics(y_true, y_pred, groups, num_classes, min_support=None):
    uniq_groups = sorted(pd.Series(groups).astype(str).unique())
    per_group_acc, per_group_tprs, counts = {}, {}, {}
    y_true = np.asarray(y_true); y_pred = np.asarray(y_pred)

    for g in uniq_groups:
        mask = (np.asarray(groups) == g)
        counts[g] = int(mask.sum())
        yt = y_true[mask]; yp = y_pred[mask]
        acc_g = (yt == yp).mean() if len(yt) > 0 else np.nan
        tprs = []
        for c in range(num_classes):
            tp = ((yt == c) & (yp == c)).sum()
            pos = (yt == c).sum()
            tprs.append(tp/pos if pos > 0 else np.nan)
        per_group_acc[g] = acc_g
        per_group_tprs[g] = np.array(tprs, dtype=float)

    if min_support is not None:
        filt_keys = [g for g in uniq_groups if counts[g] >= min_support]
    else:
        filt_keys = uniq_groups

    if filt_keys:
        wga = float(np.nanmin([per_group_acc[g] for g in filt_keys]))
    else:
        wga = float('nan')

    def macro_eod(keys, per_group_tprs, K):
        max_gap, worst = 0.0, (None, None)
        for i in range(len(keys)):
            for j in range(i+1, len(keys)):
                gi, gj = keys[i], keys[j]
                a, b = per_group_tprs[gi], per_group_tprs[gj]
                mask = ~(np.isnan(a) | np.isnan(b))
                if mask.sum() == 0:
                    continue
                gap = float(np.nanmean(np.abs(a[mask] - b[mask])))
                if gap > max_gap:
                    max_gap, worst = gap, (gi, gj)
        return max_gap, worst

    eod_val, worst_pair = macro_eod(filt_keys, per_group_tprs, num_classes)

    return {
        'per_group_acc': per_group_acc,
        'per_group_tprs': per_group_tprs,
        'counts': counts,
        'keys_used': filt_keys,
        'wga': wga,
        'eod': eod_val,
        'worst_pair': worst_pair
    }

```

```

# *** NEW PLOTTING HELPER FUNCTIONS ***
def plot_and_save_roc_curves(y_true_bin, y_prob, class_names, fold_id, results_dir)
    """Plots and saves per-class and macro-average ROC curves."""
    if not HAVE_PLOTTING:
        return
    n_classes = len(class_names)
    fpr, tpr, roc_auc = dict(), dict(), dict()

    for i in range(n_classes):
        fpr[i], tpr[i], _ = roc_curve(y_true_bin[:, i], y_prob[:, i])
        roc_auc[i] = auc(fpr[i], tpr[i])

    # Compute macro-average ROC curve
    all_fpr = np.unique(np.concatenate([fpr[i] for i in range(n_classes)]))
    mean_tpr = np.zeros_like(all_fpr)
    for i in range(n_classes):
        mean_tpr += np.interp(all_fpr, fpr[i], tpr[i])
    mean_tpr /= n_classes
    fpr["macro"] = all_fpr
    tpr["macro"] = mean_tpr
    roc_auc["macro"] = auc(fpr["macro"], tpr["macro"])

    # Plot
    plt.figure(figsize=(12, 10))
    colors = cycle(['aqua', 'darkorange', 'cornflowerblue', 'green', 'red', 'purple'])
    for i, color in zip(range(n_classes), colors):
        plt.plot(fpr[i], tpr[i], color=color, lw=2,
                 label=f'ROC curve of class {class_names[i]} (area = {roc_auc[i]:0.2f})')

    plt.plot(fpr["macro"], tpr["macro"],
             label=f'Macro-average ROC curve (area = {roc_auc["macro"]:0.2f})',
             color='navy', linestyle=':', linewidth=4)

    plt.plot([0, 1], [0, 1], 'k--', lw=2)
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(f'Multi-class Receiver Operating Characteristic (ROC) - Fold {fold_id}')
    plt.legend(loc="lower right")
    plt.grid(True)

    roc_path = os.path.join(results_dir, f'roc_curves_fold_{fold_id}.png')
    plt.savefig(roc_path, dpi=300)
    plt.show()
    plt.close()
    print(f"Saved ROC curves plot to: {roc_path}")

def plot_and_save_scalar_metrics(metrics_dict, fold_id, results_dir):
    """Plots and saves a bar chart of key scalar metrics."""
    if not HAVE_PLOTTING:
        return

    names = list(metrics_dict.keys())
    values = list(metrics_dict.values())

```

```

plt.figure(figsize=(10, 6))
bars = sns.barplot(x=names, y=values, palette="viridis")
plt.title(f'Overall Scalar Metrics Summary - Fold {fold_id}')
plt.ylabel('Score')
plt.ylim(0, 1.05)

for bar in bars.patches:
    plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.01,
             f'{bar.get_height():.4f}', ha='center', va='bottom', fontsize=10)

scalar_path = os.path.join(results_dir, f'scalar_metrics_fold_{fold_id}.png')
plt.savefig(scalar_path, dpi=300)
plt.show()
plt.close()
print(f"Saved scalar metrics plot to: {scalar_path}")

def evaluate_and_visualize(model, dl_test, class_names, df_fold, fold_id):
    model.eval()
    gpu_norm = GPUAugOnlyNorm().to(config.DEVICE)
    y_true, y_pred, y_prob = [], [], []

    with torch.no_grad():
        for images, labels, metas in dl_test:
            images = images.to(config.DEVICE)
            labels = torch.as_tensor(labels, dtype=torch.long, device=config.DEVICE)
            meta_idx = _to_device_meta(metas, config.DEVICE)
            images = gpu_norm(images)
            logits = model(pixel_values=images, meta_idx=meta_idx)
            probs = torch.softmax(logits, dim=1)
            y_true += labels.cpu().numpy().tolist()
            y_pred += logits.argmax(1).cpu().numpy().tolist()
            y_prob += probs.cpu().numpy().tolist()

    y_true = np.array(y_true); y_pred = np.array(y_pred); y_prob = np.array(y_prob)

    _print_section(f"OVERALL PERFORMANCE (FOLD {fold_id})")
    print(classification_report(y_true, y_pred,
                                labels=list(range(len(class_names))),
                                target_names=class_names,
                                zero_division=0))

    # Calculate scalar metrics
    acc = accuracy_score(y_true, y_pred)
    kappa = cohen_kappa_score(y_true, y_pred)
    mcc = matthews_corrcoef(y_true, y_pred)
    try:
        y_true_bin = label_binarize(y_true, classes=list(range(len(class_names))))
        # Handle case where a class might not be in y_true for a small test set
        if y_true_bin.shape[1] != len(class_names):
            y_true_bin = label_binarize(y_true, classes=np.arange(len(class_names)))
        roc_auc_macro = roc_auc_score(y_true_bin, y_prob, average='macro', multi_cl
    except Exception:
        roc_auc_macro = float('nan')
        y_true_bin = None # Ensure it exists

```

```

print("\n--- Overall Scalar Metrics ---")
print(f"Accuracy          : {acc:.4f}")
print(f"Macro ROC AUC      : {roc_auc_macro:.4f}")
print(f"Cohen's Kappa       : {kappa:.4f}")
print(f"Matthews CorrCoef: {mcc:.4f}\n")

# *** MODIFIED: Generate and save all plots ***
if HAVE_PLOTTING:
    # 1. Normalized Confusion Matrix
    cm = confusion_matrix(y_true, y_pred, labels=list(range(len(class_names))))
    with np.errstate(divide='ignore', invalid='ignore'):
        cm_norm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
    cm_norm = np.nan_to_num(cm_norm)

    plt.figure(figsize=(12, 10))
    sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Blues',
                xticklabels=class_names, yticklabels=class_names, vmin=0, vmax=1)
    plt.title(f'Normalized Confusion Matrix (Fold {fold_id})\n(Normalized by im
    plt.ylabel('True Label')
    plt.xlabel('Predicted Label')
    plt.tight_layout()
    cm_path = os.path.join(config.RESULTS_DIR, f'confusion_matrix_fold_{fold_id}')
    plt.savefig(cm_path, dpi=300)
    plt.show()
    plt.close()
    print(f"Saved normalized confusion matrix to: {cm_path}")

    # 2. ROC Curves
    if y_true_bin is not None and y_true_bin.shape[1] == len(class_names):
        plot_and_save_roc_curves(y_true_bin, y_prob, class_names, fold_id, con
    else:
        print("Skipping ROC curve plot due to missing classes in the test set.")

    # 3. Scalar Metrics Bar Chart
    scalar_metrics = {
        'Accuracy': acc,
        'Macro AUC': roc_auc_macro,
        'Kappa': kappa,
        'MCC': mcc
    }
    plot_and_save_scalar_metrics(scalar_metrics, fold_id, config.RESULTS_DIR)

else:
    print("Skipping plots: matplotlib or seaborn not installed.")

# Align Test metadata rows
df_test = df_fold[df_fold['split'] == 'Test'][['age_group', 'gender', 'skin_tone']]
if len(df_test) != len(y_true):
    m = min(len(df_test), len(y_true))
    df_test = df_test.iloc[:m].copy()
    y_true = y_true[:m]; y_pred = y_pred[:m]; y_prob = y_prob[:m]

# BY GENDER
_print_section("PERFORMANCE ANALYSIS BY: DOMINANT_GENDER")
for g_key, g_title in [('male', 'Man'), ('female', 'Woman')]:

```

```

mask = (df_test['gender'].astype(str) == g_key).values
if mask.sum() == 0: continue
_print_group_report(g_title, y_true[mask], y_pred[mask], class_names)

# BY AGE BUCKETS
_print_section("PERFORMANCE ANALYSIS BY: AGE_GROUP")
def bucket(x):
    x = str(x)
    if x == 'child': return '0-17'
    if x in ('young_adult', 'adult', 'middle_aged'): return '18-59'
    if x == 'senior': return '60+'
    return 'unknown'
age_disp = df_test['age_group'].apply(bucket)
for group in ['0-17', '18-59', '60+']:
    mask = (age_disp == group).values
    if mask.sum() == 0: continue
    _print_group_report(group, y_true[mask], y_pred[mask], class_names)

# BY SKIN TONE
_print_section("PERFORMANCE ANALYSIS BY: DOMINANT_RACE")
for tone in sorted(df_test['skin_tone'].astype(str).unique()):
    if tone == 'unknown': continue
    mask = (df_test['skin_tone'].astype(str) == tone).values
    if mask.sum() == 0: continue
    _print_group_report(tone, y_true[mask], y_pred[mask], class_names)

# ----- FAIRNESS SUMMARY -----
groups_composite = (age_disp.astype(str) + "|" +
                    df_test['gender'].astype(str) + "|" +
                    df_test['skin_tone'].astype(str)).values

fair_all = _group_fairness_metrics(
    y_true, y_pred, groups_composite, num_classes=len(class_names), min_support
)
fair_filt = _group_fairness_metrics(
    y_true, y_pred, groups_composite, num_classes=len(class_names), min_support
)

print("\n" + "="*80)
print(f"FAIRNESS SUMMARY (ALL GROUPS) | WGA: {fair_all['wga']:.4f} EOD (max
if fair_all['worst_pair'][0] is not None:
    print(f"Worst pair: {fair_all['worst_pair'][0]} vs {fair_all['worst_pair']
print("="*80)

print("\n" + "="*80)
print(f"FAIRNESS SUMMARY (FILTERED, n >= {config.MIN_SUPPORT}) | "
      f"WGA: {fair_filt['wga']:.4f} EOD (max TPR gap): {fair_filt['eod']:.4f}"
if fair_filt['worst_pair'][0] is not None:
    print(f"Worst pair (among filtered): {fair_filt['worst_pair'][0]} vs {fai
print("Groups used (filtered):", fair_filt['keys_used'])
print("="*80)

# ----- per-subgroup accuracy & counts for pooling across folds -----
def acc_counts_from_series(series_vals):
    """returns dict: group -> (correct, total)"""
    d = {}

```

```

s = series_vals.astype(str).values
for g in np.unique(s):
    mask = (s == g)
    tot = int(mask.sum())
    if tot == 0: continue
    cor = int((y_true[mask] == y_pred[mask]).sum())
    d[str(g)] = (cor, tot)
return d

pooled_info = {
    'gender': acc_counts_from_series(df_test['gender']),
    'age': acc_counts_from_series(age_disp),
    'skin_tone': acc_counts_from_series(df_test['skin_tone']),
    'composite': acc_counts_from_series(pd.Series(groups_composite)),
}

return {
    'accuracy': acc, 'roc_auc_macro': roc_auc_macro, 'kappa': kappa, 'mcc': mcc
    'wga_all': fair_all['wga'], 'eod_all': fair_all['eod'],
    'wga_filtered': fair_filt['wga'], 'eod_filtered': fair_filt['eod'],
    'pooled_groups': pooled_info
}

# -----
# 7) K-FOLD: BUILD SPLITS -> TRAIN -> EVAL -> AGGREGATE
# -----
def build_kfold_splits(df_std, n_splits=7, seed=42, val_within_remainder=0.1765):
    """
    For each fold:
    - Test = 1/n_splits of data (stratified by label) ~15%
    - From the remainder (~85%), Val = val_within_remainder of remainder (so over 15%)
    - Train = the rest (~70%)
    Returns a list of fold DataFrames with 'split' = Train/Val/Test.
    """
    df = df_std.copy().reset_index(drop=True)
    y = df['label'].astype(str).values
    idx = np.arange(len(df))
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)

    folds = []
    for fold_id, (trainval_idx, test_idx) in enumerate(skf.split(idx, y), 1):
        rest_labels = y[trainval_idx]
        sss2 = StratifiedShuffleSplit(n_splits=1, test_size=val_within_remainder, random_state=seed)
        for tr_rel, val_rel in sss2.split(trainval_idx, rest_labels):
            tr_idx = trainval_idx[tr_rel]
            val_idx = trainval_idx[val_rel]

            split = pd.Series(index=np.arange(len(df)), data="Ignore", dtype=str)
            split.loc[tr_idx] = "Train"
            split.loc[val_idx] = "Val"
            split.loc[test_idx] = "Test"

            dfi = df.copy()
            dfi['split'] = split.values
            dfi = dfi[dfi['split'].isin(['Train', 'Val', 'Test'])].reset_index(drop=True)

```

```

    # Quick fold report
    nT = (dfi.split=='Train').sum(); nV = (dfi.split=='Val').sum(); nE = (dfi.s
    print(f"Fold {fold_id}: Train={nT} ({100*nT/len(dfi):.1f}%) Val={nV} ({100
    folds.append(dfi)
    return folds

def run_one_fold(fold_df, fold_id, config):
    print("\n" + "#" * 88)
    print(f"### FOLD {fold_id} ### Train={sum(fold_df.split=='Train')}, Val={sum(f
    print("#" * 88)

    dls, sizes, class_names, meta_vocab = make_dataloaders_from_df(config, fold_df)
    model = DINOv3Multimodal(config.MODEL_NAME, num_classes=len(class_names), meta_

    # Optional model size / FLOPs
    total_params = sum(p.numel() for p in model.parameters())
    trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad
    print(f"Params: {total_params/1e6:.2f}M | Trainable: {trainable_params/1e6:.2f}
    if HAVE_THOP:
        try:
            dummy_img = torch.randn(1,3,config.IMG_SIZE,config.IMG_SIZE).to(config.
            dummy_meta = {
                'age_group': torch.zeros(1, dtype=torch.long, device=config.DEVICE)
                'gender': torch.zeros(1, dtype=torch.long, device=config.DEVICE)
                'skin_tone': torch.zeros(1, dtype=torch.long, device=config.DEVICE)
            }
            macs, params = profile(model, inputs=(dummy_img, dummy_meta), verbose=F
            print(f"Approx MACs: {macs/1e9:.2f} G, Params (THOP): {params/1e6:.2f}
        except Exception as e:
            print("THOP profile failed (skipped):", e)

    model, _ = train_model(model, dls, sizes, num_epochs=config.EPOCHS, fold_id=fol

    best_path = os.path.join(config.RESULTS_DIR, f'best_multimodal_dsaf_fold_{fold_
    if os.path.exists(best_path):
        model.load_state_dict(torch.load(best_path, map_location=config.DEVICE))

    metrics = evaluate_and_visualize(model, dls['Test'], class_names, fold_df, fold
    return metrics

def kfold_cross_val(df_std, config):
    folds = build_kfold_splits(df_std,
                                n_splits=config.N_SPLITS,
                                seed=42,
                                val_within_remainder=config.VAL_WITHIN_REMAINDER)

    per_fold = []
    for i, fold_df in enumerate(folds, 1):
        m = run_one_fold(fold_df, i, config)
        m['fold'] = i
        per_fold.append(m)

    perf = pd.DataFrame(per_fold).set_index('fold')
    def mean_std(col):
        vals = perf[col].astype(float).values
        return float(np.nanmean(vals)), float(np.nanstd(vals))

```

```

summary = {
    'acc_mean': mean_std('accuracy')[0], 'acc_std': mean_std('accuracy')[1],
    'roc_macro_mean': mean_std('roc_auc_macro')[0], 'roc_macro_std': mean_std('
    'kappa_mean': mean_std('kappa')[0], 'kappa_std': mean_std('kappa')[1],
    'mcc_mean': mean_std('mcc')[0], 'mcc_std': mean_std('mcc')[1],
    'wga_all_mean': mean_std('wga_all')[0], 'wga_all_std': mean_std('wga_all')[
    'eod_all_mean': mean_std('eod_all')[0], 'eod_all_std': mean_std('eod_all')[
    'wga_filt_mean': mean_std('wga_filtered')[0], 'wga_filt_std': mean_std('wga
    'eod_filt_mean': mean_std('eod_filtered')[0], 'eod_filt_std': mean_std('eod
}

print("\n" + "="*88)
print(f"Cross-validated over {config.N_SPLITS} folds (~ Train 70%, Val 15%, Tes
print("Overall accuracy   : {:.3f} ± {:.3f}".format(summary['acc_mean'], summary
print("Macro AUC           : {:.3f} ± {:.3f}".format(summary['roc_macro_mean'], s
print("Kappa               : {:.3f} ± {:.3f}".format(summary['kappa_mean'], summa
print("MCC                 : {:.3f} ± {:.3f}".format(summary['mcc_mean'], summary
print("WGA (all groups)    : {:.3f} ± {:.3f}".format(summary['wga_all_mean'], sum
print("EOD (all groups)   : {:.3f} ± {:.3f}".format(summary['eod_all_mean'], sum
print("WGA (n≥{})         : {:.3f} ± {:.3f}".format(config.MIN_SUPPORT, summary['wg
print("EOD (n≥{})         : {:.3f} ± {:.3f}".format(config.MIN_SUPPORT, summary['eo
print("="*88)

# ----- POOLED SUBGROUP OVERALL ACCURACIES ACROSS FOLDS -----
def merge_weighted(pooled_list):
    """pooled_list: list of dict group->(correct,total)"""
    agg = {}
    for d in pooled_list:
        for g, (c,t) in d.items():
            if g not in agg: agg[g] = [0,0]
            agg[g][0] += c; agg[g][1] += t
    out = []
    for g, (c, t) in agg.items():
        if t == 0: continue
        out.append((g, c / t, t))
    out.sort(key=lambda x: (-x[2], x[0])) # by count desc
    return out

# collect per-fold pooled dicts
pooled_gender = [m['pooled_groups']['gender'] for m in per_fold]
pooled_age = [m['pooled_groups']['age'] for m in per_fold]
pooled_skin = [m['pooled_groups']['skin_tone'] for m in per_fold]
pooled_comp = [m['pooled_groups']['composite'] for m in per_fold]

tbl_gender = merge_weighted(pooled_gender)
tbl_age = merge_weighted(pooled_age)
tbl_skin = merge_weighted(pooled_skin)
tbl_comp = merge_weighted(pooled_comp)

def print_table(title, rows, min_support=None):
    print("\n" + "-"*80)
    print(title)
    print("-"*80)
    print(f"{'Group':>35s} {'Acc':>8s} {'Count':>8s}")
    for g, acc, cnt in rows:

```

```

        if (min_support is not None) and (cnt < min_support):
            continue
        print(f"{g:35s} {acc:8.3f} {cnt:8d}")

print_table("POOLED SUBGROUP OVERALL ACCURACY – GENDER (all)", tbl_gender, min_
print_table(f"POOLED SUBGROUP OVERALL ACCURACY – GENDER (n≥{config.MIN_SUPPORT})",

print_table("POOLED SUBGROUP OVERALL ACCURACY – AGE (all)", tbl_age, min_suppor
print_table(f"POOLED SUBGROUP OVERALL ACCURACY – AGE (n≥{config.MIN_SUPPORT})",

print_table("POOLED SUBGROUP OVERALL ACCURACY – SKIN TONE (all)", tbl_skin, min
print_table(f"POOLED SUBGROUP OVERALL ACCURACY – SKIN TONE (n≥{config.MIN_SUPPO

print_table("POOLED SUBGROUP OVERALL ACCURACY – COMPOSITE age|gender|skin (all)
print_table(f"POOLED SUBGROUP OVERALL ACCURACY – COMPOSITE age|gender|skin (n≥{

# Save per-fold metrics and CV summary
perf_path = os.path.join(config.RESULTS_DIR, "kfold_performance.csv")
perf.to_csv(perf_path)
with open(os.path.join(config.RESULTS_DIR, "kfold_summary.json"), "w", encoding
    json.dump(summary, f, indent=2)

# Save pooled subgroup tables
pd.DataFrame(tbl_gender, columns=['group', 'accuracy', 'count']).to_csv(
    os.path.join(config.RESULTS_DIR, "pooled_accuracy_gender.csv"), index=False
pd.DataFrame(tbl_age, columns=['group', 'accuracy', 'count']).to_csv(
    os.path.join(config.RESULTS_DIR, "pooled_accuracy_age.csv"), index=False)
pd.DataFrame(tbl_skin, columns=['group', 'accuracy', 'count']).to_csv(
    os.path.join(config.RESULTS_DIR, "pooled_accuracy_skin_tone.csv"), index=Fa
pd.DataFrame(tbl_comp, columns=['group', 'accuracy', 'count']).to_csv(
    os.path.join(config.RESULTS_DIR, "pooled_accuracy_composite.csv"), index=Fa

print("\nSaved pooled subgroup accuracy tables in:", config.RESULTS_DIR)
return perf, summary

# -----
# 8) MAIN – LOAD CSV -> K-FOLD CV
# -----
if __name__ == "__main__":
    raw = pd.read_csv(config.ORIGINAL_METADATA_CSV)
    df_std = standardize_metadata_columns(raw)

    print("\nRunning K-Fold CV targeting overall split ≈ 70% Train / 15% Val / 15%
    perf_df, summary = kfold_cross_val(df_std, config)
    print("\nDone.")

```

Device: cuda

K-Fold: N_SPLITS=7 (~Test 14.3%), Val_in_remainder=0.175

Running K-Fold CV targeting overall split $\approx$ 70% Train / 15% Val / 15% Test ...

Fold 1: Train=399 (70.6%) Val=85 (15.0%) Test=81 (14.3%)
Fold 2: Train=399 (70.6%) Val=85 (15.0%) Test=81 (14.3%)
Fold 3: Train=399 (70.6%) Val=85 (15.0%) Test=81 (14.3%)
Fold 4: Train=399 (70.6%) Val=85 (15.0%) Test=81 (14.3%)
Fold 5: Train=399 (70.6%) Val=85 (15.0%) Test=81 (14.3%)
Fold 6: Train=400 (70.8%) Val=85 (15.0%) Test=80 (14.2%)
Fold 7: Train=400 (70.8%) Val=85 (15.0%) Test=80 (14.2%)

#####

####

FOLD 1 ### Train=399, Val=85, Test=81

#####

####

Params: 91.58M | Trainable: 5.92M (6.47%)

Approx MACs: 17.68 G, Params (THOP): 89.20 M

=====
START TRAINING (Multimodal DSAF) for 200 epochs
=====

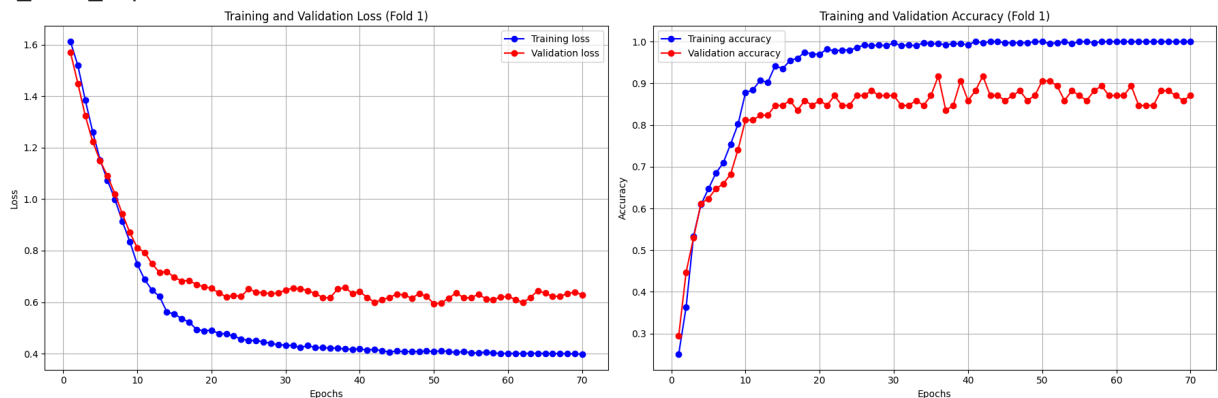
Epoch 1/200	Train 1.6113/0.2506	Val 1.5696/0.2941	LR 2.00e-05
Epoch 2/200	Train 1.5188/0.3634	Val 1.4492/0.4471	LR 3.00e-05
Epoch 3/200	Train 1.3842/0.5338	Val 1.3234/0.5294	LR 4.00e-05
Epoch 4/200	Train 1.2608/0.6090	Val 1.2230/0.6118	LR 5.00e-05
Epoch 5/200	Train 1.1522/0.6466	Val 1.1485/0.6235	LR 6.00e-05
Epoch 6/200	Train 1.0717/0.6842	Val 1.0896/0.6471	LR 7.00e-05
Epoch 7/200	Train 0.9993/0.7093	Val 1.0195/0.6588	LR 8.00e-05
Epoch 8/200	Train 0.9128/0.7544	Val 0.9423/0.6824	LR 9.00e-05
Epoch 9/200	Train 0.8353/0.8020	Val 0.8703/0.7412	LR 1.00e-04
Epoch 10/200	Train 0.7474/0.8772	Val 0.8113/0.8118	LR 1.00e-04
Epoch 11/200	Train 0.6876/0.8847	Val 0.7929/0.8118	LR 1.00e-04
Epoch 12/200	Train 0.6465/0.9073	Val 0.7487/0.8235	LR 1.00e-04
Epoch 13/200	Train 0.6222/0.9023	Val 0.7151/0.8235	LR 9.99e-05
Epoch 14/200	Train 0.5614/0.9424	Val 0.7186/0.8471	LR 9.99e-05
Epoch 15/200	Train 0.5542/0.9348	Val 0.6959/0.8471	LR 9.98e-05
Epoch 16/200	Train 0.5354/0.9549	Val 0.6819/0.8588	LR 9.98e-05
Epoch 17/200	Train 0.5225/0.9599	Val 0.6842/0.8353	LR 9.97e-05
Epoch 18/200	Train 0.4938/0.9749	Val 0.6681/0.8588	LR 9.96e-05
Epoch 19/200	Train 0.4884/0.9699	Val 0.6596/0.8471	LR 9.94e-05
Epoch 20/200	Train 0.4910/0.9699	Val 0.6534/0.8588	LR 9.93e-05
Epoch 21/200	Train 0.4779/0.9825	Val 0.6364/0.8471	LR 9.92e-05
Epoch 22/200	Train 0.4767/0.9774	Val 0.6202/0.8706	LR 9.90e-05
Epoch 23/200	Train 0.4683/0.9799	Val 0.6262/0.8471	LR 9.88e-05
Epoch 24/200	Train 0.4564/0.9799	Val 0.6222/0.8471	LR 9.87e-05
Epoch 25/200	Train 0.4513/0.9850	Val 0.6505/0.8706	LR 9.85e-05
Epoch 26/200	Train 0.4512/0.9925	Val 0.6396/0.8706	LR 9.83e-05
Epoch 27/200	Train 0.4456/0.9900	Val 0.6359/0.8824	LR 9.80e-05
Epoch 28/200	Train 0.4406/0.9925	Val 0.6336/0.8706	LR 9.78e-05
Epoch 29/200	Train 0.4337/0.9900	Val 0.6350/0.8706	LR 9.76e-05
Epoch 30/200	Train 0.4330/0.9975	Val 0.6468/0.8706	LR 9.73e-05
Epoch 31/200	Train 0.4310/0.9900	Val 0.6545/0.8471	LR 9.70e-05
Epoch 32/200	Train 0.4228/0.9925	Val 0.6512/0.8471	LR 9.67e-05
Epoch 33/200	Train 0.4324/0.9900	Val 0.6448/0.8588	LR 9.64e-05

Epoch 34/200	Train 0.4229/0.9975	Val 0.6326/0.8471	LR 9.61e-05
Epoch 35/200	Train 0.4229/0.9950	Val 0.6183/0.8706	LR 9.58e-05
Epoch 36/200	Train 0.4221/0.9950	Val 0.6166/0.9176	LR 9.55e-05
Epoch 37/200	Train 0.4212/0.9925	Val 0.6504/0.8353	LR 9.51e-05
Epoch 38/200	Train 0.4180/0.9950	Val 0.6565/0.8471	LR 9.47e-05
Epoch 39/200	Train 0.4168/0.9950	Val 0.6337/0.9059	LR 9.44e-05
Epoch 40/200	Train 0.4179/0.9925	Val 0.6408/0.8588	LR 9.40e-05
Epoch 41/200	Train 0.4138/1.0000	Val 0.6180/0.8824	LR 9.36e-05
Epoch 42/200	Train 0.4169/0.9975	Val 0.5991/0.9176	LR 9.32e-05
Epoch 43/200	Train 0.4108/1.0000	Val 0.6093/0.8706	LR 9.27e-05
Epoch 44/200	Train 0.4066/1.0000	Val 0.6177/0.8706	LR 9.23e-05
Epoch 45/200	Train 0.4096/0.9975	Val 0.6314/0.8588	LR 9.19e-05
Epoch 46/200	Train 0.4073/0.9975	Val 0.6272/0.8706	LR 9.14e-05
Epoch 47/200	Train 0.4081/0.9975	Val 0.6150/0.8824	LR 9.09e-05
Epoch 48/200	Train 0.4077/0.9975	Val 0.6332/0.8588	LR 9.05e-05
Epoch 49/200	Train 0.4096/1.0000	Val 0.6225/0.8706	LR 9.00e-05
Epoch 50/200	Train 0.4082/1.0000	Val 0.5931/0.9059	LR 8.95e-05
Epoch 51/200	Train 0.4108/0.9950	Val 0.5966/0.9059	LR 8.89e-05
Epoch 52/200	Train 0.4081/0.9975	Val 0.6155/0.8941	LR 8.84e-05
Epoch 53/200	Train 0.4045/1.0000	Val 0.6358/0.8588	LR 8.79e-05
Epoch 54/200	Train 0.4086/0.9950	Val 0.6167/0.8824	LR 8.73e-05
Epoch 55/200	Train 0.4031/1.0000	Val 0.6168/0.8706	LR 8.68e-05
Epoch 56/200	Train 0.4031/1.0000	Val 0.6309/0.8588	LR 8.62e-05
Epoch 57/200	Train 0.4062/0.9975	Val 0.6125/0.8824	LR 8.56e-05
Epoch 58/200	Train 0.4024/1.0000	Val 0.6099/0.8941	LR 8.51e-05
Epoch 59/200	Train 0.4005/1.0000	Val 0.6190/0.8706	LR 8.45e-05
Epoch 60/200	Train 0.4014/1.0000	Val 0.6223/0.8706	LR 8.39e-05
Epoch 61/200	Train 0.4002/1.0000	Val 0.6098/0.8706	LR 8.33e-05
Epoch 62/200	Train 0.4008/1.0000	Val 0.5985/0.8941	LR 8.26e-05
Epoch 63/200	Train 0.4011/1.0000	Val 0.6178/0.8471	LR 8.20e-05
Epoch 64/200	Train 0.4013/1.0000	Val 0.6445/0.8471	LR 8.14e-05
Epoch 65/200	Train 0.4005/1.0000	Val 0.6346/0.8471	LR 8.07e-05
Epoch 66/200	Train 0.4003/1.0000	Val 0.6218/0.8824	LR 8.01e-05
Epoch 67/200	Train 0.4002/1.0000	Val 0.6235/0.8824	LR 7.94e-05
Epoch 68/200	Train 0.3995/1.0000	Val 0.6325/0.8706	LR 7.87e-05
Epoch 69/200	Train 0.3990/1.0000	Val 0.6396/0.8588	LR 7.80e-05
Epoch 70/200	Train 0.3981/1.0000	Val 0.6286/0.8706	LR 7.73e-05

Early stopping.

Training complete in 5.6 minutes. Best Val Loss 0.5931

Best model for fold saved to: new_results_Multimodal_DSAF_KFold\best_multimodal_dsaf_fold_1.pth



Saved training history plot to: new_results_Multimodal_DSAF_KFold\training_history_fold_1.png

=====

OVERALL PERFORMANCE (FOLD 1)

=====

	precision	recall	f1-score	support
Herpes	1.00	0.71	0.83	7
Melanoma	1.00	1.00	1.00	3
Monkeypox	0.95	1.00	0.97	18
Sarampion	0.86	0.86	0.86	29
Varicela	0.84	0.88	0.86	24
accuracy			0.89	81
macro avg	0.93	0.89	0.91	81
weighted avg	0.89	0.89	0.89	81

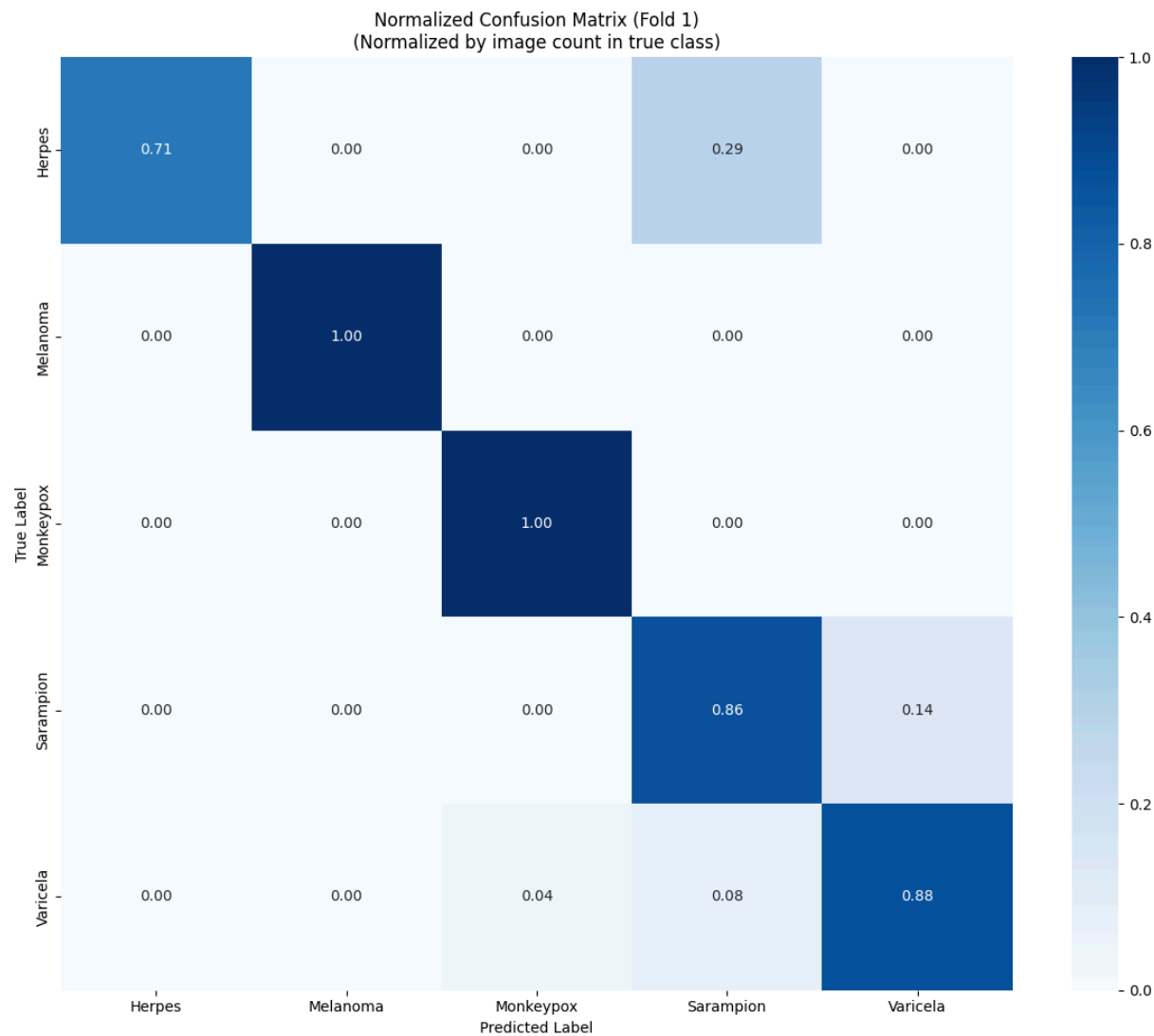
--- Overall Scalar Metrics ---

Accuracy : 0.8889

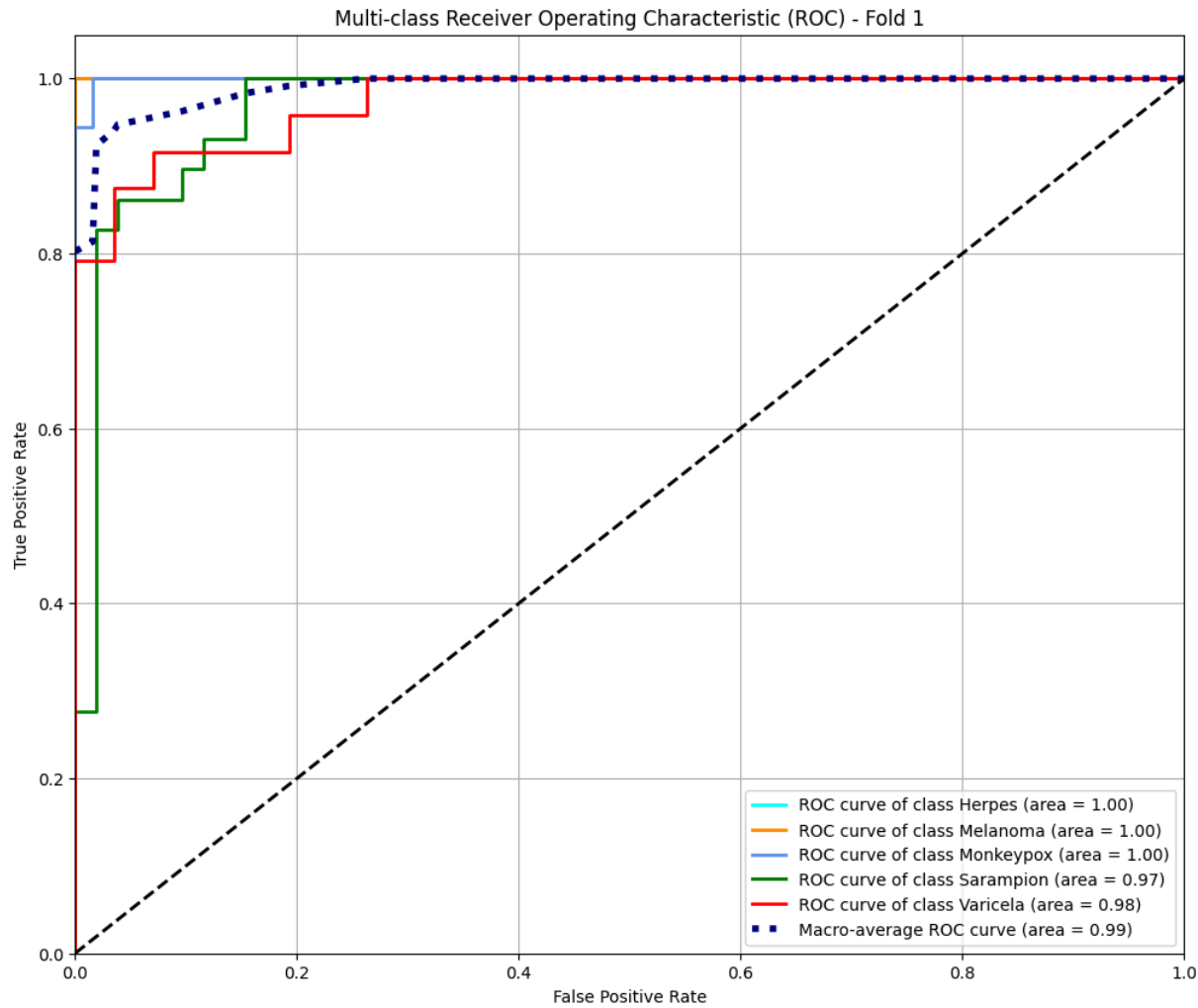
Macro ROC AUC : 0.9889

Cohen's Kappa : 0.8460

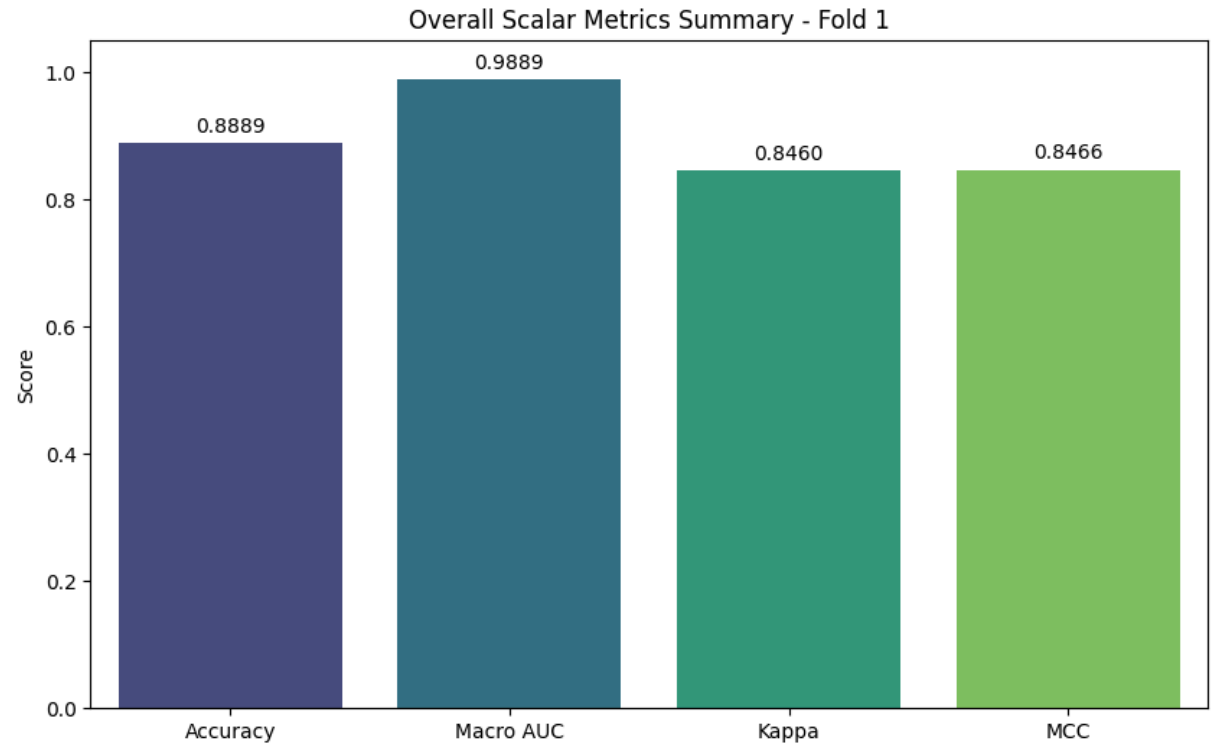
Matthews CorrCoef: 0.8466



Saved normalized confusion matrix to: new_results_Multimodal_DSAF_KFold\confusion_matrix_fold_1.png



Saved ROC curves plot to: new_results_Multimodal_DSAF_KFold\roc_curves_fold_1.png



Saved scalar metrics plot to: new_results_Multimodal_DSAF_KFold\scalar_metrics_fold_1.png

=====

PERFORMANCE ANALYSIS BY: DOMINANT_GENDER

=====

--- Results for Group: Man ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	4
Melanoma	1.00	1.00	1.00	3
Monkeypox	0.94	1.00	0.97	16
Sarampion	0.93	0.82	0.88	17
Varicela	0.80	0.86	0.83	14
accuracy			0.91	54
macro avg	0.93	0.94	0.93	54
weighted avg	0.91	0.91	0.91	54

--- Results for Group: Woman ---

	precision	recall	f1-score	support
Herpes	1.00	0.33	0.50	3
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	0.79	0.92	0.85	12
Varicela	0.90	0.90	0.90	10
accuracy			0.85	27
macro avg	0.74	0.63	0.65	27
weighted avg	0.87	0.85	0.84	27

=====

PERFORMANCE ANALYSIS BY: AGE_GROUP

=====

--- Results for Group: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	0.75	0.75	0.75	4
Varicela	0.80	1.00	0.89	4
accuracy			0.82	11
macro avg	0.51	0.55	0.53	11
weighted avg	0.75	0.82	0.78	11

--- Results for Group: 18-59 ---

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Herpes	1.00	0.83	0.91	6
Melanoma	1.00	1.00	1.00	3
Monkeypox	0.94	1.00	0.97	16
Sarampion	0.88	0.88	0.88	25
Varicela	0.85	0.85	0.85	20
accuracy			0.90	70
macro avg	0.93	0.91	0.92	70
weighted avg	0.90	0.90	0.90	70

=====

PERFORMANCE ANALYSIS BY: DOMINANT_RACE

=====

--- Results for Group: asian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	0.50	1.00	0.67	1
Varicela	1.00	1.00	1.00	4
accuracy			0.89	9
macro avg	0.50	0.60	0.53	9
weighted avg	0.83	0.89	0.85	9

--- Results for Group: black ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	7
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			1.00	9
macro avg	0.60	0.60	0.60	9
weighted avg	1.00	1.00	1.00	9

--- Results for Group: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	0.80	1.00	0.89	4
Varicela	0.00	0.00	0.00	0
accuracy			0.88	8
macro avg	0.36	0.40	0.38	8

weighted avg	0.78	0.88	0.82	8
--------------	------	------	------	---

--- Results for Group: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	1.00	1.00	1.00	4
Varicela	0.00	0.00	0.00	0
accuracy			1.00	5
macro avg	0.40	0.40	0.40	5
weighted avg	1.00	1.00	1.00	5

--- Results for Group: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	1
macro avg	0.20	0.20	0.20	1
weighted avg	1.00	1.00	1.00	1

--- Results for Group: white ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	4
Melanoma	1.00	1.00	1.00	3
Monkeypox	0.75	1.00	0.86	3
Sarampion	0.88	0.79	0.83	19
Varicela	0.81	0.85	0.83	20
accuracy			0.86	49
macro avg	0.89	0.93	0.90	49
weighted avg	0.86	0.86	0.86	49

```
=====
FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.0000 EOD (max TPR gap): 1.0000
Worst pair: 0-17|female|asian vs 18-59|female|white
=====
```

```
=====
FAIRNESS SUMMARY (FILTERED, n >= 15) | WGA: 0.8261 EOD (max TPR gap): 0.1554
Worst pair (among filtered): 18-59|female|white vs 18-59|male|white
Groups used (filtered): ['18-59|female|white', '18-59|male|white']
=====
```

```
#####  
####  
### FOLD 2 ### Train=399, Val=85, Test=81  
#####  
####  
Params: 91.58M | Trainable: 5.92M (6.47%)  
Approx MACs: 17.68 G, Params (THOP): 89.20 M
```

```
=====
```

START TRAINING (Multimodal DSAF) for 200 epochs

```
=====
```

Epoch 1/200	Train 1.6971/0.1128	Val 1.6264/0.2941	LR 2.00e-05
Epoch 2/200	Train 1.5884/0.2982	Val 1.4927/0.3647	LR 3.00e-05
Epoch 3/200	Train 1.4455/0.4286	Val 1.3623/0.4235	LR 4.00e-05
Epoch 4/200	Train 1.3150/0.4962	Val 1.2662/0.5412	LR 5.00e-05
Epoch 5/200	Train 1.2234/0.5714	Val 1.2043/0.5765	LR 6.00e-05
Epoch 6/200	Train 1.1489/0.6366	Val 1.1521/0.6235	LR 7.00e-05
Epoch 7/200	Train 1.0719/0.6867	Val 1.0929/0.6471	LR 8.00e-05
Epoch 8/200	Train 0.9938/0.7293	Val 1.0228/0.6471	LR 9.00e-05
Epoch 9/200	Train 0.9009/0.7945	Val 0.9635/0.7529	LR 1.00e-04
Epoch 10/200	Train 0.8169/0.8521	Val 0.8807/0.8118	LR 1.00e-04
Epoch 11/200	Train 0.7330/0.8847	Val 0.8209/0.8118	LR 1.00e-04
Epoch 12/200	Train 0.6760/0.8972	Val 0.7886/0.8000	LR 1.00e-04
Epoch 13/200	Train 0.6184/0.9198	Val 0.7621/0.8471	LR 9.99e-05
Epoch 14/200	Train 0.5966/0.9123	Val 0.7410/0.8235	LR 9.99e-05
Epoch 15/200	Train 0.5607/0.9273	Val 0.7369/0.8118	LR 9.98e-05
Epoch 16/200	Train 0.5486/0.9398	Val 0.7252/0.8353	LR 9.98e-05
Epoch 17/200	Train 0.5302/0.9474	Val 0.7145/0.8353	LR 9.97e-05
Epoch 18/200	Train 0.5138/0.9574	Val 0.7103/0.8235	LR 9.96e-05
Epoch 19/200	Train 0.5062/0.9674	Val 0.7090/0.8235	LR 9.94e-05
Epoch 20/200	Train 0.4905/0.9624	Val 0.6676/0.8353	LR 9.93e-05
Epoch 21/200	Train 0.4784/0.9774	Val 0.6574/0.8353	LR 9.92e-05
Epoch 22/200	Train 0.4692/0.9749	Val 0.6804/0.8353	LR 9.90e-05
Epoch 23/200	Train 0.4627/0.9799	Val 0.6808/0.8471	LR 9.88e-05
Epoch 24/200	Train 0.4587/0.9825	Val 0.6553/0.8471	LR 9.87e-05
Epoch 25/200	Train 0.4491/0.9850	Val 0.6650/0.8588	LR 9.85e-05
Epoch 26/200	Train 0.4510/0.9825	Val 0.6415/0.8588	LR 9.83e-05
Epoch 27/200	Train 0.4415/0.9850	Val 0.6493/0.8706	LR 9.80e-05
Epoch 28/200	Train 0.4468/0.9825	Val 0.6451/0.8588	LR 9.78e-05
Epoch 29/200	Train 0.4359/0.9900	Val 0.6291/0.8824	LR 9.76e-05
Epoch 30/200	Train 0.4288/0.9950	Val 0.6330/0.8824	LR 9.73e-05
Epoch 31/200	Train 0.4336/0.9900	Val 0.6308/0.8588	LR 9.70e-05
Epoch 32/200	Train 0.4341/0.9825	Val 0.6323/0.8706	LR 9.67e-05
Epoch 33/200	Train 0.4269/0.9900	Val 0.6673/0.8588	LR 9.64e-05
Epoch 34/200	Train 0.4264/0.9950	Val 0.6358/0.8706	LR 9.61e-05
Epoch 35/200	Train 0.4359/0.9925	Val 0.6343/0.8706	LR 9.58e-05
Epoch 36/200	Train 0.4211/0.9900	Val 0.6544/0.8824	LR 9.55e-05
Epoch 37/200	Train 0.4181/0.9975	Val 0.6351/0.8941	LR 9.51e-05
Epoch 38/200	Train 0.4291/0.9925	Val 0.6235/0.8941	LR 9.47e-05
Epoch 39/200	Train 0.4234/0.9925	Val 0.6259/0.8941	LR 9.44e-05
Epoch 40/200	Train 0.4160/0.9975	Val 0.6254/0.8706	LR 9.40e-05
Epoch 41/200	Train 0.4138/0.9975	Val 0.6214/0.8706	LR 9.36e-05
Epoch 42/200	Train 0.4142/0.9950	Val 0.6126/0.8824	LR 9.32e-05
Epoch 43/200	Train 0.4156/0.9925	Val 0.6093/0.8824	LR 9.27e-05
Epoch 44/200	Train 0.4136/0.9975	Val 0.6098/0.8706	LR 9.23e-05

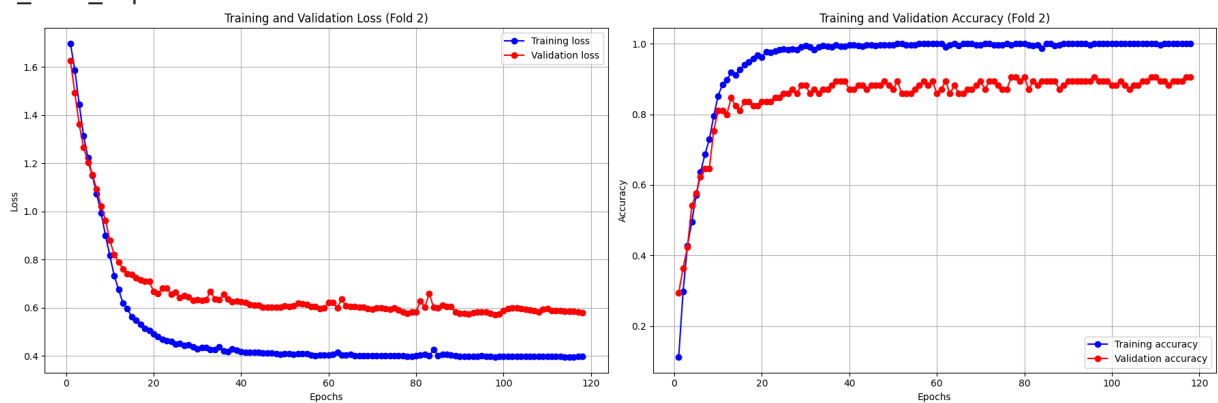
Epoch 45/200		Train 0.4122/0.9975		Val 0.6007/0.8824		LR 9.19e-05
Epoch 46/200		Train 0.4114/0.9950		Val 0.6020/0.8824		LR 9.14e-05
Epoch 47/200		Train 0.4113/0.9975		Val 0.6019/0.8824		LR 9.09e-05
Epoch 48/200		Train 0.4091/0.9975		Val 0.6022/0.8941		LR 9.05e-05
Epoch 49/200		Train 0.4070/0.9975		Val 0.6030/0.8824		LR 9.00e-05
Epoch 50/200		Train 0.4096/0.9975		Val 0.6060/0.8706		LR 8.95e-05
Epoch 51/200		Train 0.4076/1.0000		Val 0.6035/0.8941		LR 8.89e-05
Epoch 52/200		Train 0.4047/1.0000		Val 0.6069/0.8588		LR 8.84e-05
Epoch 53/200		Train 0.4086/0.9975		Val 0.6175/0.8588		LR 8.79e-05
Epoch 54/200		Train 0.4087/0.9975		Val 0.6161/0.8588		LR 8.73e-05
Epoch 55/200		Train 0.4094/0.9975		Val 0.6130/0.8706		LR 8.68e-05
Epoch 56/200		Train 0.4041/1.0000		Val 0.6041/0.8824		LR 8.62e-05
Epoch 57/200		Train 0.4008/1.0000		Val 0.6038/0.8941		LR 8.56e-05
Epoch 58/200		Train 0.4025/1.0000		Val 0.5963/0.8824		LR 8.51e-05
Epoch 59/200		Train 0.4020/1.0000		Val 0.5985/0.8941		LR 8.45e-05
Epoch 60/200		Train 0.4033/1.0000		Val 0.6206/0.8588		LR 8.39e-05
Epoch 61/200		Train 0.4061/1.0000		Val 0.6216/0.8706		LR 8.33e-05
Epoch 62/200		Train 0.4157/0.9900		Val 0.5988/0.8941		LR 8.26e-05
Epoch 63/200		Train 0.4043/0.9975		Val 0.6348/0.8588		LR 8.20e-05
Epoch 64/200		Train 0.4033/1.0000		Val 0.6068/0.8824		LR 8.14e-05
Epoch 65/200		Train 0.4058/0.9950		Val 0.6035/0.8588		LR 8.07e-05
Epoch 66/200		Train 0.3998/1.0000		Val 0.6032/0.8588		LR 8.01e-05
Epoch 67/200		Train 0.4000/1.0000		Val 0.6027/0.8706		LR 7.94e-05
Epoch 68/200		Train 0.4005/1.0000		Val 0.6021/0.8706		LR 7.87e-05
Epoch 69/200		Train 0.4002/0.9975		Val 0.5974/0.8824		LR 7.80e-05
Epoch 70/200		Train 0.4007/0.9975		Val 0.5944/0.8941		LR 7.73e-05
Epoch 71/200		Train 0.4005/1.0000		Val 0.5988/0.8706		LR 7.67e-05
Epoch 72/200		Train 0.4000/1.0000		Val 0.5996/0.8941		LR 7.59e-05
Epoch 73/200		Train 0.4008/0.9975		Val 0.5960/0.8941		LR 7.52e-05
Epoch 74/200		Train 0.4007/0.9975		Val 0.5939/0.8824		LR 7.45e-05
Epoch 75/200		Train 0.4005/0.9975		Val 0.5994/0.8706		LR 7.38e-05
Epoch 76/200		Train 0.3994/1.0000		Val 0.5917/0.8706		LR 7.31e-05
Epoch 77/200		Train 0.4014/0.9975		Val 0.5806/0.9059		LR 7.23e-05
Epoch 78/200		Train 0.3981/1.0000		Val 0.5771/0.9059		LR 7.16e-05
Epoch 79/200		Train 0.3977/1.0000		Val 0.5812/0.8941		LR 7.08e-05
Epoch 80/200		Train 0.4007/1.0000		Val 0.5821/0.9059		LR 7.01e-05
Epoch 81/200		Train 0.4028/0.9975		Val 0.6286/0.8706		LR 6.93e-05
Epoch 82/200		Train 0.4056/0.9950		Val 0.6015/0.8941		LR 6.86e-05
Epoch 83/200		Train 0.3996/0.9975		Val 0.6598/0.8824		LR 6.78e-05
Epoch 84/200		Train 0.4247/0.9875		Val 0.6008/0.8941		LR 6.70e-05
Epoch 85/200		Train 0.4014/1.0000		Val 0.5980/0.8941		LR 6.62e-05
Epoch 86/200		Train 0.4046/1.0000		Val 0.6114/0.8941		LR 6.55e-05
Epoch 87/200		Train 0.4051/0.9950		Val 0.6033/0.8941		LR 6.47e-05
Epoch 88/200		Train 0.4029/0.9975		Val 0.6053/0.8706		LR 6.39e-05
Epoch 89/200		Train 0.4015/1.0000		Val 0.5820/0.8824		LR 6.31e-05
Epoch 90/200		Train 0.3976/1.0000		Val 0.5753/0.8941		LR 6.23e-05
Epoch 91/200		Train 0.3979/1.0000		Val 0.5762/0.8941		LR 6.15e-05
Epoch 92/200		Train 0.3984/1.0000		Val 0.5746/0.8941		LR 6.07e-05
Epoch 93/200		Train 0.3975/1.0000		Val 0.5777/0.8941		LR 5.99e-05
Epoch 94/200		Train 0.3969/1.0000		Val 0.5815/0.8941		LR 5.90e-05
Epoch 95/200		Train 0.4002/0.9975		Val 0.5820/0.8941		LR 5.82e-05
Epoch 96/200		Train 0.3963/1.0000		Val 0.5805/0.9059		LR 5.74e-05
Epoch 97/200		Train 0.3968/1.0000		Val 0.5751/0.8941		LR 5.66e-05
Epoch 98/200		Train 0.3958/1.0000		Val 0.5719/0.8941		LR 5.58e-05
Epoch 99/200		Train 0.3961/1.0000		Val 0.5736/0.8941		LR 5.50e-05
Epoch 100/200		Train 0.3968/1.0000		Val 0.5882/0.8824		LR 5.41e-05

Epoch 101/200	Train 0.3974/1.0000	Val 0.5966/0.8824	LR 5.33e-05
Epoch 102/200	Train 0.3970/1.0000	Val 0.5997/0.8941	LR 5.25e-05
Epoch 103/200	Train 0.3963/1.0000	Val 0.6000/0.8824	LR 5.17e-05
Epoch 104/200	Train 0.3964/1.0000	Val 0.5975/0.8706	LR 5.08e-05
Epoch 105/200	Train 0.3966/1.0000	Val 0.5920/0.8824	LR 5.00e-05
Epoch 106/200	Train 0.3966/1.0000	Val 0.5914/0.8824	LR 4.92e-05
Epoch 107/200	Train 0.3968/1.0000	Val 0.5872/0.8941	LR 4.83e-05
Epoch 108/200	Train 0.3971/1.0000	Val 0.5823/0.8941	LR 4.75e-05
Epoch 109/200	Train 0.3964/1.0000	Val 0.5926/0.9059	LR 4.67e-05
Epoch 110/200	Train 0.3968/1.0000	Val 0.5947/0.9059	LR 4.59e-05
Epoch 111/200	Train 0.3985/0.9975	Val 0.5872/0.8941	LR 4.50e-05
Epoch 112/200	Train 0.3963/1.0000	Val 0.5889/0.8941	LR 4.42e-05
Epoch 113/200	Train 0.3967/1.0000	Val 0.5884/0.8824	LR 4.34e-05
Epoch 114/200	Train 0.3956/1.0000	Val 0.5859/0.8941	LR 4.26e-05
Epoch 115/200	Train 0.3950/1.0000	Val 0.5844/0.8941	LR 4.18e-05
Epoch 116/200	Train 0.3956/1.0000	Val 0.5850/0.8941	LR 4.10e-05
Epoch 117/200	Train 0.3963/1.0000	Val 0.5824/0.9059	LR 4.01e-05
Epoch 118/200	Train 0.3959/1.0000	Val 0.5796/0.9059	LR 3.93e-05

Early stopping.

Training complete in 9.2 minutes. Best Val Loss 0.5719

Best model for fold saved to: new_results_Multimodal_DSAF_KFold\best_multimodal_dsaf_fold_2.pth



Saved training history plot to: new_results_Multimodal_DSAF_KFold\training_history_fold_2.png

=====

OVERALL PERFORMANCE (FOLD 2)

=====

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	7
Melanoma	1.00	1.00	1.00	3
Monkeypox	1.00	0.89	0.94	18
Sarampion	0.97	0.97	0.97	29
Varicela	0.88	0.96	0.92	24
accuracy			0.95	81
macro avg	0.97	0.96	0.97	81
weighted avg	0.95	0.95	0.95	81

--- Overall Scalar Metrics ---

Accuracy : 0.9506

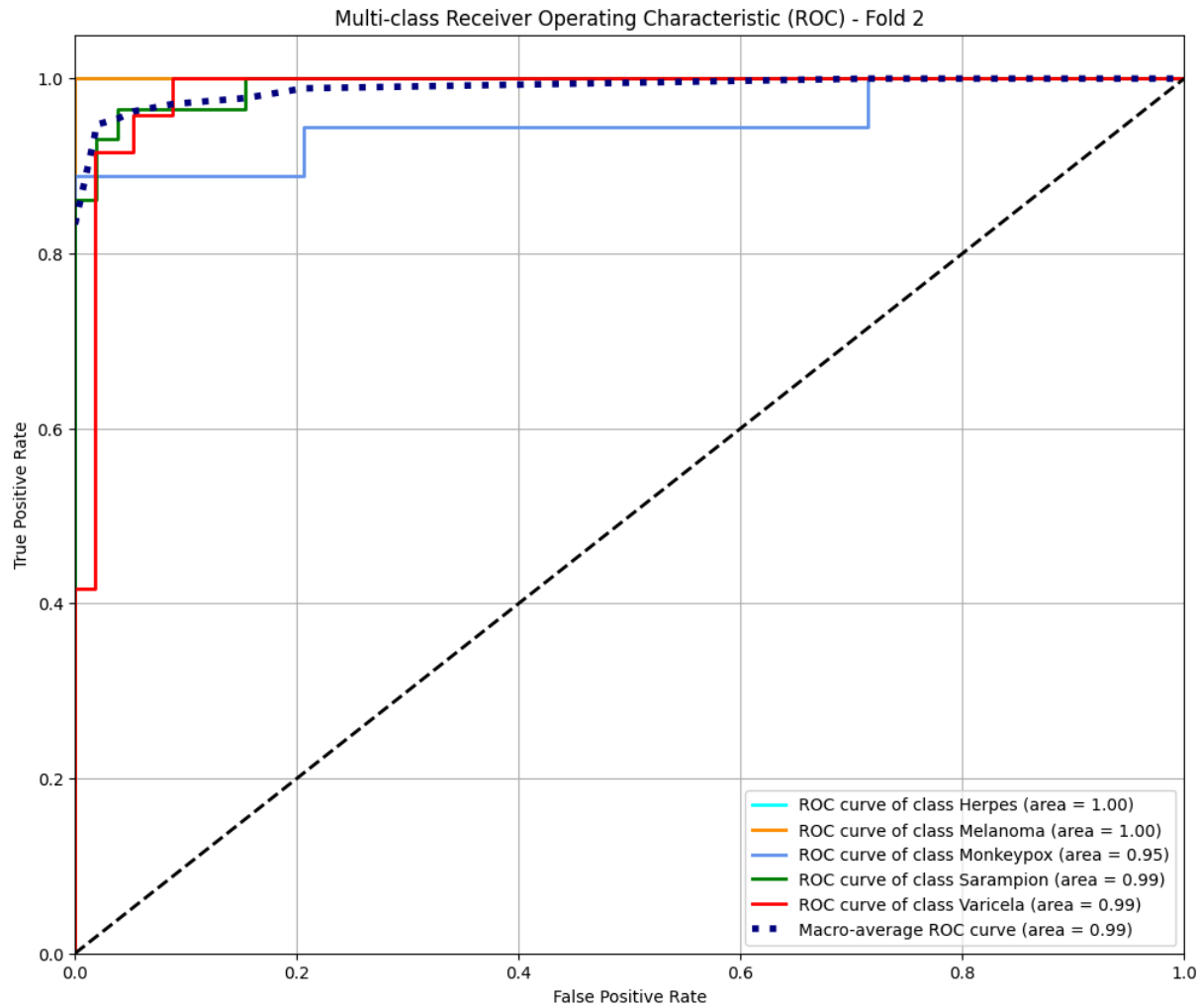
Macro ROC AUC : 0.9853

Cohen's Kappa : 0.9318

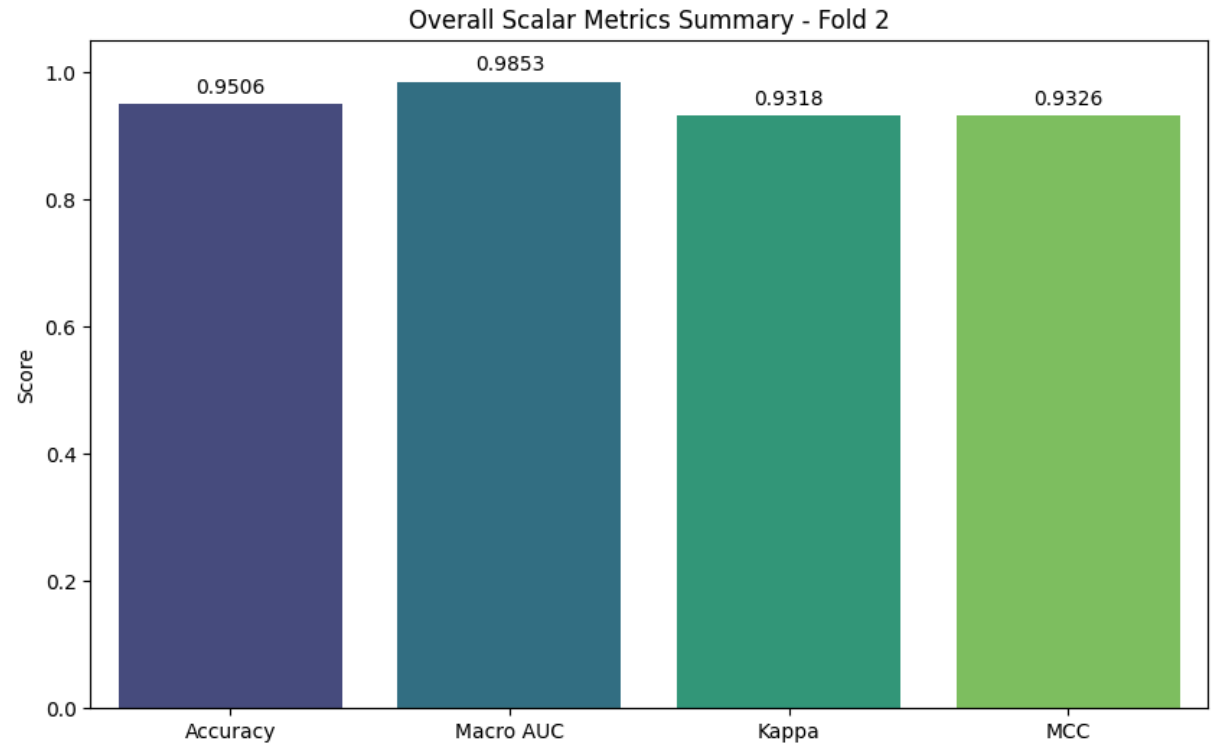
Matthews CorrCoef: 0.9326



Saved normalized confusion matrix to: new_results_Multimodal_DSAF_KFold\confusion_matrix_fold_2.png



Saved ROC curves plot to: new_results_Multimodal_DSAF_KFold\roc_curves_fold_2.png



Saved scalar metrics plot to: new_results_Multimodal_DSAF_KFold\scalar_metrics_fold_2.png

=====

PERFORMANCE ANALYSIS BY: DOMINANT_GENDER

=====

--- Results for Group: Man ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	5
Melanoma	1.00	1.00	1.00	3
Monkeypox	1.00	1.00	1.00	14
Sarampion	0.93	0.93	0.93	15
Varicela	0.91	0.91	0.91	11
accuracy			0.96	48
macro avg	0.97	0.97	0.97	48
weighted avg	0.96	0.96	0.96	48

--- Results for Group: Woman ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	2
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	0.50	0.67	4
Sarampion	1.00	1.00	1.00	14
Varicela	0.87	1.00	0.93	13
accuracy			0.94	33
macro avg	0.77	0.70	0.72	33
weighted avg	0.95	0.94	0.93	33

=====

PERFORMANCE ANALYSIS BY: AGE_GROUP

=====

--- Results for Group: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	1
Sarampion	1.00	1.00	1.00	3
Varicela	0.80	1.00	0.89	4
accuracy			0.88	8
macro avg	0.36	0.40	0.38	8
weighted avg	0.78	0.88	0.82	8

--- Results for Group: 18-59 ---

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Herpes	1.00	1.00	1.00	6
Melanoma	1.00	1.00	1.00	3
Monkeypox	1.00	0.94	0.97	17
Sarampion	0.96	0.96	0.96	26
Varicela	0.90	0.95	0.93	20
accuracy			0.96	72
macro avg	0.97	0.97	0.97	72
weighted avg	0.96	0.96	0.96	72

--- Results for Group: 60+ ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	1
macro avg	0.20	0.20	0.20	1
weighted avg	1.00	1.00	1.00	1

=====

PERFORMANCE ANALYSIS BY: DOMINANT_RACE

=====

--- Results for Group: asian ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	3
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	4
Sarampion	1.00	1.00	1.00	2
Varicela	1.00	1.00	1.00	2
accuracy			1.00	11
macro avg	0.80	0.80	0.80	11
weighted avg	1.00	1.00	1.00	11

--- Results for Group: black ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	2
macro avg	0.20	0.20	0.20	2

weighted avg	1.00	1.00	1.00	2
--------------	------	------	------	---

--- Results for Group: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	2
Varicela	0.00	0.00	0.00	0
accuracy			1.00	4
macro avg	0.40	0.40	0.40	4
weighted avg	1.00	1.00	1.00	4

--- Results for Group: latino hispanic ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	0.67	0.80	3
Sarampion	1.00	1.00	1.00	7
Varicela	0.00	0.00	0.00	0
accuracy			0.91	11
macro avg	0.60	0.53	0.56	11
weighted avg	1.00	0.91	0.95	11

--- Results for Group: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	0.00	0.00	0.00	1
Varicela	0.00	0.00	0.00	1
accuracy			0.60	5
macro avg	0.20	0.20	0.20	5
weighted avg	0.60	0.60	0.60	5

--- Results for Group: white ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	3
Melanoma	1.00	1.00	1.00	3
Monkeypox	1.00	0.75	0.86	4
Sarampion	1.00	1.00	1.00	17
Varicela	0.95	1.00	0.98	21
accuracy			0.98	48
macro avg	0.99	0.95	0.97	48

weighted avg 0.98 0.98 0.98 48

```
=====
FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.6000 EOD (max TPR gap): 1.0000
Worst pair: 0-17|female|white vs 18-59|female|indian
=====
```

```
=====
FAIRNESS SUMMARY (FILTERED, n >= 15) | WGA: 1.0000 EOD (max TPR gap): 0.0000
Groups used (filtered): ['18-59|female|white', '18-59|male|white']
=====
```

```
#####
####
### FOLD 3 ### Train=399, Val=85, Test=81
#####
####
Params: 91.58M | Trainable: 5.92M (6.47%)
Approx MACs: 17.68 G, Params (THOP): 89.20 M
```

```
=====
START TRAINING (Multimodal DSAF) for 200 epochs
=====
```

Epoch 1/200	Train 1.6483/0.1704	Val 1.5883/0.2706	LR 2.00e-05
Epoch 2/200	Train 1.5639/0.3208	Val 1.4596/0.5765	LR 3.00e-05
Epoch 3/200	Train 1.4438/0.5564	Val 1.3252/0.6471	LR 4.00e-05
Epoch 4/200	Train 1.3311/0.5990	Val 1.2104/0.6941	LR 5.00e-05
Epoch 5/200	Train 1.2340/0.6065	Val 1.1180/0.6941	LR 6.00e-05
Epoch 6/200	Train 1.1484/0.6341	Val 1.0335/0.7176	LR 7.00e-05
Epoch 7/200	Train 1.0612/0.6491	Val 0.9492/0.7529	LR 8.00e-05
Epoch 8/200	Train 0.9833/0.7193	Val 0.8745/0.8235	LR 9.00e-05
Epoch 9/200	Train 0.9079/0.7694	Val 0.8055/0.8588	LR 1.00e-04
Epoch 10/200	Train 0.8163/0.8371	Val 0.7407/0.8824	LR 1.00e-04
Epoch 11/200	Train 0.7404/0.8747	Val 0.7120/0.8824	LR 1.00e-04
Epoch 12/200	Train 0.6954/0.8722	Val 0.6840/0.8706	LR 1.00e-04
Epoch 13/200	Train 0.6617/0.9073	Val 0.6686/0.8706	LR 9.99e-05
Epoch 14/200	Train 0.6267/0.8972	Val 0.6350/0.9059	LR 9.99e-05
Epoch 15/200	Train 0.6004/0.9148	Val 0.6150/0.9059	LR 9.98e-05
Epoch 16/200	Train 0.5901/0.9248	Val 0.6037/0.9176	LR 9.98e-05
Epoch 17/200	Train 0.5590/0.9449	Val 0.5862/0.8941	LR 9.97e-05
Epoch 18/200	Train 0.5440/0.9449	Val 0.5794/0.9059	LR 9.96e-05
Epoch 19/200	Train 0.5311/0.9499	Val 0.5771/0.9412	LR 9.94e-05
Epoch 20/200	Train 0.5155/0.9474	Val 0.5640/0.9412	LR 9.93e-05
Epoch 21/200	Train 0.5013/0.9624	Val 0.5494/0.9412	LR 9.92e-05
Epoch 22/200	Train 0.4902/0.9649	Val 0.5386/0.9294	LR 9.90e-05
Epoch 23/200	Train 0.4797/0.9724	Val 0.5318/0.9294	LR 9.88e-05
Epoch 24/200	Train 0.4719/0.9774	Val 0.5310/0.9294	LR 9.87e-05
Epoch 25/200	Train 0.4673/0.9825	Val 0.5318/0.9294	LR 9.85e-05
Epoch 26/200	Train 0.4631/0.9799	Val 0.5297/0.9294	LR 9.83e-05
Epoch 27/200	Train 0.4671/0.9724	Val 0.5071/0.9412	LR 9.80e-05
Epoch 28/200	Train 0.4531/0.9825	Val 0.5062/0.9412	LR 9.78e-05
Epoch 29/200	Train 0.4511/0.9799	Val 0.5043/0.9412	LR 9.76e-05
Epoch 30/200	Train 0.4512/0.9850	Val 0.4822/0.9529	LR 9.73e-05
Epoch 31/200	Train 0.4520/0.9799	Val 0.4820/0.9529	LR 9.70e-05

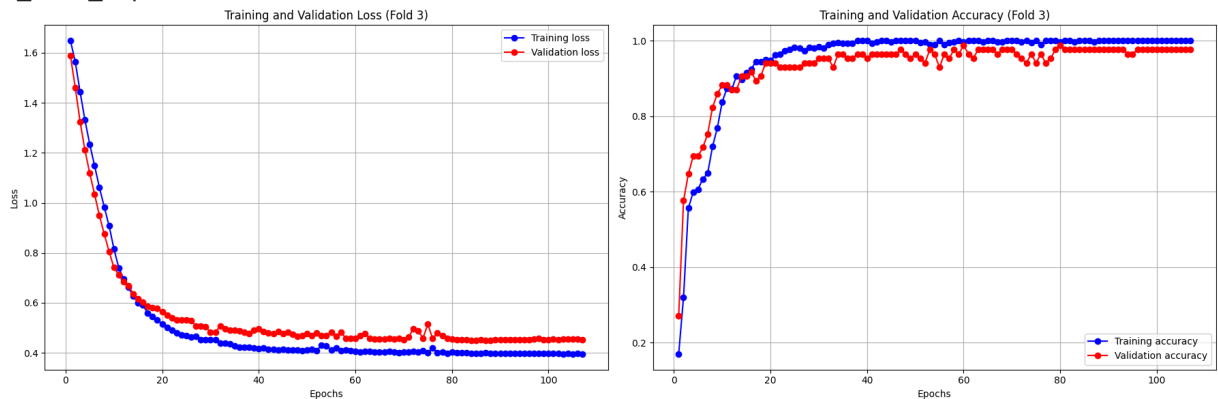
Epoch 32/200		Train 0.4394/0.9900		Val 0.5055/0.9529		LR 9.67e-05
Epoch 33/200		Train 0.4397/0.9925		Val 0.4957/0.9294		LR 9.64e-05
Epoch 34/200		Train 0.4352/0.9950		Val 0.4913/0.9647		LR 9.61e-05
Epoch 35/200		Train 0.4280/0.9925		Val 0.4899/0.9647		LR 9.58e-05
Epoch 36/200		Train 0.4228/0.9925		Val 0.4885/0.9529		LR 9.55e-05
Epoch 37/200		Train 0.4218/0.9925		Val 0.4827/0.9529		LR 9.51e-05
Epoch 38/200		Train 0.4214/1.0000		Val 0.4769/0.9647		LR 9.47e-05
Epoch 39/200		Train 0.4183/1.0000		Val 0.4902/0.9647		LR 9.44e-05
Epoch 40/200		Train 0.4161/1.0000		Val 0.4959/0.9529		LR 9.40e-05
Epoch 41/200		Train 0.4190/0.9925		Val 0.4857/0.9647		LR 9.36e-05
Epoch 42/200		Train 0.4144/0.9975		Val 0.4787/0.9647		LR 9.32e-05
Epoch 43/200		Train 0.4127/1.0000		Val 0.4765/0.9647		LR 9.27e-05
Epoch 44/200		Train 0.4109/1.0000		Val 0.4838/0.9647		LR 9.23e-05
Epoch 45/200		Train 0.4130/0.9975		Val 0.4761/0.9647		LR 9.19e-05
Epoch 46/200		Train 0.4122/1.0000		Val 0.4816/0.9647		LR 9.14e-05
Epoch 47/200		Train 0.4105/1.0000		Val 0.4734/0.9765		LR 9.09e-05
Epoch 48/200		Train 0.4099/1.0000		Val 0.4661/0.9647		LR 9.05e-05
Epoch 49/200		Train 0.4075/1.0000		Val 0.4686/0.9529		LR 9.00e-05
Epoch 50/200		Train 0.4099/1.0000		Val 0.4756/0.9647		LR 8.95e-05
Epoch 51/200		Train 0.4151/0.9950		Val 0.4680/0.9529		LR 8.89e-05
Epoch 52/200		Train 0.4097/0.9975		Val 0.4783/0.9412		LR 8.84e-05
Epoch 53/200		Train 0.4295/0.9900		Val 0.4698/0.9765		LR 8.79e-05
Epoch 54/200		Train 0.4273/0.9900		Val 0.4691/0.9647		LR 8.73e-05
Epoch 55/200		Train 0.4112/1.0000		Val 0.4818/0.9294		LR 8.68e-05
Epoch 56/200		Train 0.4182/0.9900		Val 0.4659/0.9647		LR 8.62e-05
Epoch 57/200		Train 0.4093/0.9950		Val 0.4833/0.9529		LR 8.56e-05
Epoch 58/200		Train 0.4118/0.9975		Val 0.4580/0.9765		LR 8.51e-05
Epoch 59/200		Train 0.4078/1.0000		Val 0.4578/0.9647		LR 8.45e-05
Epoch 60/200		Train 0.4068/0.9975		Val 0.4563/0.9882		LR 8.39e-05
Epoch 61/200		Train 0.4027/1.0000		Val 0.4698/0.9647		LR 8.33e-05
Epoch 62/200		Train 0.4061/1.0000		Val 0.4770/0.9529		LR 8.26e-05
Epoch 63/200		Train 0.4068/1.0000		Val 0.4585/0.9765		LR 8.20e-05
Epoch 64/200		Train 0.4033/0.9975		Val 0.4548/0.9765		LR 8.14e-05
Epoch 65/200		Train 0.4021/1.0000		Val 0.4545/0.9765		LR 8.07e-05
Epoch 66/200		Train 0.4025/1.0000		Val 0.4543/0.9765		LR 8.01e-05
Epoch 67/200		Train 0.4048/0.9975		Val 0.4569/0.9647		LR 7.94e-05
Epoch 68/200		Train 0.4023/0.9975		Val 0.4556/0.9765		LR 7.87e-05
Epoch 69/200		Train 0.4012/1.0000		Val 0.4575/0.9765		LR 7.80e-05
Epoch 70/200		Train 0.4017/1.0000		Val 0.4513/0.9765		LR 7.73e-05
Epoch 71/200		Train 0.4017/1.0000		Val 0.4617/0.9647		LR 7.67e-05
Epoch 72/200		Train 0.4061/0.9975		Val 0.4960/0.9529		LR 7.59e-05
Epoch 73/200		Train 0.4030/1.0000		Val 0.4872/0.9412		LR 7.52e-05
Epoch 74/200		Train 0.4087/0.9950		Val 0.4586/0.9647		LR 7.45e-05
Epoch 75/200		Train 0.3999/1.0000		Val 0.5141/0.9412		LR 7.38e-05
Epoch 76/200		Train 0.4181/0.9900		Val 0.4573/0.9647		LR 7.31e-05
Epoch 77/200		Train 0.4005/1.0000		Val 0.4782/0.9412		LR 7.23e-05
Epoch 78/200		Train 0.4030/1.0000		Val 0.4673/0.9529		LR 7.16e-05
Epoch 79/200		Train 0.3985/1.0000		Val 0.4575/0.9765		LR 7.08e-05
Epoch 80/200		Train 0.4018/0.9975		Val 0.4552/0.9882		LR 7.01e-05
Epoch 81/200		Train 0.3991/1.0000		Val 0.4527/0.9765		LR 6.93e-05
Epoch 82/200		Train 0.4000/1.0000		Val 0.4511/0.9765		LR 6.86e-05
Epoch 83/200		Train 0.3999/0.9975		Val 0.4508/0.9765		LR 6.78e-05
Epoch 84/200		Train 0.3981/1.0000		Val 0.4495/0.9765		LR 6.70e-05
Epoch 85/200		Train 0.3982/1.0000		Val 0.4497/0.9765		LR 6.62e-05
Epoch 86/200		Train 0.3973/1.0000		Val 0.4509/0.9765		LR 6.55e-05
Epoch 87/200		Train 0.3995/0.9975		Val 0.4483/0.9765		LR 6.47e-05

Epoch 88/200	Train 0.3971/1.0000	Val 0.4490/0.9765	LR 6.39e-05
Epoch 89/200	Train 0.3972/1.0000	Val 0.4513/0.9765	LR 6.31e-05
Epoch 90/200	Train 0.3972/1.0000	Val 0.4523/0.9765	LR 6.23e-05
Epoch 91/200	Train 0.3970/1.0000	Val 0.4533/0.9765	LR 6.15e-05
Epoch 92/200	Train 0.3979/1.0000	Val 0.4532/0.9765	LR 6.07e-05
Epoch 93/200	Train 0.3973/1.0000	Val 0.4510/0.9765	LR 5.99e-05
Epoch 94/200	Train 0.3985/1.0000	Val 0.4513/0.9647	LR 5.90e-05
Epoch 95/200	Train 0.3985/1.0000	Val 0.4525/0.9647	LR 5.82e-05
Epoch 96/200	Train 0.3966/1.0000	Val 0.4527/0.9765	LR 5.74e-05
Epoch 97/200	Train 0.3969/1.0000	Val 0.4558/0.9765	LR 5.66e-05
Epoch 98/200	Train 0.3979/1.0000	Val 0.4564/0.9765	LR 5.58e-05
Epoch 99/200	Train 0.3967/1.0000	Val 0.4532/0.9765	LR 5.50e-05
Epoch 100/200	Train 0.3970/1.0000	Val 0.4521/0.9765	LR 5.41e-05
Epoch 101/200	Train 0.3974/1.0000	Val 0.4561/0.9765	LR 5.33e-05
Epoch 102/200	Train 0.3970/1.0000	Val 0.4534/0.9765	LR 5.25e-05
Epoch 103/200	Train 0.3960/1.0000	Val 0.4540/0.9765	LR 5.17e-05
Epoch 104/200	Train 0.3964/1.0000	Val 0.4539/0.9765	LR 5.08e-05
Epoch 105/200	Train 0.3953/1.0000	Val 0.4554/0.9765	LR 5.00e-05
Epoch 106/200	Train 0.3969/1.0000	Val 0.4554/0.9765	LR 4.92e-05
Epoch 107/200	Train 0.3955/1.0000	Val 0.4526/0.9765	LR 4.83e-05

Early stopping.

Training complete in 7.8 minutes. Best Val Loss 0.4483

Best model for fold saved to: new_results_Multimodal_DSAF_KFold\best_multimodal_dsaf_fold_3.pth



Saved training history plot to: new_results_Multimodal_DSAF_KFold\training_history_fold_3.png

=====

OVERALL PERFORMANCE (FOLD 3)

=====

	precision	recall	f1-score	support
Herpes	1.00	0.71	0.83	7
Melanoma	1.00	1.00	1.00	3
Monkeypox	0.95	1.00	0.97	18
Sarampion	1.00	0.97	0.98	29
Varicela	0.92	1.00	0.96	24
accuracy			0.96	81
macro avg	0.97	0.94	0.95	81
weighted avg	0.97	0.96	0.96	81

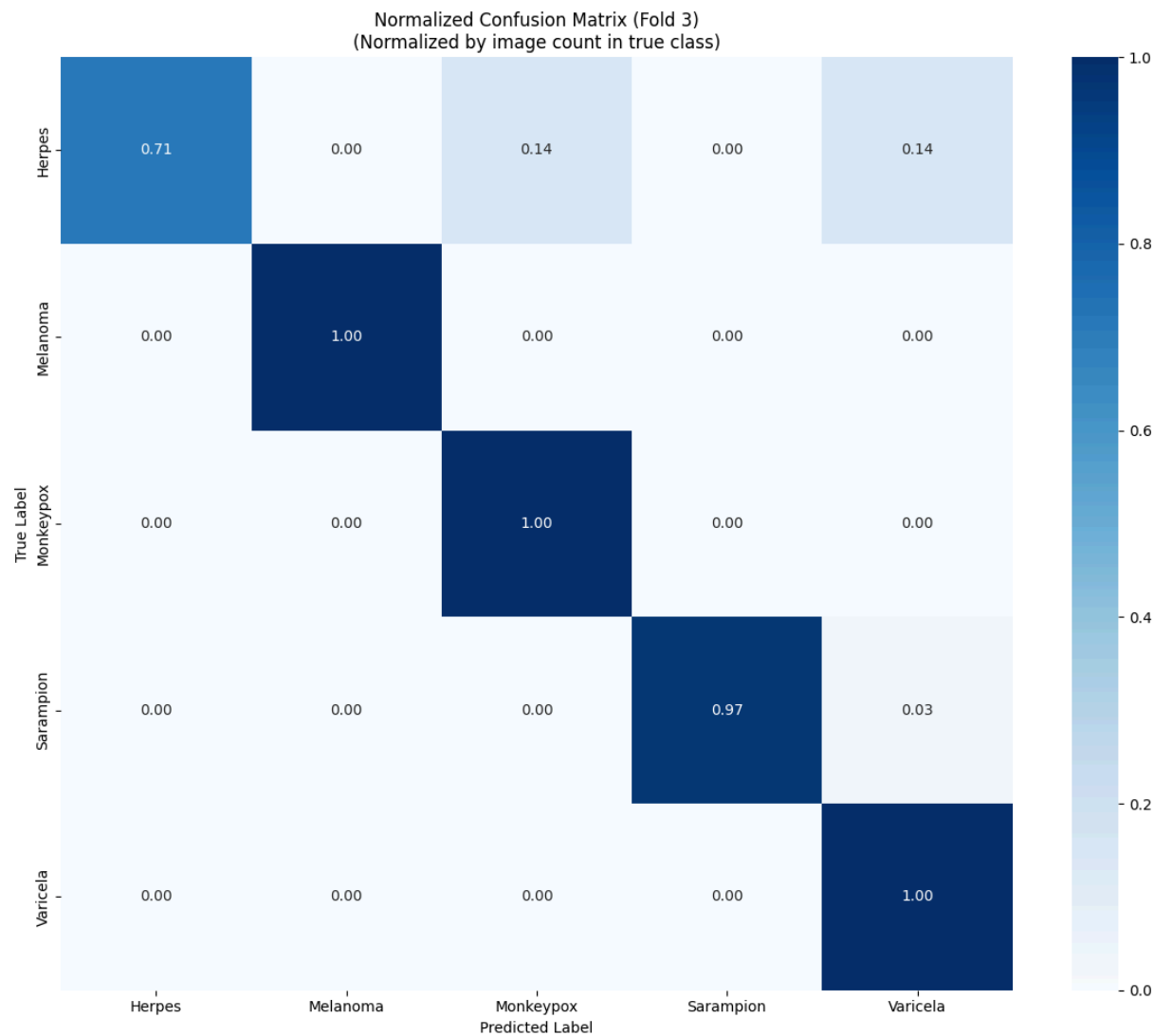
--- Overall Scalar Metrics ---

Accuracy : 0.9630

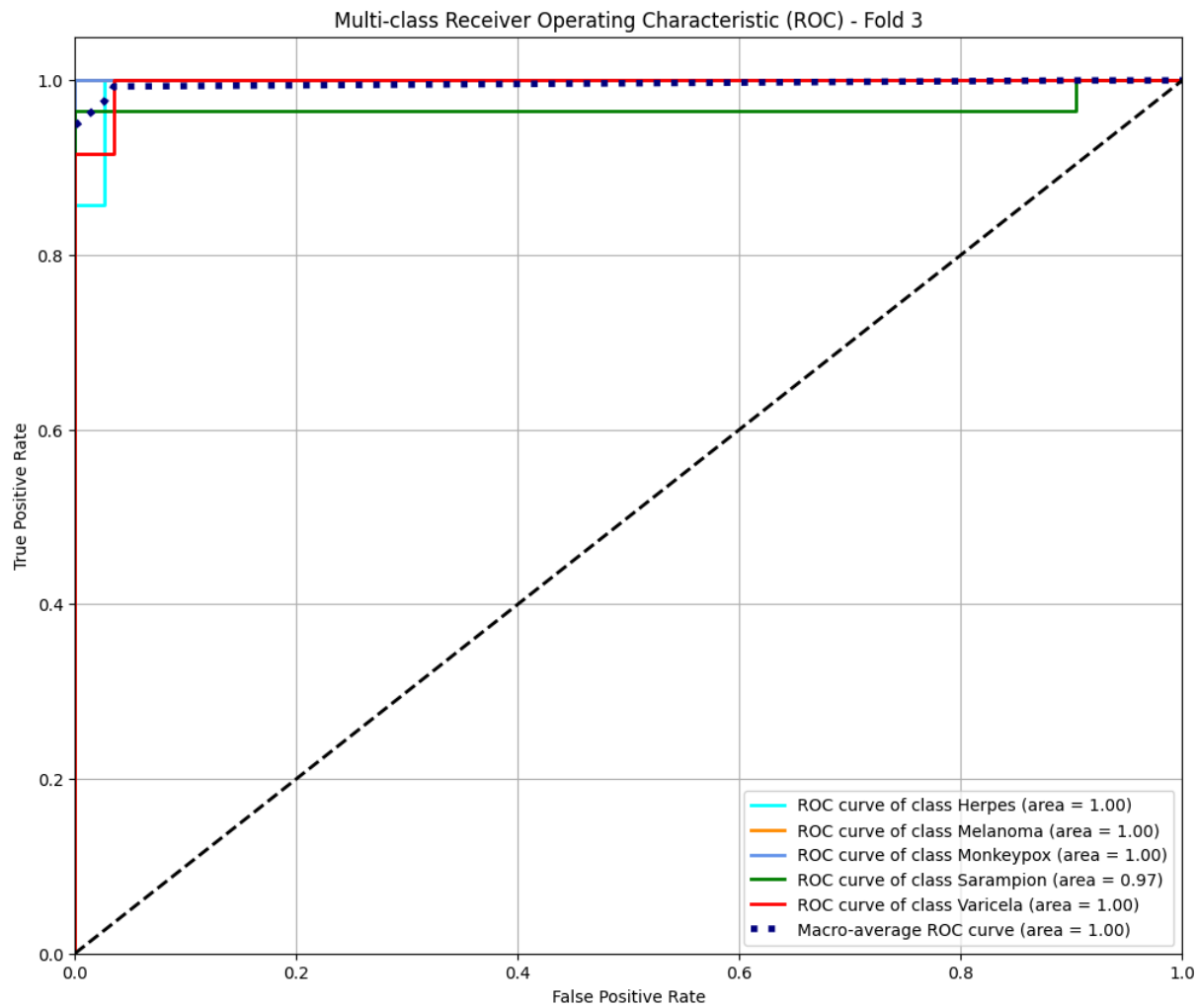
Macro ROC AUC : 0.9924

Cohen's Kappa : 0.9487

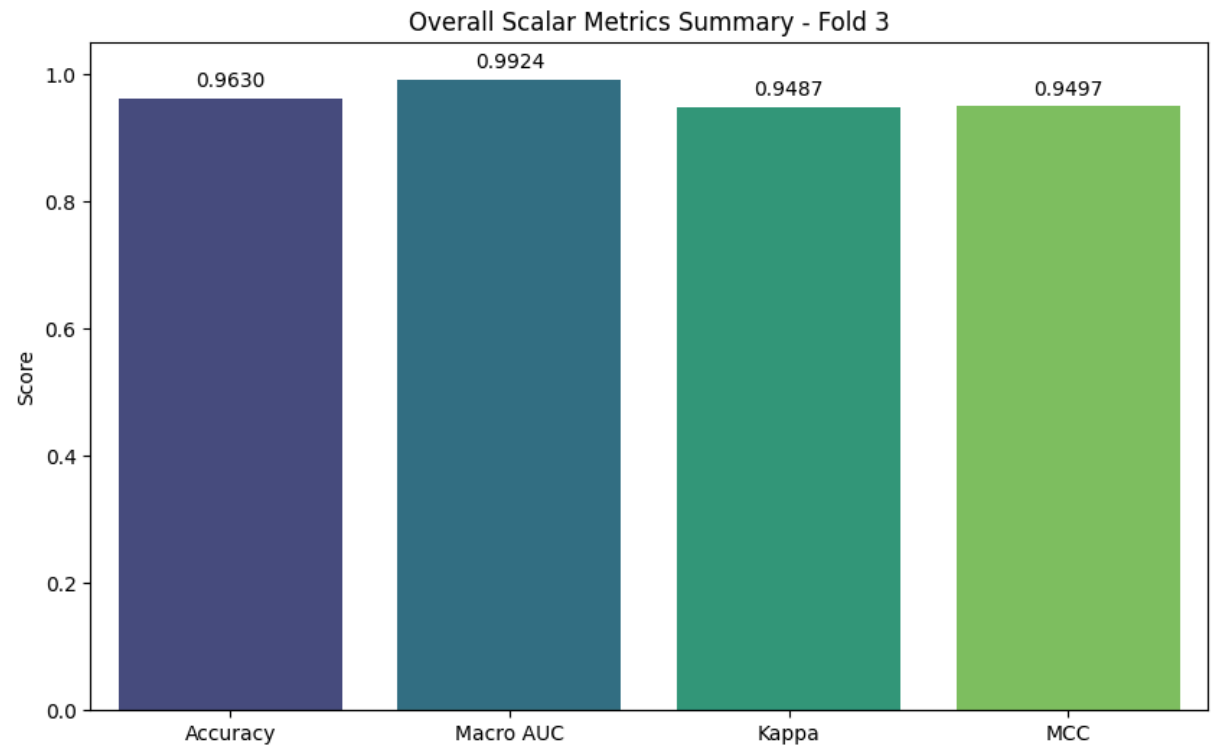
Matthews CorrCoef: 0.9497



Saved normalized confusion matrix to: new_results_Multimodal_DSAF_KFold\confusion_matrix_fold_3.png



Saved ROC curves plot to: new_results_Multimodal_DSAF_KFold\roc_curves_fold_3.png



Saved scalar metrics plot to: new_results_Multimodal_DSAF_KFold\scalar_metrics_fold_3.png

=====

PERFORMANCE ANALYSIS BY: DOMINANT_GENDER

=====

--- Results for Group: Man ---

	precision	recall	f1-score	support
Herpes	1.00	0.67	0.80	6
Melanoma	1.00	1.00	1.00	3
Monkeypox	0.94	1.00	0.97	15
Sarampion	1.00	1.00	1.00	7
Varicela	0.93	1.00	0.97	14
accuracy			0.96	45
macro avg	0.97	0.93	0.95	45
weighted avg	0.96	0.96	0.95	45

--- Results for Group: Woman ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	1.00	0.95	0.98	22
Varicela	0.91	1.00	0.95	10
accuracy			0.97	36
macro avg	0.78	0.79	0.79	36
weighted avg	0.97	0.97	0.97	36

=====

PERFORMANCE ANALYSIS BY: AGE_GROUP

=====

--- Results for Group: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	1.00	0.67	0.80	3
Varicela	0.83	1.00	0.91	5
accuracy			0.88	8
macro avg	0.37	0.33	0.34	8
weighted avg	0.90	0.88	0.87	8

--- Results for Group: 18-59 ---

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Herpes	1.00	0.71	0.83	7
Melanoma	1.00	1.00	1.00	3
Monkeypox	0.95	1.00	0.97	18
Sarampion	1.00	1.00	1.00	26
Varicela	0.95	1.00	0.97	19
accuracy			0.97	73
macro avg	0.98	0.94	0.96	73
weighted avg	0.97	0.97	0.97	73

=====

PERFORMANCE ANALYSIS BY: DOMINANT_RACE

=====

--- Results for Group: asian ---

	precision	recall	f1-score	support
Herpes	1.00	0.50	0.67	2
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	1.00	1.00	1.00	4
Varicela	0.67	1.00	0.80	2
accuracy			0.91	11
macro avg	0.73	0.70	0.69	11
weighted avg	0.94	0.91	0.90	11

--- Results for Group: black ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	3
macro avg	0.20	0.20	0.20	3
weighted avg	1.00	1.00	1.00	3

--- Results for Group: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.80	1.00	0.89	4
Sarampion	0.00	0.00	0.00	1
Varicela	0.00	0.00	0.00	0
accuracy			0.67	6
macro avg	0.16	0.20	0.18	6

weighted avg	0.53	0.67	0.59	6
--------------	------	------	------	---

--- Results for Group: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	1.00	1.00	1.00	1
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	7
Varicela	0.00	0.00	0.00	0
accuracy			1.00	10
macro avg	0.60	0.60	0.60	10
weighted avg	1.00	1.00	1.00	10

--- Results for Group: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	1.00	1.00	1.00	1
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			1.00	4
macro avg	0.60	0.60	0.60	4
weighted avg	1.00	1.00	1.00	4

--- Results for Group: white ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	4
Melanoma	1.00	1.00	1.00	1
Monkeypox	1.00	1.00	1.00	4
Sarampion	1.00	1.00	1.00	16
Varicela	1.00	1.00	1.00	22
accuracy			1.00	47
macro avg	1.00	1.00	1.00	47
weighted avg	1.00	1.00	1.00	47

```
=====
FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.0000 EOD (max TPR gap): 1.0000
Worst pair: 0-17|female|indian vs 0-17|female|white
=====
```

```
=====
FAIRNESS SUMMARY (FILTERED, n >= 15) | WGA: 1.0000 EOD (max TPR gap): 0.0000
Groups used (filtered): ['18-59|female|white', '18-59|male|white']
=====
```

```
#####  
####  
### FOLD 4 ### Train=399, Val=85, Test=81  
#####  
####  
Params: 91.58M | Trainable: 5.92M (6.47%)  
Approx MACs: 17.68 G, Params (THOP): 89.20 M
```

```
=====
```

START TRAINING (Multimodal DSAF) for 200 epochs

```
=====
```

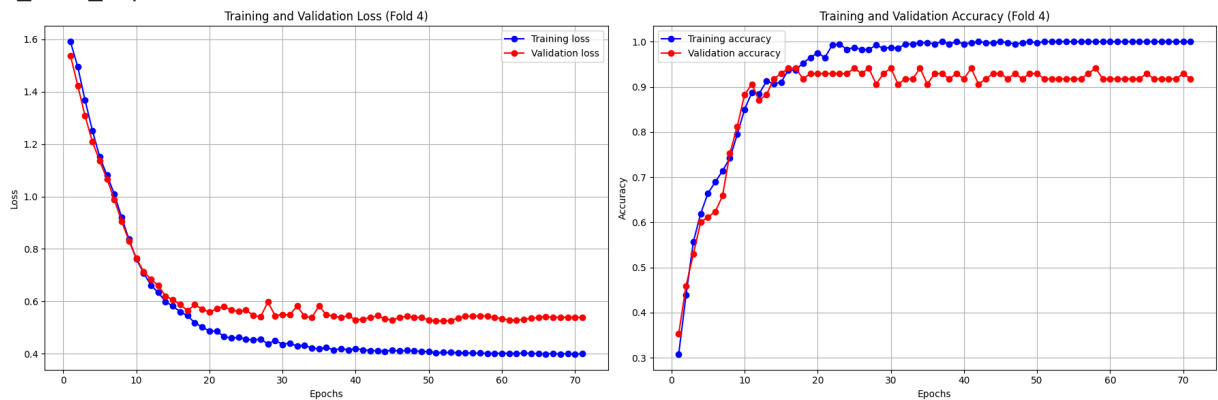
Epoch 1/200	Train 1.5913/0.3083	Val 1.5386/0.3529	LR 2.00e-05
Epoch 2/200	Train 1.4950/0.4386	Val 1.4236/0.4588	LR 3.00e-05
Epoch 3/200	Train 1.3675/0.5564	Val 1.3074/0.5294	LR 4.00e-05
Epoch 4/200	Train 1.2523/0.6190	Val 1.2097/0.6000	LR 5.00e-05
Epoch 5/200	Train 1.1510/0.6642	Val 1.1354/0.6118	LR 6.00e-05
Epoch 6/200	Train 1.0807/0.6892	Val 1.0669/0.6235	LR 7.00e-05
Epoch 7/200	Train 1.0094/0.7143	Val 0.9879/0.6588	LR 8.00e-05
Epoch 8/200	Train 0.9199/0.7419	Val 0.9057/0.7529	LR 9.00e-05
Epoch 9/200	Train 0.8364/0.7945	Val 0.8305/0.8118	LR 1.00e-04
Epoch 10/200	Train 0.7619/0.8496	Val 0.7651/0.8824	LR 1.00e-04
Epoch 11/200	Train 0.7083/0.8872	Val 0.7132/0.9059	LR 1.00e-04
Epoch 12/200	Train 0.6613/0.8847	Val 0.6844/0.8706	LR 1.00e-04
Epoch 13/200	Train 0.6332/0.9123	Val 0.6598/0.8824	LR 9.99e-05
Epoch 14/200	Train 0.5985/0.9073	Val 0.6195/0.9176	LR 9.99e-05
Epoch 15/200	Train 0.5811/0.9098	Val 0.6052/0.9294	LR 9.98e-05
Epoch 16/200	Train 0.5597/0.9373	Val 0.5879/0.9412	LR 9.98e-05
Epoch 17/200	Train 0.5447/0.9373	Val 0.5639/0.9412	LR 9.97e-05
Epoch 18/200	Train 0.5177/0.9524	Val 0.5866/0.9176	LR 9.96e-05
Epoch 19/200	Train 0.5019/0.9649	Val 0.5705/0.9294	LR 9.94e-05
Epoch 20/200	Train 0.4860/0.9749	Val 0.5580/0.9294	LR 9.93e-05
Epoch 21/200	Train 0.4864/0.9649	Val 0.5712/0.9294	LR 9.92e-05
Epoch 22/200	Train 0.4641/0.9925	Val 0.5792/0.9294	LR 9.90e-05
Epoch 23/200	Train 0.4594/0.9950	Val 0.5663/0.9294	LR 9.88e-05
Epoch 24/200	Train 0.4614/0.9825	Val 0.5607/0.9294	LR 9.87e-05
Epoch 25/200	Train 0.4560/0.9875	Val 0.5664/0.9412	LR 9.85e-05
Epoch 26/200	Train 0.4511/0.9825	Val 0.5458/0.9294	LR 9.83e-05
Epoch 27/200	Train 0.4560/0.9825	Val 0.5404/0.9412	LR 9.80e-05
Epoch 28/200	Train 0.4368/0.9925	Val 0.5967/0.9059	LR 9.78e-05
Epoch 29/200	Train 0.4503/0.9850	Val 0.5444/0.9294	LR 9.76e-05
Epoch 30/200	Train 0.4353/0.9875	Val 0.5489/0.9412	LR 9.73e-05
Epoch 31/200	Train 0.4400/0.9850	Val 0.5480/0.9059	LR 9.70e-05
Epoch 32/200	Train 0.4283/0.9950	Val 0.5820/0.9176	LR 9.67e-05
Epoch 33/200	Train 0.4315/0.9950	Val 0.5422/0.9176	LR 9.64e-05
Epoch 34/200	Train 0.4211/0.9975	Val 0.5374/0.9412	LR 9.61e-05
Epoch 35/200	Train 0.4185/0.9975	Val 0.5822/0.9059	LR 9.58e-05
Epoch 36/200	Train 0.4244/0.9950	Val 0.5479/0.9294	LR 9.55e-05
Epoch 37/200	Train 0.4138/1.0000	Val 0.5432/0.9294	LR 9.51e-05
Epoch 38/200	Train 0.4189/0.9950	Val 0.5374/0.9176	LR 9.47e-05
Epoch 39/200	Train 0.4142/1.0000	Val 0.5454/0.9294	LR 9.44e-05
Epoch 40/200	Train 0.4186/0.9950	Val 0.5285/0.9176	LR 9.40e-05
Epoch 41/200	Train 0.4144/0.9975	Val 0.5312/0.9412	LR 9.36e-05
Epoch 42/200	Train 0.4112/1.0000	Val 0.5378/0.9059	LR 9.32e-05
Epoch 43/200	Train 0.4112/0.9975	Val 0.5448/0.9176	LR 9.27e-05
Epoch 44/200	Train 0.4089/0.9975	Val 0.5319/0.9294	LR 9.23e-05
Epoch 45/200	Train 0.4128/1.0000	Val 0.5285/0.9294	LR 9.19e-05

Epoch 46/200	Train 0.4094/0.9975	Val 0.5382/0.9176	LR 9.14e-05
Epoch 47/200	Train 0.4120/0.9950	Val 0.5422/0.9294	LR 9.09e-05
Epoch 48/200	Train 0.4100/0.9975	Val 0.5392/0.9176	LR 9.05e-05
Epoch 49/200	Train 0.4086/1.0000	Val 0.5377/0.9294	LR 9.00e-05
Epoch 50/200	Train 0.4072/0.9975	Val 0.5284/0.9294	LR 8.95e-05
Epoch 51/200	Train 0.4035/1.0000	Val 0.5246/0.9176	LR 8.89e-05
Epoch 52/200	Train 0.4044/1.0000	Val 0.5251/0.9176	LR 8.84e-05
Epoch 53/200	Train 0.4046/1.0000	Val 0.5263/0.9176	LR 8.79e-05
Epoch 54/200	Train 0.4035/1.0000	Val 0.5352/0.9176	LR 8.73e-05
Epoch 55/200	Train 0.4021/1.0000	Val 0.5434/0.9176	LR 8.68e-05
Epoch 56/200	Train 0.4026/1.0000	Val 0.5432/0.9176	LR 8.62e-05
Epoch 57/200	Train 0.4017/1.0000	Val 0.5445/0.9294	LR 8.56e-05
Epoch 58/200	Train 0.4011/1.0000	Val 0.5433/0.9412	LR 8.51e-05
Epoch 59/200	Train 0.4010/1.0000	Val 0.5386/0.9176	LR 8.45e-05
Epoch 60/200	Train 0.4014/1.0000	Val 0.5330/0.9176	LR 8.39e-05
Epoch 61/200	Train 0.4009/1.0000	Val 0.5280/0.9176	LR 8.33e-05
Epoch 62/200	Train 0.4009/1.0000	Val 0.5274/0.9176	LR 8.26e-05
Epoch 63/200	Train 0.4015/1.0000	Val 0.5306/0.9176	LR 8.20e-05
Epoch 64/200	Train 0.4011/1.0000	Val 0.5346/0.9176	LR 8.14e-05
Epoch 65/200	Train 0.3996/1.0000	Val 0.5380/0.9294	LR 8.07e-05
Epoch 66/200	Train 0.3987/1.0000	Val 0.5411/0.9176	LR 8.01e-05
Epoch 67/200	Train 0.4003/1.0000	Val 0.5386/0.9176	LR 7.94e-05
Epoch 68/200	Train 0.3987/1.0000	Val 0.5389/0.9176	LR 7.87e-05
Epoch 69/200	Train 0.3993/1.0000	Val 0.5391/0.9176	LR 7.80e-05
Epoch 70/200	Train 0.3981/1.0000	Val 0.5389/0.9294	LR 7.73e-05
Epoch 71/200	Train 0.3990/1.0000	Val 0.5381/0.9176	LR 7.67e-05

Early stopping.

Training complete in 5.4 minutes. Best Val Loss 0.5246

Best model for fold saved to: new_results_Multimodal_DSAF_KFold\best_multimodal_dsaf_fold_4.pth



Saved training history plot to: new_results_Multimodal_DSAF_KFold\training_history_fold_4.png

=====

OVERALL PERFORMANCE (FOLD 4)

=====

	precision	recall	f1-score	support
Herpes	0.67	0.67	0.67	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.95	0.95	0.95	19
Sarampion	0.85	0.93	0.89	30
Varicela	0.86	0.75	0.80	24
accuracy			0.86	81
macro avg	0.86	0.86	0.86	81
weighted avg	0.86	0.86	0.86	81

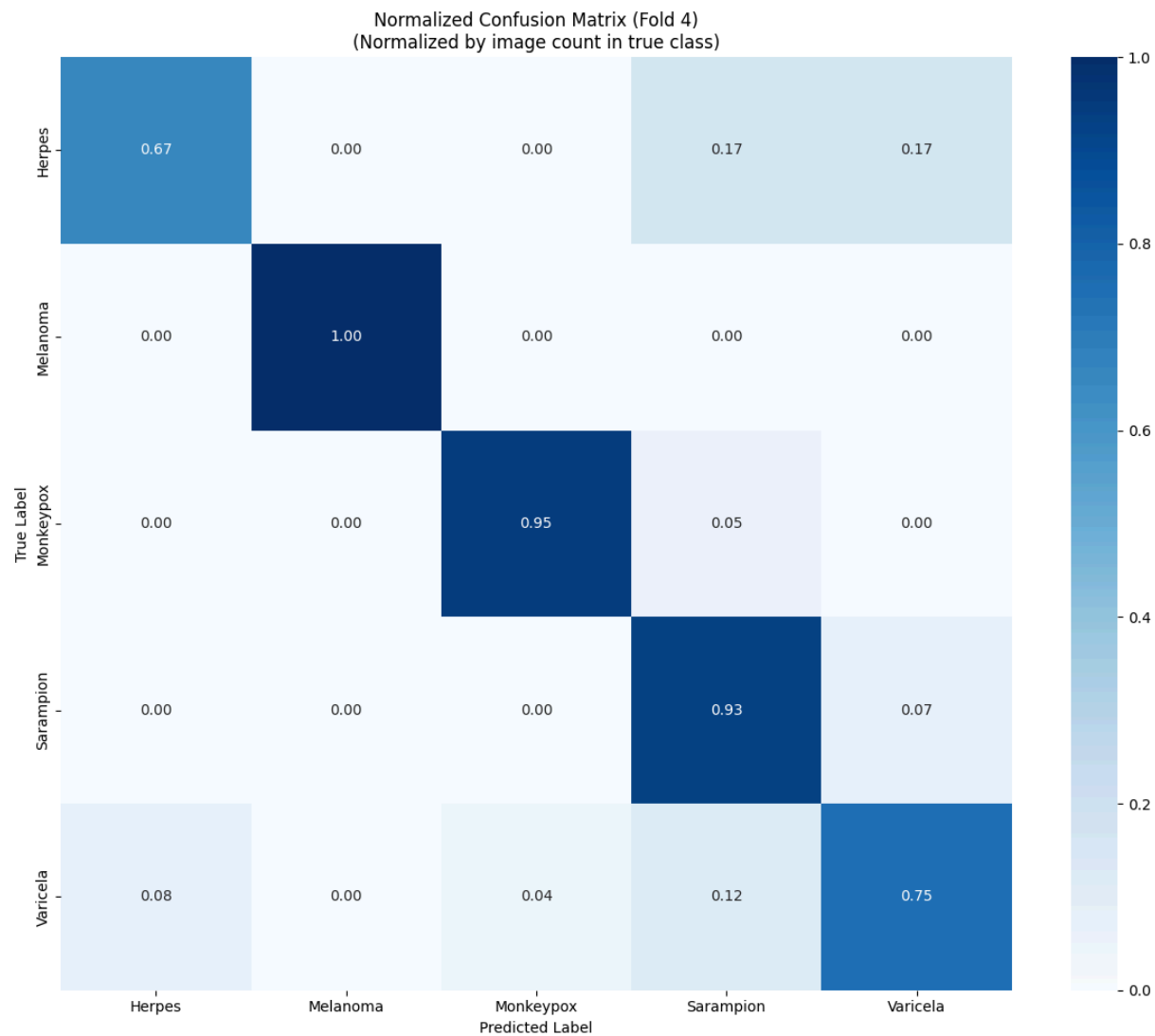
--- Overall Scalar Metrics ---

Accuracy : 0.8642

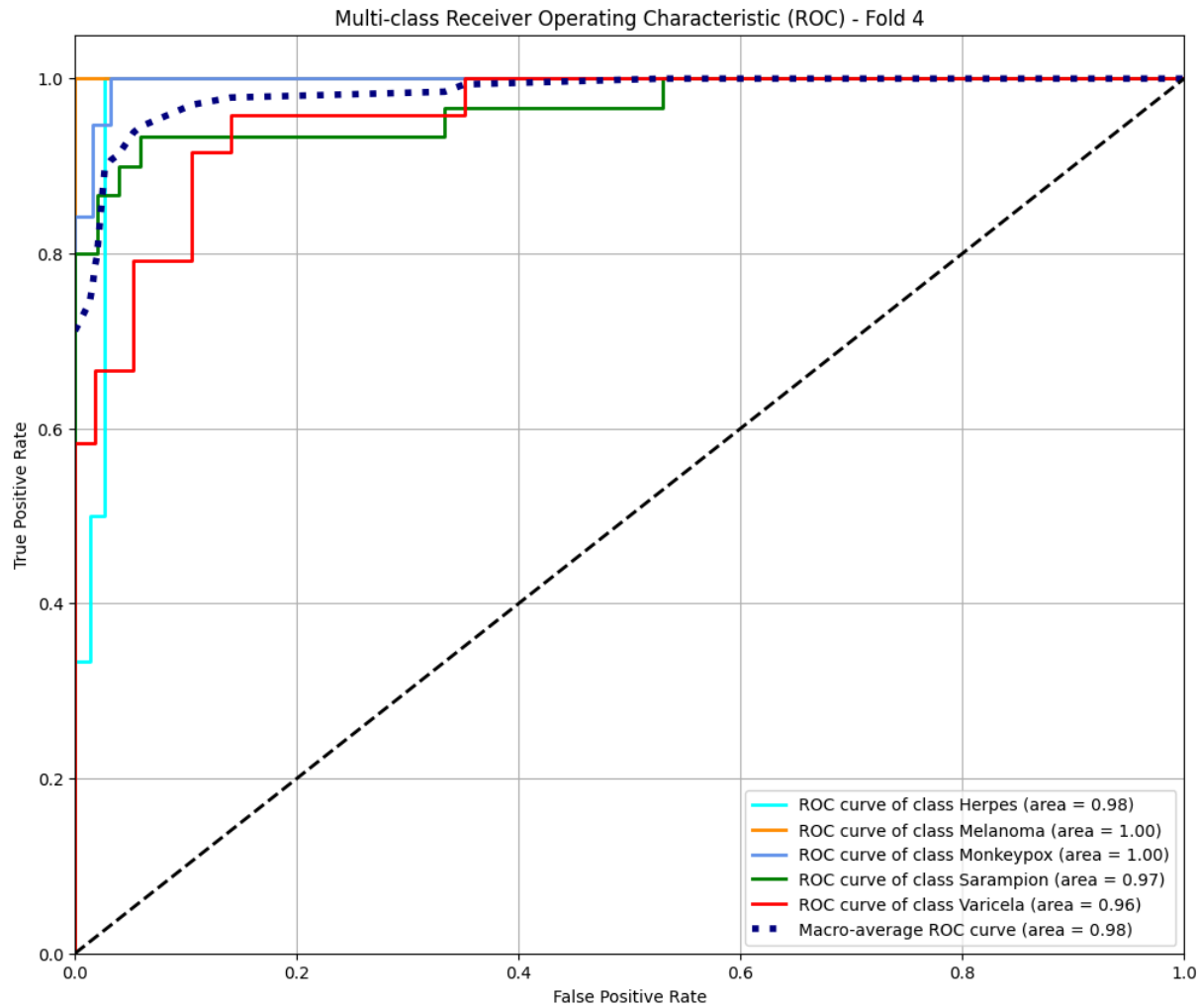
Macro ROC AUC : 0.9812

Cohen's Kappa : 0.8090

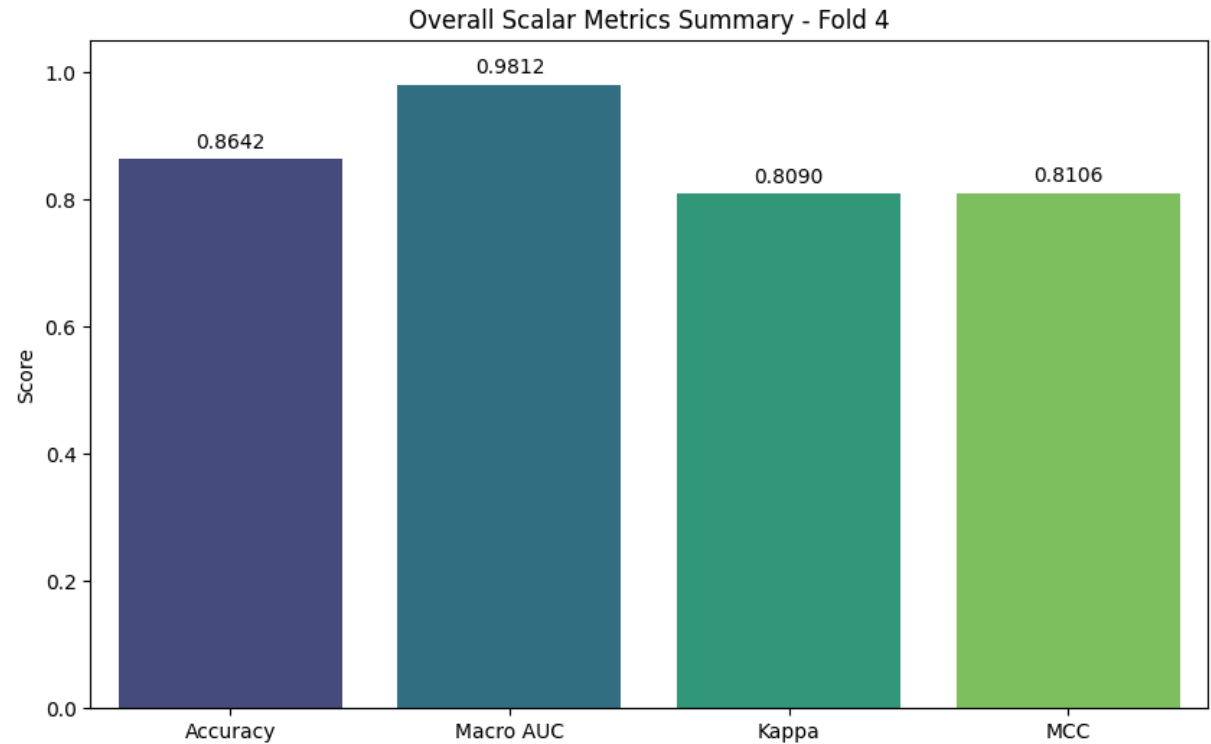
Matthews CorrCoef: 0.8106



Saved normalized confusion matrix to: new_results_Multimodal_DSAF_KFold\confusion_matrix_fold_4.png



Saved ROC curves plot to: new_results_Multimodal_DSAF_KFold\roc_curves_fold_4.png



Saved scalar metrics plot to: new_results_Multimodal_DSAF_KFold\scalar_metrics_fold_4.png

=====

PERFORMANCE ANALYSIS BY: DOMINANT_GENDER

=====

--- Results for Group: Man ---

	precision	recall	f1-score	support
Herpes	0.67	0.80	0.73	5
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	1.00	1.00	16
Sarampion	0.93	0.88	0.90	16
Varicela	0.77	0.77	0.77	13
accuracy			0.88	52
macro avg	0.87	0.89	0.88	52
weighted avg	0.89	0.88	0.89	52

--- Results for Group: Woman ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.67	0.67	0.67	3
Sarampion	0.78	1.00	0.88	14
Varicela	1.00	0.73	0.84	11
accuracy			0.83	29
macro avg	0.49	0.48	0.48	29
weighted avg	0.82	0.83	0.81	29

=====

PERFORMANCE ANALYSIS BY: AGE_GROUP

=====

--- Results for Group: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	0.33	0.50	0.40	2
Varicela	0.50	0.33	0.40	3
accuracy			0.40	5
macro avg	0.17	0.17	0.16	5
weighted avg	0.43	0.40	0.40	5

--- Results for Group: 18-59 ---

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Herpes	0.67	0.67	0.67	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.95	0.95	0.95	19
Sarampion	0.90	0.96	0.93	28
Varicela	0.89	0.81	0.85	21
accuracy			0.89	76
macro avg	0.88	0.88	0.88	76
weighted avg	0.89	0.89	0.89	76

=====

PERFORMANCE ANALYSIS BY: DOMINANT_RACE

=====

--- Results for Group: asian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	1.00	0.67	0.80	3
Varicela	0.67	0.80	0.73	5
accuracy			0.70	10
macro avg	0.53	0.49	0.51	10
weighted avg	0.73	0.70	0.70	10

--- Results for Group: black ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	1.00	1.00	1.00	2
Varicela	0.00	0.00	0.00	0
accuracy			1.00	5
macro avg	0.40	0.40	0.40	5
weighted avg	1.00	1.00	1.00	5

--- Results for Group: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	4
Varicela	1.00	1.00	1.00	1
accuracy			1.00	7
macro avg	0.60	0.60	0.60	7

weighted avg	1.00	1.00	1.00	7
--------------	------	------	------	---

--- Results for Group: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	0.67	0.80	3
Sarampion	0.80	1.00	0.89	4
Varicela	0.00	0.00	0.00	0
accuracy			0.86	7
macro avg	0.36	0.33	0.34	7
weighted avg	0.89	0.86	0.85	7

--- Results for Group: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	1.00	1.00	1.00	1
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			1.00	4
macro avg	0.60	0.60	0.60	4
weighted avg	1.00	1.00	1.00	4

--- Results for Group: white ---

	precision	recall	f1-score	support
Herpes	0.80	0.80	0.80	5
Melanoma	1.00	1.00	1.00	1
Monkeypox	0.89	1.00	0.94	8
Sarampion	0.79	0.94	0.86	16
Varicela	0.93	0.72	0.81	18
accuracy			0.85	48
macro avg	0.88	0.89	0.88	48
weighted avg	0.86	0.85	0.85	48

```
=====
FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.0000 EOD (max TPR gap): 1.0000
Worst pair: 0-17|female|white vs 18-59|female|asian
=====
```

```
=====
FAIRNESS SUMMARY (FILTERED, n >= 15) | WGA: 0.8421 EOD (max TPR gap): 0.4907
Worst pair (among filtered): 18-59|female|white vs 18-59|male|white
Groups used (filtered): ['18-59|female|white', '18-59|male|white']
=====
```

```
#####  
####  
### FOLD 5 ### Train=399, Val=85, Test=81  
#####  
####  
Params: 91.58M | Trainable: 5.92M (6.47%)  
Approx MACs: 17.68 G, Params (THOP): 89.20 M
```

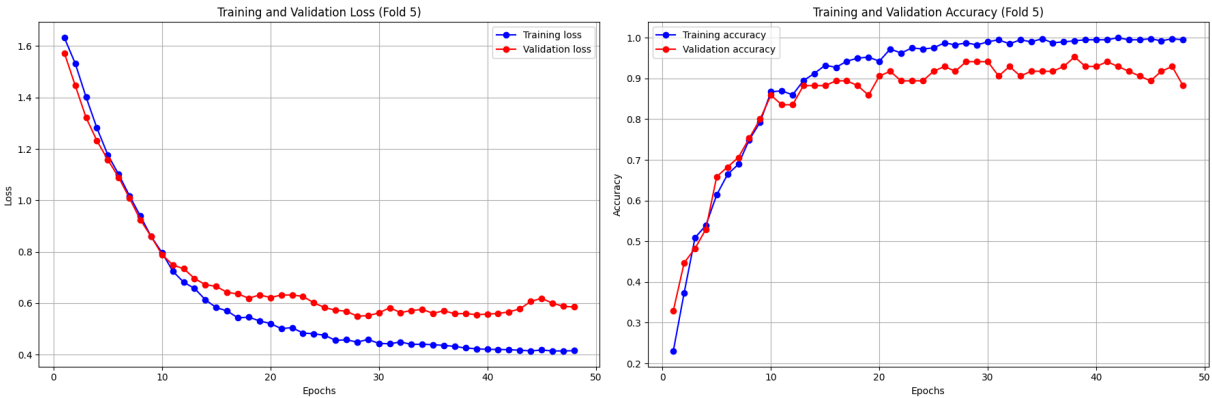
```
=====
```

START TRAINING (Multimodal DSAF) for 200 epochs

```
=====
```

Epoch 1/200	Train 1.6325/0.2306	Val 1.5729/0.3294	LR 2.00e-05
Epoch 2/200	Train 1.5326/0.3734	Val 1.4468/0.4471	LR 3.00e-05
Epoch 3/200	Train 1.4017/0.5088	Val 1.3221/0.4824	LR 4.00e-05
Epoch 4/200	Train 1.2813/0.5388	Val 1.2320/0.5294	LR 5.00e-05
Epoch 5/200	Train 1.1774/0.6140	Val 1.1584/0.6588	LR 6.00e-05
Epoch 6/200	Train 1.1013/0.6642	Val 1.0897/0.6824	LR 7.00e-05
Epoch 7/200	Train 1.0180/0.6892	Val 1.0090/0.7059	LR 8.00e-05
Epoch 8/200	Train 0.9393/0.7494	Val 0.9246/0.7529	LR 9.00e-05
Epoch 9/200	Train 0.8593/0.7920	Val 0.8594/0.8000	LR 1.00e-04
Epoch 10/200	Train 0.7969/0.8672	Val 0.7894/0.8588	LR 1.00e-04
Epoch 11/200	Train 0.7243/0.8697	Val 0.7485/0.8353	LR 1.00e-04
Epoch 12/200	Train 0.6816/0.8596	Val 0.7344/0.8353	LR 1.00e-04
Epoch 13/200	Train 0.6577/0.8947	Val 0.6960/0.8824	LR 9.99e-05
Epoch 14/200	Train 0.6134/0.9123	Val 0.6719/0.8824	LR 9.99e-05
Epoch 15/200	Train 0.5830/0.9323	Val 0.6655/0.8824	LR 9.98e-05
Epoch 16/200	Train 0.5700/0.9273	Val 0.6420/0.8941	LR 9.98e-05
Epoch 17/200	Train 0.5431/0.9424	Val 0.6363/0.8941	LR 9.97e-05
Epoch 18/200	Train 0.5457/0.9499	Val 0.6193/0.8824	LR 9.96e-05
Epoch 19/200	Train 0.5310/0.9524	Val 0.6320/0.8588	LR 9.94e-05
Epoch 20/200	Train 0.5209/0.9424	Val 0.6221/0.9059	LR 9.93e-05
Epoch 21/200	Train 0.5011/0.9724	Val 0.6317/0.9176	LR 9.92e-05
Epoch 22/200	Train 0.5049/0.9624	Val 0.6321/0.8941	LR 9.90e-05
Epoch 23/200	Train 0.4837/0.9749	Val 0.6263/0.8941	LR 9.88e-05
Epoch 24/200	Train 0.4812/0.9724	Val 0.6016/0.8941	LR 9.87e-05
Epoch 25/200	Train 0.4752/0.9749	Val 0.5828/0.9176	LR 9.85e-05
Epoch 26/200	Train 0.4554/0.9875	Val 0.5729/0.9294	LR 9.83e-05
Epoch 27/200	Train 0.4576/0.9825	Val 0.5687/0.9176	LR 9.80e-05
Epoch 28/200	Train 0.4488/0.9875	Val 0.5496/0.9412	LR 9.78e-05
Epoch 29/200	Train 0.4589/0.9825	Val 0.5516/0.9412	LR 9.76e-05
Epoch 30/200	Train 0.4437/0.9900	Val 0.5620/0.9412	LR 9.73e-05
Epoch 31/200	Train 0.4422/0.9950	Val 0.5824/0.9059	LR 9.70e-05
Epoch 32/200	Train 0.4492/0.9850	Val 0.5635/0.9294	LR 9.67e-05
Epoch 33/200	Train 0.4398/0.9950	Val 0.5717/0.9059	LR 9.64e-05
Epoch 34/200	Train 0.4404/0.9900	Val 0.5756/0.9176	LR 9.61e-05
Epoch 35/200	Train 0.4386/0.9975	Val 0.5607/0.9176	LR 9.58e-05
Epoch 36/200	Train 0.4359/0.9875	Val 0.5707/0.9176	LR 9.55e-05
Epoch 37/200	Train 0.4320/0.9900	Val 0.5586/0.9294	LR 9.51e-05
Epoch 38/200	Train 0.4257/0.9925	Val 0.5601/0.9529	LR 9.47e-05
Epoch 39/200	Train 0.4227/0.9950	Val 0.5550/0.9294	LR 9.44e-05
Epoch 40/200	Train 0.4204/0.9950	Val 0.5578/0.9294	LR 9.40e-05
Epoch 41/200	Train 0.4204/0.9950	Val 0.5598/0.9412	LR 9.36e-05
Epoch 42/200	Train 0.4191/1.0000	Val 0.5664/0.9294	LR 9.32e-05
Epoch 43/200	Train 0.4172/0.9950	Val 0.5779/0.9176	LR 9.27e-05
Epoch 44/200	Train 0.4143/0.9950	Val 0.6067/0.9059	LR 9.23e-05

```
Epoch 45/200 | Train 0.4178/0.9975 | Val 0.6190/0.8941 | LR 9.19e-05
Epoch 46/200 | Train 0.4144/0.9925 | Val 0.6003/0.9176 | LR 9.14e-05
Epoch 47/200 | Train 0.4143/0.9975 | Val 0.5878/0.9294 | LR 9.09e-05
Epoch 48/200 | Train 0.4152/0.9950 | Val 0.5851/0.8824 | LR 9.05e-05
Early stopping.
Training complete in 3.6 minutes. Best Val Loss 0.5496
Best model for fold saved to: new_results_Multimodal_DSAF_KFold\best_multimodal_dsaf_fold_5.pth
```



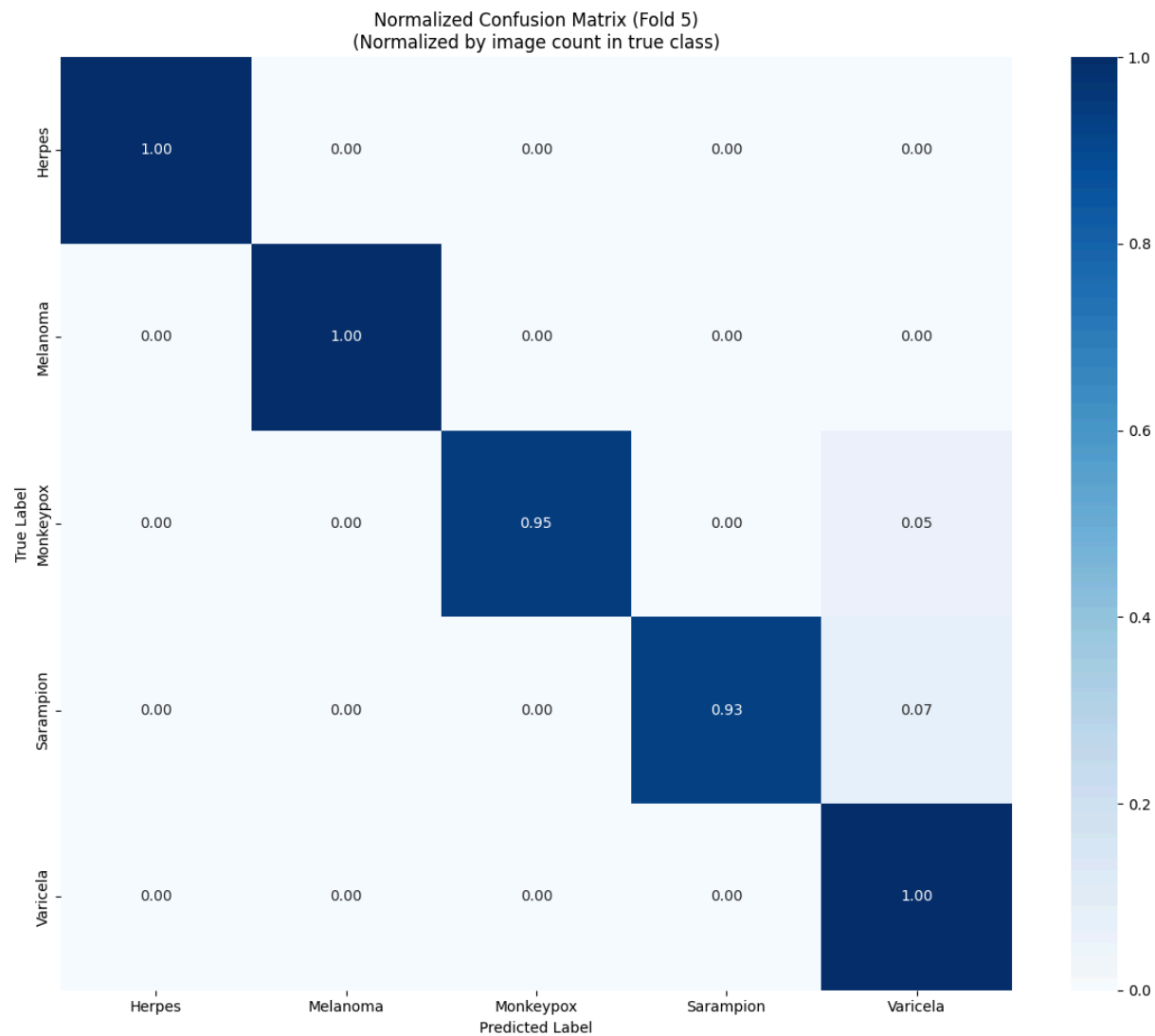
Saved training history plot to: new_results_Multimodal_DSAF_KFold\training_history_fold_5.png

```
=====
OVERALL PERFORMANCE (FOLD 5)
=====
```

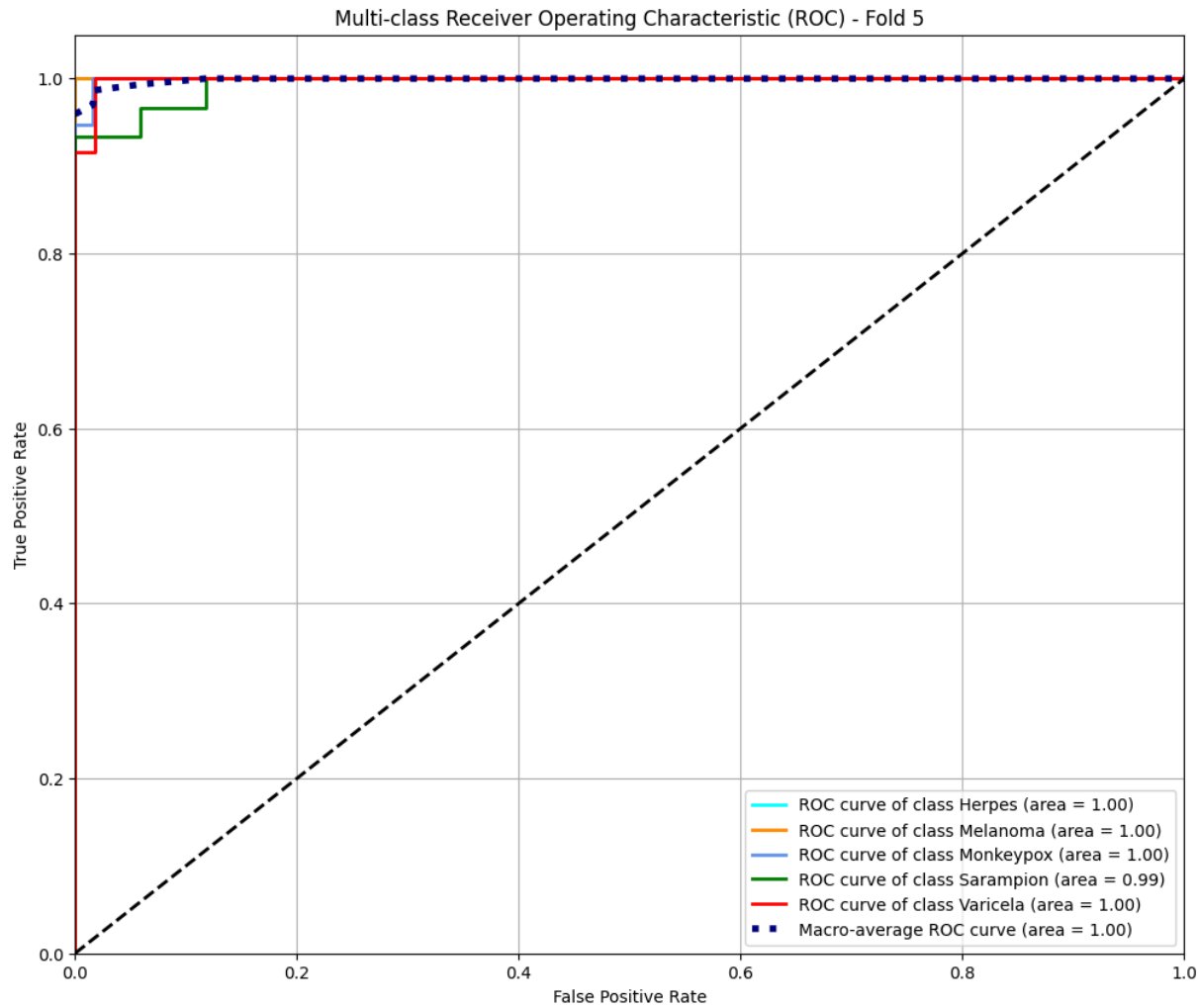
	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	0.95	0.97	19
Sarampion	1.00	0.93	0.97	30
Varicela	0.89	1.00	0.94	24
accuracy			0.96	81
macro avg	0.98	0.98	0.98	81
weighted avg	0.97	0.96	0.96	81

--- Overall Scalar Metrics ---

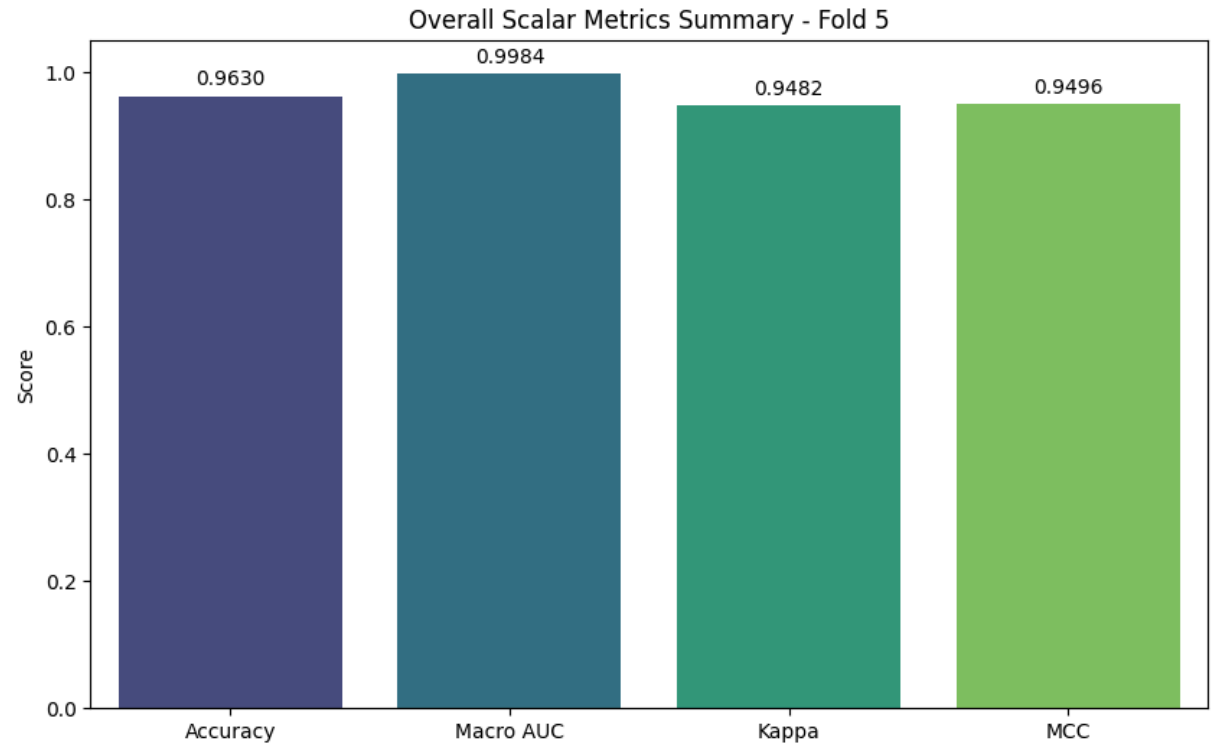
```
Accuracy          : 0.9630
Macro ROC AUC     : 0.9984
Cohen's Kappa     : 0.9482
Matthews CorrCoef: 0.9496
```



Saved normalized confusion matrix to: new_results_Multimodal_DSAF_KFold\confusion_matrix_fold_5.png



Saved ROC curves plot to: new_results_Multimodal_DSAF_KFold\roc_curves_fold_5.png



Saved scalar metrics plot to: new_results_Multimodal_DSAF_KFold\scalar_metrics_fold_5.png

=====

PERFORMANCE ANALYSIS BY: DOMINANT_GENDER

=====

--- Results for Group: Man ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	4
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	0.93	0.97	15
Sarampion	1.00	0.93	0.97	15
Varicela	0.89	1.00	0.94	17
accuracy			0.96	53
macro avg	0.98	0.97	0.98	53
weighted avg	0.97	0.96	0.96	53

--- Results for Group: Woman ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	2
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	4
Sarampion	1.00	0.93	0.97	15
Varicela	0.88	1.00	0.93	7
accuracy			0.96	28
macro avg	0.78	0.79	0.78	28
weighted avg	0.97	0.96	0.96	28

=====

PERFORMANCE ANALYSIS BY: AGE_GROUP

=====

--- Results for Group: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	1.00	1.00	1.00	1
Varicela	1.00	1.00	1.00	5
accuracy			1.00	6
macro avg	0.40	0.40	0.40	6
weighted avg	1.00	1.00	1.00	6

--- Results for Group: 18-59 ---

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Herpes	1.00	1.00	1.00	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	0.95	0.97	19
Sarampion	1.00	0.93	0.96	29
Varicela	0.86	1.00	0.93	19
accuracy			0.96	75
macro avg	0.97	0.98	0.97	75
weighted avg	0.97	0.96	0.96	75

=====

PERFORMANCE ANALYSIS BY: DOMINANT_RACE

=====

--- Results for Group: asian ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	1.00	0.50	0.67	4
Varicela	0.60	1.00	0.75	3
accuracy			0.78	9
macro avg	0.72	0.70	0.68	9
weighted avg	0.87	0.78	0.77	9

--- Results for Group: black ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	4
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			1.00	6
macro avg	0.60	0.60	0.60	6
weighted avg	1.00	1.00	1.00	6

--- Results for Group: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	1.00	1.00	1.00	4
Varicela	1.00	1.00	1.00	3
accuracy			1.00	10
macro avg	0.60	0.60	0.60	10

weighted avg	1.00	1.00	1.00	10
--------------	------	------	------	----

--- Results for Group: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	7
Varicela	0.00	0.00	0.00	0
accuracy			1.00	9
macro avg	0.40	0.40	0.40	9
weighted avg	1.00	1.00	1.00	9

--- Results for Group: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			1.00	2
macro avg	0.40	0.40	0.40	2
weighted avg	1.00	1.00	1.00	2

--- Results for Group: white ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	4
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	0.88	0.93	8
Sarampion	1.00	1.00	1.00	13
Varicela	0.95	1.00	0.97	18
accuracy			0.98	45
macro avg	0.99	0.97	0.98	45
weighted avg	0.98	0.98	0.98	45

```
=====
FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.7500 EOD (max TPR gap): 1.0000
Worst pair: 0-17|male|black vs 18-59|female|asian
=====
```

```
=====
FAIRNESS SUMMARY (FILTERED, n >= 15) | WGA: 0.9615 EOD (max TPR gap): 0.0000
Groups used (filtered): ['18-59|male|white']
=====
```

```
#####  
####  
### FOLD 6 ### Train=400, Val=85, Test=80  
#####  
####  
Params: 91.58M | Trainable: 5.92M (6.47%)  
Approx MACs: 17.68 G, Params (THOP): 89.20 M
```

```
=====
```

START TRAINING (Multimodal DSAF) for 200 epochs

```
=====
```

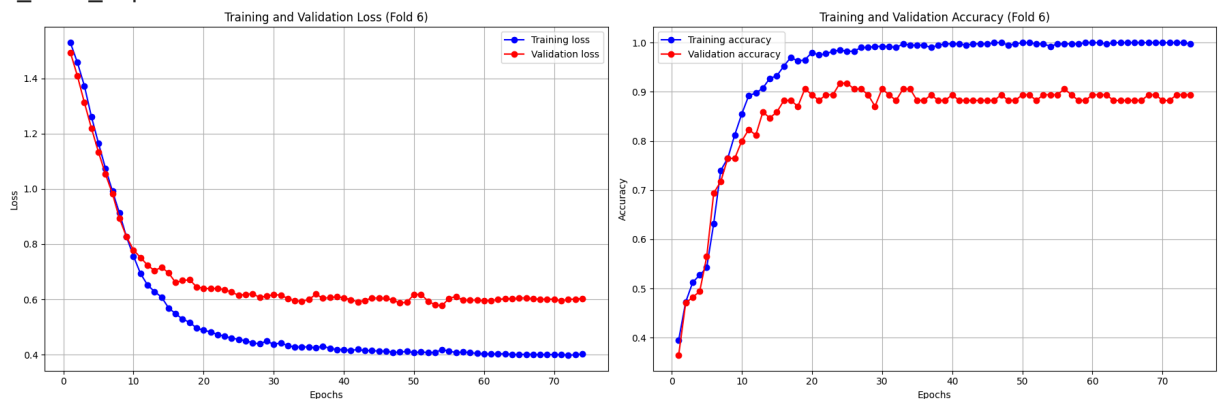
Epoch 1/200	Train 1.5296/0.3950	Val 1.4943/0.3647	LR 2.00e-05
Epoch 2/200	Train 1.4579/0.4725	Val 1.4094/0.4706	LR 3.00e-05
Epoch 3/200	Train 1.3732/0.5125	Val 1.3127/0.4824	LR 4.00e-05
Epoch 4/200	Train 1.2619/0.5275	Val 1.2202/0.4941	LR 5.00e-05
Epoch 5/200	Train 1.1662/0.5425	Val 1.1341/0.5647	LR 6.00e-05
Epoch 6/200	Train 1.0748/0.6325	Val 1.0538/0.6941	LR 7.00e-05
Epoch 7/200	Train 0.9922/0.7400	Val 0.9830/0.7176	LR 8.00e-05
Epoch 8/200	Train 0.9124/0.7650	Val 0.8928/0.7647	LR 9.00e-05
Epoch 9/200	Train 0.8265/0.8125	Val 0.8268/0.7647	LR 1.00e-04
Epoch 10/200	Train 0.7566/0.8550	Val 0.7789/0.8000	LR 1.00e-04
Epoch 11/200	Train 0.6936/0.8925	Val 0.7516/0.8235	LR 1.00e-04
Epoch 12/200	Train 0.6522/0.8975	Val 0.7240/0.8118	LR 1.00e-04
Epoch 13/200	Train 0.6281/0.9075	Val 0.7039/0.8588	LR 9.99e-05
Epoch 14/200	Train 0.6081/0.9275	Val 0.7163/0.8471	LR 9.99e-05
Epoch 15/200	Train 0.5691/0.9325	Val 0.6958/0.8588	LR 9.98e-05
Epoch 16/200	Train 0.5482/0.9525	Val 0.6612/0.8824	LR 9.98e-05
Epoch 17/200	Train 0.5282/0.9700	Val 0.6680/0.8824	LR 9.97e-05
Epoch 18/200	Train 0.5170/0.9625	Val 0.6710/0.8706	LR 9.96e-05
Epoch 19/200	Train 0.4959/0.9650	Val 0.6452/0.9059	LR 9.94e-05
Epoch 20/200	Train 0.4884/0.9800	Val 0.6404/0.8941	LR 9.93e-05
Epoch 21/200	Train 0.4818/0.9750	Val 0.6383/0.8824	LR 9.92e-05
Epoch 22/200	Train 0.4723/0.9775	Val 0.6403/0.8941	LR 9.90e-05
Epoch 23/200	Train 0.4667/0.9825	Val 0.6350/0.8941	LR 9.88e-05
Epoch 24/200	Train 0.4594/0.9850	Val 0.6262/0.9176	LR 9.87e-05
Epoch 25/200	Train 0.4552/0.9825	Val 0.6144/0.9176	LR 9.85e-05
Epoch 26/200	Train 0.4499/0.9825	Val 0.6181/0.9059	LR 9.83e-05
Epoch 27/200	Train 0.4429/0.9900	Val 0.6202/0.9059	LR 9.80e-05
Epoch 28/200	Train 0.4387/0.9900	Val 0.6070/0.8941	LR 9.78e-05
Epoch 29/200	Train 0.4494/0.9925	Val 0.6125/0.8706	LR 9.76e-05
Epoch 30/200	Train 0.4365/0.9925	Val 0.6164/0.9059	LR 9.73e-05
Epoch 31/200	Train 0.4430/0.9925	Val 0.6159/0.8941	LR 9.70e-05
Epoch 32/200	Train 0.4316/0.9900	Val 0.6028/0.8824	LR 9.67e-05
Epoch 33/200	Train 0.4274/0.9975	Val 0.5948/0.9059	LR 9.64e-05
Epoch 34/200	Train 0.4280/0.9950	Val 0.5931/0.9059	LR 9.61e-05
Epoch 35/200	Train 0.4278/0.9950	Val 0.5992/0.8824	LR 9.58e-05
Epoch 36/200	Train 0.4244/0.9950	Val 0.6186/0.8824	LR 9.55e-05
Epoch 37/200	Train 0.4309/0.9900	Val 0.6042/0.8941	LR 9.51e-05
Epoch 38/200	Train 0.4226/0.9950	Val 0.6062/0.8824	LR 9.47e-05
Epoch 39/200	Train 0.4185/0.9975	Val 0.6090/0.8824	LR 9.44e-05
Epoch 40/200	Train 0.4183/0.9975	Val 0.6059/0.8941	LR 9.40e-05
Epoch 41/200	Train 0.4146/0.9975	Val 0.5984/0.8824	LR 9.36e-05
Epoch 42/200	Train 0.4187/0.9950	Val 0.5912/0.8824	LR 9.32e-05
Epoch 43/200	Train 0.4138/0.9975	Val 0.5958/0.8824	LR 9.27e-05
Epoch 44/200	Train 0.4150/0.9975	Val 0.6056/0.8824	LR 9.23e-05
Epoch 45/200	Train 0.4122/0.9975	Val 0.6043/0.8824	LR 9.19e-05

Epoch 46/200	Train 0.4129/1.0000	Val 0.6046/0.8824	LR 9.14e-05
Epoch 47/200	Train 0.4079/1.0000	Val 0.5969/0.8941	LR 9.09e-05
Epoch 48/200	Train 0.4088/0.9950	Val 0.5865/0.8824	LR 9.05e-05
Epoch 49/200	Train 0.4115/0.9975	Val 0.5900/0.8824	LR 9.00e-05
Epoch 50/200	Train 0.4070/1.0000	Val 0.6177/0.8941	LR 8.95e-05
Epoch 51/200	Train 0.4095/1.0000	Val 0.6183/0.8941	LR 8.89e-05
Epoch 52/200	Train 0.4071/0.9975	Val 0.5935/0.8824	LR 8.84e-05
Epoch 53/200	Train 0.4079/0.9975	Val 0.5801/0.8941	LR 8.79e-05
Epoch 54/200	Train 0.4181/0.9925	Val 0.5769/0.8941	LR 8.73e-05
Epoch 55/200	Train 0.4120/0.9975	Val 0.6037/0.8941	LR 8.68e-05
Epoch 56/200	Train 0.4076/0.9975	Val 0.6102/0.9059	LR 8.62e-05
Epoch 57/200	Train 0.4089/0.9975	Val 0.5977/0.8941	LR 8.56e-05
Epoch 58/200	Train 0.4075/0.9975	Val 0.5975/0.8824	LR 8.51e-05
Epoch 59/200	Train 0.4039/1.0000	Val 0.5971/0.8824	LR 8.45e-05
Epoch 60/200	Train 0.4030/1.0000	Val 0.5957/0.8941	LR 8.39e-05
Epoch 61/200	Train 0.4027/1.0000	Val 0.5960/0.8941	LR 8.33e-05
Epoch 62/200	Train 0.4023/0.9975	Val 0.5996/0.8941	LR 8.26e-05
Epoch 63/200	Train 0.4033/1.0000	Val 0.6024/0.8824	LR 8.20e-05
Epoch 64/200	Train 0.4005/1.0000	Val 0.6031/0.8824	LR 8.14e-05
Epoch 65/200	Train 0.4000/1.0000	Val 0.6048/0.8824	LR 8.07e-05
Epoch 66/200	Train 0.4011/1.0000	Val 0.6053/0.8824	LR 8.01e-05
Epoch 67/200	Train 0.4008/1.0000	Val 0.6027/0.8824	LR 7.94e-05
Epoch 68/200	Train 0.3992/1.0000	Val 0.6000/0.8941	LR 7.87e-05
Epoch 69/200	Train 0.4008/1.0000	Val 0.6007/0.8941	LR 7.80e-05
Epoch 70/200	Train 0.3992/1.0000	Val 0.6006/0.8824	LR 7.73e-05
Epoch 71/200	Train 0.4000/1.0000	Val 0.5952/0.8824	LR 7.67e-05
Epoch 72/200	Train 0.3982/1.0000	Val 0.5995/0.8941	LR 7.59e-05
Epoch 73/200	Train 0.3990/1.0000	Val 0.6012/0.8941	LR 7.52e-05
Epoch 74/200	Train 0.4024/0.9975	Val 0.6027/0.8941	LR 7.45e-05

Early stopping.

Training complete in 5.5 minutes. Best Val Loss 0.5769

Best model for fold saved to: new_results_Multimodal_DSAF_KFold\best_multimodal_dsaf_fold_6.pth



Saved training history plot to: new_results_Multimodal_DSAF_KFold\training_history_fold_6.png

=====

OVERALL PERFORMANCE (FOLD 6)

=====

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	1.00	1.00	18
Sarampion	0.90	0.97	0.93	29
Varicela	0.96	0.88	0.92	25
accuracy			0.95	80
macro avg	0.97	0.97	0.97	80
weighted avg	0.95	0.95	0.95	80

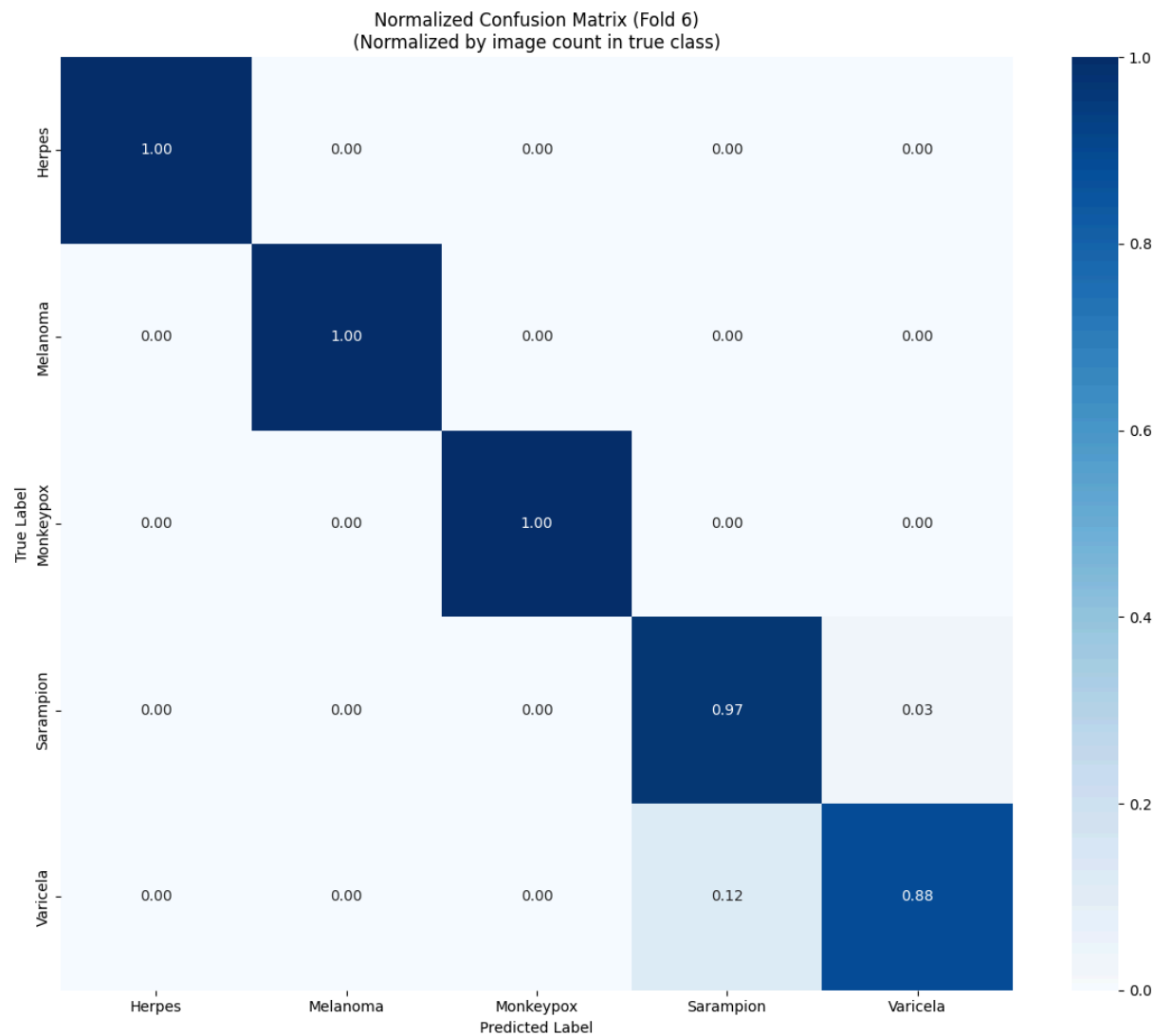
--- Overall Scalar Metrics ---

Accuracy : 0.9500

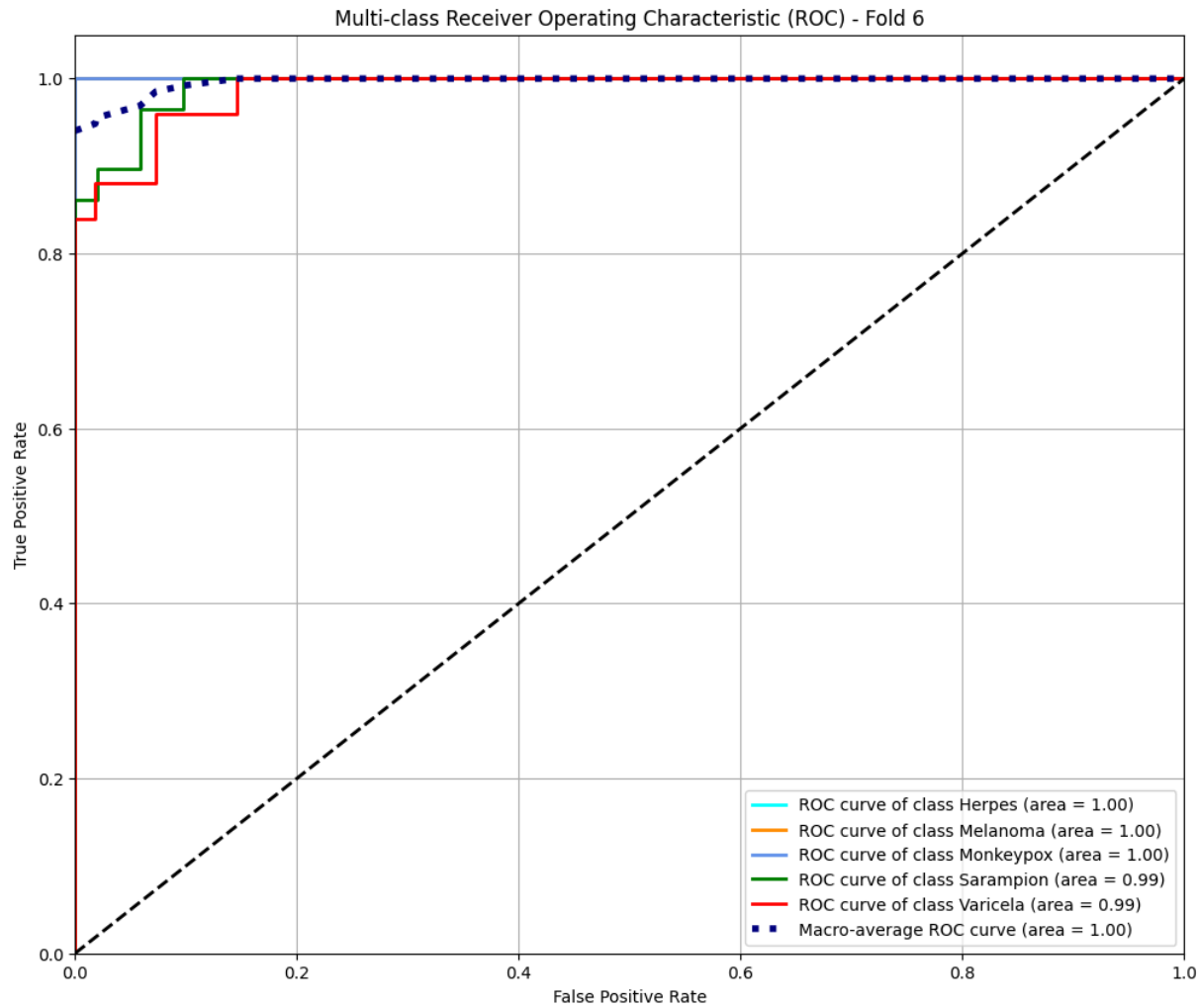
Macro ROC AUC : 0.9959

Cohen's Kappa : 0.9299

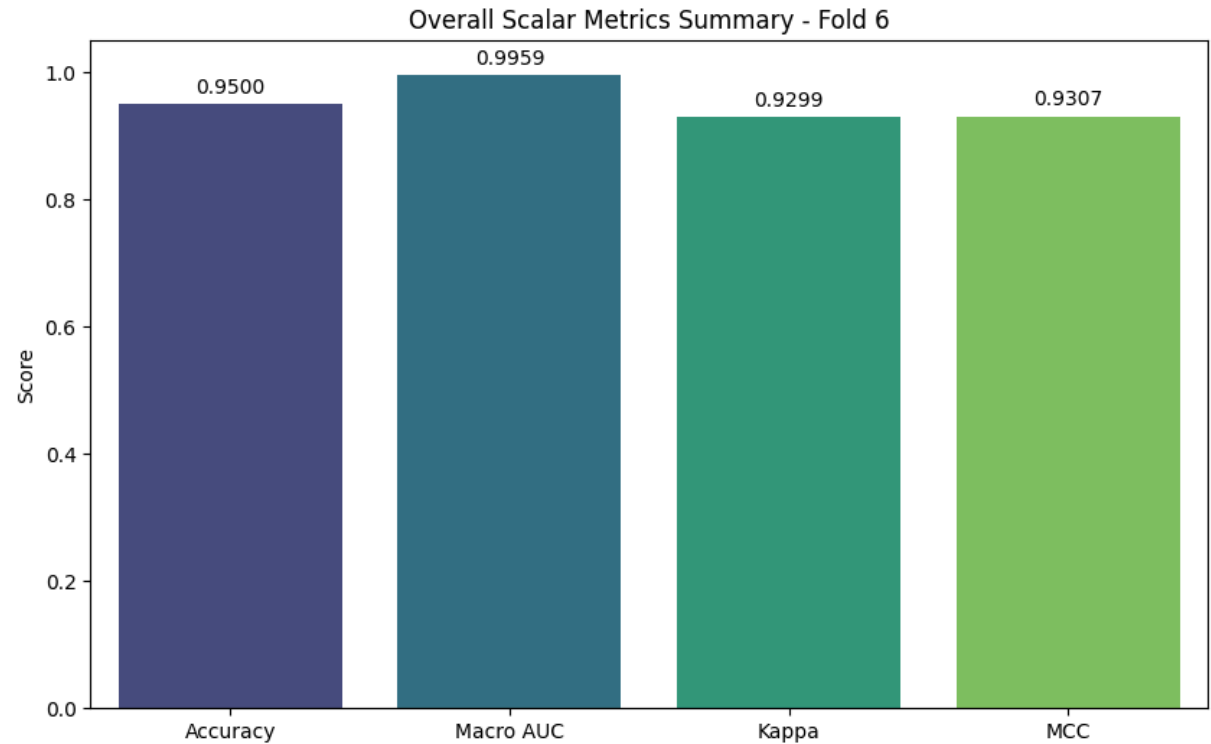
Matthews CorrCoef: 0.9307



Saved normalized confusion matrix to: new_results_Multimodal_DSAF_KFold\confusion_matrix_fold_6.png



Saved ROC curves plot to: new_results_Multimodal_DSAF_KFold\roc_curves_fold_6.png



Saved scalar metrics plot to: new_results_Multimodal_DSAF_KFold\scalar_metrics_fold_6.png

=====

PERFORMANCE ANALYSIS BY: DOMINANT_GENDER

=====

--- Results for Group: Man ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	5
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	1.00	1.00	16
Sarampion	0.90	0.95	0.92	19
Varicela	0.90	0.82	0.86	11
accuracy			0.94	53
macro avg	0.96	0.95	0.96	53
weighted avg	0.94	0.94	0.94	53

--- Results for Group: Woman ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	0.91	1.00	0.95	10
Varicela	1.00	0.93	0.96	14
accuracy			0.96	27
macro avg	0.78	0.79	0.78	27
weighted avg	0.97	0.96	0.96	27

=====

PERFORMANCE ANALYSIS BY: AGE_GROUP

=====

--- Results for Group: 0-17 ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	0.67	1.00	0.80	4
Varicela	1.00	0.60	0.75	5
accuracy			0.83	12
macro avg	0.73	0.72	0.71	12
weighted avg	0.89	0.83	0.83	12

--- Results for Group: 18-59 ---

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Herpes	1.00	1.00	1.00	4
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	1.00	1.00	16
Sarampion	0.96	0.96	0.96	25
Varicela	0.95	0.95	0.95	20
accuracy			0.97	67
macro avg	0.98	0.98	0.98	67
weighted avg	0.97	0.97	0.97	67

--- Results for Group: 60+ ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	1
macro avg	0.20	0.20	0.20	1
weighted avg	1.00	1.00	1.00	1

=====

PERFORMANCE ANALYSIS BY: DOMINANT_RACE

=====

--- Results for Group: asian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	0.50	1.00	0.67	1
Varicela	1.00	0.80	0.89	5
accuracy			0.86	7
macro avg	0.50	0.56	0.51	7
weighted avg	0.93	0.86	0.87	7

--- Results for Group: black ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			1.00	4
macro avg	0.40	0.40	0.40	4

weighted avg	1.00	1.00	1.00	4
--------------	------	------	------	---

--- Results for Group: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	7
Varicela	1.00	1.00	1.00	1
accuracy			1.00	10
macro avg	0.60	0.60	0.60	10
weighted avg	1.00	1.00	1.00	10

--- Results for Group: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	1.00	1.00	1.00	3
Varicela	0.00	0.00	0.00	0
accuracy			1.00	3
macro avg	0.20	0.20	0.20	3
weighted avg	1.00	1.00	1.00	3

--- Results for Group: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	1
macro avg	0.20	0.20	0.20	1
weighted avg	1.00	1.00	1.00	1

--- Results for Group: white ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	1.00	1.00	11
Sarampion	0.89	0.94	0.91	17
Varicela	0.94	0.89	0.92	19
accuracy			0.95	55
macro avg	0.97	0.97	0.97	55

weighted avg 0.95 0.95 0.95 55

=====

FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.5000 EOD (max TPR gap): 1.0000
Worst pair: 18-59|female|asian vs 18-59|male|asian

=====

=====

FAIRNESS SUMMARY (FILTERED, n >= 15) | WGA: 0.9655 EOD (max TPR gap): 0.0667
Worst pair (among filtered): 18-59|female|white vs 18-59|male|white
Groups used (filtered): ['18-59|female|white', '18-59|male|white']

=====

#####

####

FOLD 7 ### Train=400, Val=85, Test=80

#####

####

Params: 91.58M | Trainable: 5.92M (6.47%)
Approx MACs: 17.68 G, Params (THOP): 89.20 M

=====

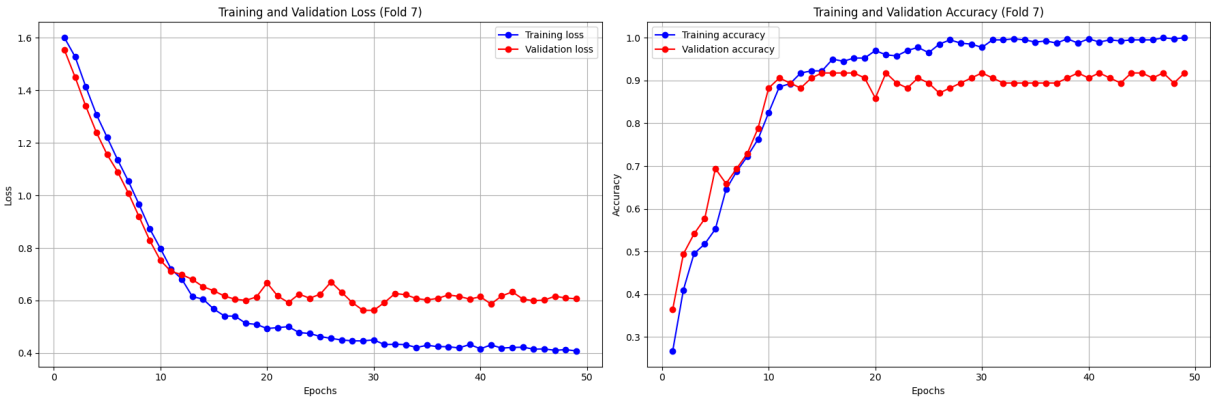
START TRAINING (Multimodal DSAF) for 200 epochs

=====

Epoch 1/200	Train 1.6004/0.2675	Val 1.5538/0.3647	LR 2.00e-05
Epoch 2/200	Train 1.5279/0.4100	Val 1.4505/0.4941	LR 3.00e-05
Epoch 3/200	Train 1.4141/0.4950	Val 1.3399/0.5412	LR 4.00e-05
Epoch 4/200	Train 1.3079/0.5175	Val 1.2398/0.5765	LR 5.00e-05
Epoch 5/200	Train 1.2214/0.5525	Val 1.1559/0.6941	LR 6.00e-05
Epoch 6/200	Train 1.1346/0.6450	Val 1.0893/0.6588	LR 7.00e-05
Epoch 7/200	Train 1.0539/0.6875	Val 1.0083/0.6941	LR 8.00e-05
Epoch 8/200	Train 0.9654/0.7225	Val 0.9190/0.7294	LR 9.00e-05
Epoch 9/200	Train 0.8729/0.7625	Val 0.8285/0.7882	LR 1.00e-04
Epoch 10/200	Train 0.7979/0.8250	Val 0.7527/0.8824	LR 1.00e-04
Epoch 11/200	Train 0.7201/0.8850	Val 0.7107/0.9059	LR 1.00e-04
Epoch 12/200	Train 0.6818/0.8925	Val 0.6988/0.8941	LR 1.00e-04
Epoch 13/200	Train 0.6148/0.9175	Val 0.6801/0.8824	LR 9.99e-05
Epoch 14/200	Train 0.6053/0.9225	Val 0.6526/0.9059	LR 9.99e-05
Epoch 15/200	Train 0.5676/0.9225	Val 0.6371/0.9176	LR 9.98e-05
Epoch 16/200	Train 0.5412/0.9500	Val 0.6169/0.9176	LR 9.98e-05
Epoch 17/200	Train 0.5406/0.9450	Val 0.6049/0.9176	LR 9.97e-05
Epoch 18/200	Train 0.5130/0.9525	Val 0.6006/0.9176	LR 9.96e-05
Epoch 19/200	Train 0.5092/0.9525	Val 0.6134/0.9059	LR 9.94e-05
Epoch 20/200	Train 0.4935/0.9700	Val 0.6670/0.8588	LR 9.93e-05
Epoch 21/200	Train 0.4969/0.9600	Val 0.6169/0.9176	LR 9.92e-05
Epoch 22/200	Train 0.5006/0.9575	Val 0.5926/0.8941	LR 9.90e-05
Epoch 23/200	Train 0.4781/0.9700	Val 0.6244/0.8824	LR 9.88e-05
Epoch 24/200	Train 0.4746/0.9775	Val 0.6087/0.9059	LR 9.87e-05
Epoch 25/200	Train 0.4628/0.9650	Val 0.6246/0.8941	LR 9.85e-05
Epoch 26/200	Train 0.4559/0.9850	Val 0.6709/0.8706	LR 9.83e-05
Epoch 27/200	Train 0.4498/0.9950	Val 0.6312/0.8824	LR 9.80e-05
Epoch 28/200	Train 0.4466/0.9875	Val 0.5920/0.8941	LR 9.78e-05
Epoch 29/200	Train 0.4462/0.9850	Val 0.5625/0.9059	LR 9.76e-05
Epoch 30/200	Train 0.4504/0.9775	Val 0.5626/0.9176	LR 9.73e-05

Epoch 31/200		Train 0.4323/0.9950		Val 0.5923/0.9059		LR 9.70e-05
Epoch 32/200		Train 0.4333/0.9950		Val 0.6266/0.8941		LR 9.67e-05
Epoch 33/200		Train 0.4322/0.9975		Val 0.6221/0.8941		LR 9.64e-05
Epoch 34/200		Train 0.4211/0.9950		Val 0.6076/0.8941		LR 9.61e-05
Epoch 35/200		Train 0.4297/0.9900		Val 0.6023/0.8941		LR 9.58e-05
Epoch 36/200		Train 0.4246/0.9925		Val 0.6082/0.8941		LR 9.55e-05
Epoch 37/200		Train 0.4242/0.9875		Val 0.6213/0.8941		LR 9.51e-05
Epoch 38/200		Train 0.4197/0.9975		Val 0.6157/0.9059		LR 9.47e-05
Epoch 39/200		Train 0.4326/0.9875		Val 0.6056/0.9176		LR 9.44e-05
Epoch 40/200		Train 0.4164/0.9975		Val 0.6143/0.9059		LR 9.40e-05
Epoch 41/200		Train 0.4309/0.9900		Val 0.5869/0.9176		LR 9.36e-05
Epoch 42/200		Train 0.4188/0.9950		Val 0.6175/0.9059		LR 9.32e-05
Epoch 43/200		Train 0.4215/0.9925		Val 0.6330/0.8941		LR 9.27e-05
Epoch 44/200		Train 0.4224/0.9950		Val 0.6051/0.9176		LR 9.23e-05
Epoch 45/200		Train 0.4149/0.9950		Val 0.5995/0.9176		LR 9.19e-05
Epoch 46/200		Train 0.4158/0.9950		Val 0.6017/0.9059		LR 9.14e-05
Epoch 47/200		Train 0.4106/1.0000		Val 0.6165/0.9176		LR 9.09e-05
Epoch 48/200		Train 0.4130/0.9975		Val 0.6095/0.8941		LR 9.05e-05
Epoch 49/200		Train 0.4081/1.0000		Val 0.6067/0.9176		LR 9.00e-05

Early stopping.
Training complete in 3.7 minutes. Best Val Loss 0.5625
Best model for fold saved to: new_results_Multimodal_DSAF_KFold\best_multimodal_dsaf_fold_7.pth



Saved training history plot to: new_results_Multimodal_DSAF_KFold\training_history_fold_7.png

=====

OVERALL PERFORMANCE (FOLD 7)

=====

	precision	recall	f1-score	support
Herpes	0.75	0.50	0.60	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.85	0.94	0.89	18
Sarampion	0.86	0.86	0.86	29
Varicela	0.88	0.88	0.88	25
accuracy			0.86	80
macro avg	0.87	0.84	0.85	80
weighted avg	0.86	0.86	0.86	80

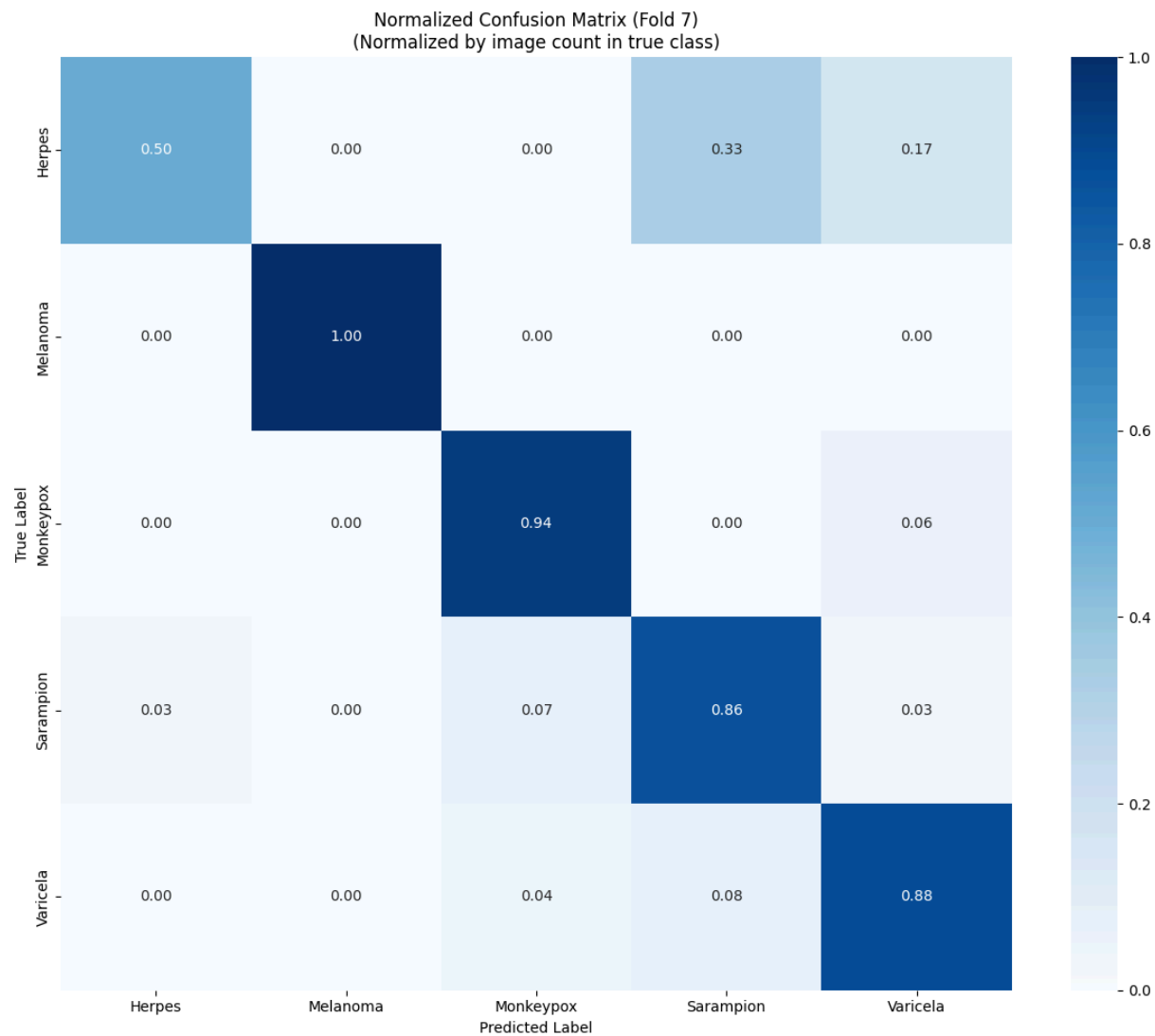
--- Overall Scalar Metrics ---

Accuracy : 0.8625

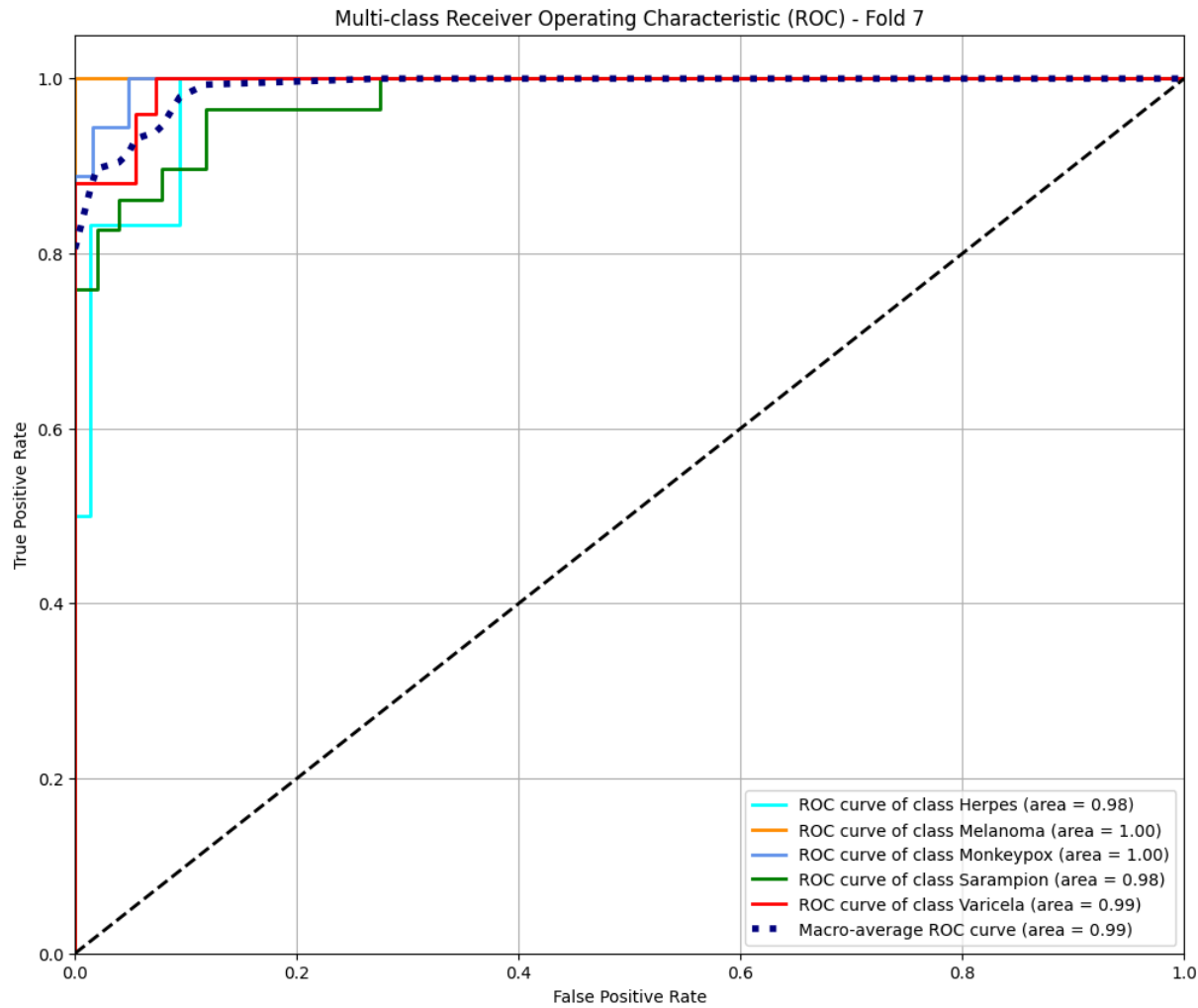
Macro ROC AUC : 0.9892

Cohen's Kappa : 0.8064

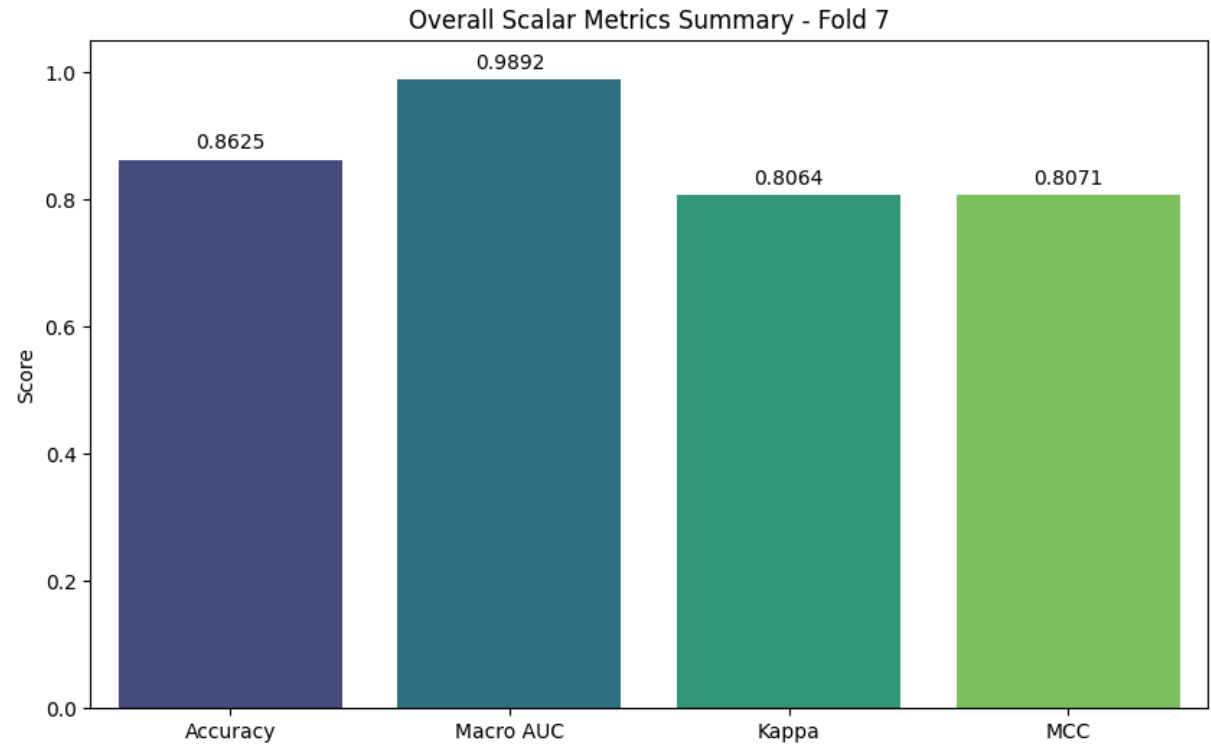
Matthews CorrCoef: 0.8071



Saved normalized confusion matrix to: new_results_Multimodal_DSAF_KFold\confusion_matrix_fold_7.png



Saved ROC curves plot to: new_results_Multimodal_DSAF_KFold\roc_curves_fold_7.png



Saved scalar metrics plot to: new_results_Multimodal_DSAF_KFold\scalar_metrics_fold_7.png

=====

PERFORMANCE ANALYSIS BY: DOMINANT_GENDER

=====

--- Results for Group: Man ---

	precision	recall	f1-score	support
Herpes	0.75	0.50	0.60	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.82	0.93	0.88	15
Sarampion	0.77	0.71	0.74	14
Varicela	0.79	0.85	0.81	13
accuracy			0.80	50
macro avg	0.83	0.80	0.81	50
weighted avg	0.80	0.80	0.79	50

--- Results for Group: Woman ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	0.94	1.00	0.97	15
Varicela	1.00	0.92	0.96	12
accuracy			0.97	30
macro avg	0.59	0.58	0.58	30
weighted avg	0.97	0.97	0.97	30

=====

PERFORMANCE ANALYSIS BY: AGE_GROUP

=====

--- Results for Group: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.67	1.00	0.80	2
Sarampion	1.00	0.67	0.80	3
Varicela	0.83	1.00	0.91	5
accuracy			0.82	11
macro avg	0.50	0.53	0.50	11
weighted avg	0.77	0.82	0.78	11

--- Results for Group: 18-59 ---

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Herpes	0.75	0.60	0.67	5
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.88	0.93	0.90	15
Sarampion	0.85	0.88	0.87	26
Varicela	0.89	0.85	0.87	20
accuracy			0.87	68
macro avg	0.87	0.85	0.86	68
weighted avg	0.87	0.87	0.87	68

--- Results for Group: 60+ ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	1
macro avg	0.20	0.20	0.20	1
weighted avg	1.00	1.00	1.00	1

=====

PERFORMANCE ANALYSIS BY: DOMINANT_RACE

=====

--- Results for Group: asian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	4
Sarampion	1.00	1.00	1.00	5
Varicela	1.00	1.00	1.00	4
accuracy			1.00	13
macro avg	0.60	0.60	0.60	13
weighted avg	1.00	1.00	1.00	13

--- Results for Group: black ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.83	1.00	0.91	5
Sarampion	0.33	0.50	0.40	2
Varicela	0.00	0.00	0.00	1
accuracy			0.67	9
macro avg	0.23	0.30	0.26	9

weighted avg	0.54	0.67	0.59	9
--------------	------	------	------	---

--- Results for Group: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	4
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			1.00	5
macro avg	0.40	0.40	0.40	5
weighted avg	1.00	1.00	1.00	5

--- Results for Group: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	3
Varicela	0.00	0.00	0.00	0
accuracy			1.00	5
macro avg	0.40	0.40	0.40	5
weighted avg	1.00	1.00	1.00	5

--- Results for Group: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	1.00	0.50	0.67	2
Varicela	0.00	0.00	0.00	0
accuracy			0.50	2
macro avg	0.20	0.10	0.13	2
weighted avg	1.00	0.50	0.67	2

--- Results for Group: white ---

	precision	recall	f1-score	support
Herpes	0.75	0.60	0.67	5
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.67	0.67	0.67	3
Sarampion	0.88	0.88	0.88	16
Varicela	0.86	0.90	0.88	20
accuracy			0.85	46
macro avg	0.83	0.81	0.82	46

weighted avg 0.85 0.85 0.85 46

```
=====
FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.5000 EOD (max TPR gap): 1.0000
Worst pair: 0-17|female|white vs 0-17|male|black
=====
```

```
=====
FAIRNESS SUMMARY (FILTERED, n >= 15) | WGA: 0.7222 EOD (max TPR gap): 0.5159
Worst pair (among filtered): 18-59|female|white vs 18-59|male|white
Groups used (filtered): ['18-59|female|white', '18-59|male|white']
=====
```

```
=====
====
Cross-validated over 7 folds (≈ Train 70%, Val 15%, Test 15%)
```

```
Overall accuracy : 0.920 ± 0.043
Macro AUC       : 0.990 ± 0.006
Kappa           : 0.889 ± 0.061
MCC             : 0.890 ± 0.061
WGA (all groups) : 0.336 ± 0.301
EOD (all groups) : 1.000 ± 0.000
WGA (n≥15)      : 0.902 ± 0.099
EOD (n≥15)      : 0.176 ± 0.214
=====
```

```
-----
POOLED SUBGROUP OVERALL ACCURACY – GENDER (all)
-----
```

Group	Acc	Count
male	0.915	355
female	0.929	210

```
-----
POOLED SUBGROUP OVERALL ACCURACY – GENDER (n≥15)
-----
```

Group	Acc	Count
male	0.915	355
female	0.929	210

```
-----
POOLED SUBGROUP OVERALL ACCURACY – AGE (all)
-----
```

Group	Acc	Count
18-59	0.932	501
0-17	0.820	61
60+	1.000	3

```
-----
POOLED SUBGROUP OVERALL ACCURACY – AGE (n≥15)
-----
```

Group	Acc	Count
18-59	0.932	501

0-17	0.820	61
------	-------	----

 POOLED SUBGROUP OVERALL ACCURACY – SKIN TONE (all)

Group	Acc	Count
white	0.923	338
asian	0.886	70
indian	0.940	50
latino hispanic	0.960	50
black	0.921	38
middle eastern	0.842	19

 POOLED SUBGROUP OVERALL ACCURACY – SKIN TONE (n≥15)

Group	Acc	Count
white	0.923	338
asian	0.886	70
indian	0.940	50
latino hispanic	0.960	50
black	0.921	38
middle eastern	0.842	19

 POOLED SUBGROUP OVERALL ACCURACY – COMPOSITE age|gender|skin (all)

Group	Acc	Count
18-59 male white	0.922	166
18-59 female white	0.952	126
18-59 male asian	0.872	39
18-59 male indian	0.971	34
0-17 male white	0.867	30
18-59 male black	0.933	30
18-59 female asian	0.962	26
18-59 female latino hispanic	0.920	25
18-59 male latino hispanic	1.000	23
18-59 male middle eastern	0.833	18
0-17 female white	0.800	15
18-59 female indian	0.923	13
0-17 male black	0.875	8
0-17 male asian	0.667	3
0-17 male indian	1.000	2
0-17 female asian	0.000	1
0-17 female indian	0.000	1
0-17 male latino hispanic	1.000	1
18-59 female middle eastern	1.000	1
60+ female asian	1.000	1
60+ female latino hispanic	1.000	1
60+ male white	1.000	1

 POOLED SUBGROUP OVERALL ACCURACY – COMPOSITE age|gender|skin (n≥15)

Group	Acc	Count
18-59 male white	0.922	166

18-59 female white	0.952	126
18-59 male asian	0.872	39
18-59 male indian	0.971	34
0-17 male white	0.867	30
18-59 male black	0.933	30
18-59 female asian	0.962	26
18-59 female latino hispanic	0.920	25
18-59 male latino hispanic	1.000	23
18-59 male middle eastern	0.833	18
0-17 female white	0.800	15

Saved pooled subgroup accuracy tables in: new_results_Multimodal_DSAF_KFold

Done.

In []: